COMP3711: Design and Analysis of Algorithms

WU Jinjun

October 31, 2026

Contents

1 Prerequisite: Asymptotic Notations **4**

2 Divide and Conquer **6**
2.1 Binary Search . . . 6
2.1.1 Variant: Find the Boundary (lower_bound) . . . 7
2.1.2 Exercise 1: Rotated Sorted Array . . . 8
2.1.3 Exercise 2: Finding the Last 0 . . . 9
2.2 Mergesort & Inversion Count . . . 11
2.2.1 Warm-up: Tower of Hanoi . . . 11
2.2.2 Merge: Combining Two Sorted Halves . . . 12
2.2.3 Mergesort Algorithm . . . 13
2.2.4 Counting Inversions . . . 14
2.3 Maximum Subarray . . . 16
2.3.1 Application: Maximizing Stock Profit . . . 18
2.4 Multiplication and Master Theorem . . . 18
2.4.1 Integer Multiplication (Karatsuba) . . . 18
2.4.2 First D&C Attempt: Four Half-Size Products . . . 19
2.4.3 Master Theorem for Recurrences . . . 20
2.5 Matrix Multiplication and Strassen's Algorithm . . . 21

3 Randomized Algorithms **23**
3.1 Why Randomization? . . . 23
3.2 Probability Review . . . 23
3.3 Linearity of Expectation . . . 23
3.4 Example: Hiring Problem . . . 24
3.5 More Indicator-Variable Examples . . . 24
3.5.1 Guessing Shuffled Cards . . . 24
3.5.2 Hash Table with Chaining . . . 24
3.5.3 Birthday Paradox . . . 25
3.6 Generating a Uniform Random Permutation . . . 25
3.7 Geometric Distribution and Coupon Collector . . . 25
3.8 Case Study: The Majority Element . . . 27

4 Quicksort **28**
4.1 Deterministic Quicksort (Lomuto Partition) . . . 28
4.2 Quicksort Analysis . . . 29
4.3 Randomized Selection (Quickselect) . . . 30
4.4 Hoare's Partition (Alternative) . . . 31

5 Heapsort **32**
5.1 Priority Queue Motivation . . . 32
5.2 Heap Data Structure . . . 32
5.3 Max-Heapify (Bubble-Down) . . . 33
5.4 Build Max-Heap . . . 33
5.5 Heapsort Algorithm . . . 35
5.6 Priority Queue Operations (Applications of Heaps) . . . 35
5.7 Applications . . . 37
5.7.1 Merging k Sorted Arrays . . . 37
5.7.2 Delete and Update-Priority . . . 37
5.7.3 The Skyline Problem . . . 38

6 Linear-Time Sorting **39**
6.1 The Decision-Tree Model and the Sorting Lower Bound . . . 39
6.2 Counting Sort . . . 40
6.3 Radix Sort . . . 41
6.4 Bucket Sort . . . 42

7 Greedy Algorithms **44**
7.1 Warm-up: Change-Making . . . 45
7.2 Warm-up: Max Sum of Non-Adjacent Elements . . . 45
7.3 Interval Scheduling . . . 45
7.3.1 Three Tempting Greedy Rules (All Fail) . . . 46
7.3.2 The Correct Rule: Earliest Finish Time . . . 46
7.3.3 Correctness Proof (Exchange Argument) . . . 46
7.4 Exercise: Interval Covering . . . 47
7.5 Fractional Knapsack . . . 47
7.6 Interval Partitioning . . . 48

8 Huffman Coding **51**
8.1 Motivation: Variable-Length Encoding . . . 51
8.2 Prefix-Free Codes . . . 51
8.3 The Encoding Problem . . . 51
8.4 Tree Correspondence and Weighted External Path Length . . . 52
8.5 Huffman's Greedy Algorithm . . . 52
8.6 Worked Example . . . 52
8.7 Correctness Proof . . . 53
8.8 Application: Optimal Two-Way Merge . . . 54

9 Dynamic Programming **56**
9.1 Warm-up: Stair Climbing and Fibonacci . . . 56

9.2 The Rod Cutting Problem . . . 57
9.3 Exercise: Maximum Sum of Non-Adjacent Elements . . . 59
9.4 Weighted Interval Scheduling . . . 60
9.5 Exercise: Highway Billboards . . . 62
9.6 Kadane's Algorithm Derived from DP . . . 62
9.7 Minimum Number of Coins . . . 63
9.8 The 0/1 Knapsack Problem . . . 64
9.9 Longest Common Subsequence . . . 65
9.10 Exercise: Choosing m Numbers with a Sliding-Window Constraint . . . 67
9.11 DP over Intervals: Longest Palindromic Substring . . . 67
9.12 Longest Palindromic Subsequence . . . 69
9.13 Matrix-Chain Multiplication . . . 70
9.14 Optimal Binary Search Trees . . . 73
9.15 RNA Secondary Structure . . . 75

10 Graph Algorithms **77**
10.1 Graph Basics . . . 77
10.2 Graph Representation . . . 79
10.3 Paths, Connectivity, and Trees . . . 79
10.4 Breadth-First Search . . . 81
10.5 Connected Components and Reachability . . . 82
10.6 Worked BFS Example . . . 84
10.7 Depth-First Search . . . 85
10.8 Worked DFS Example . . . 86
10.9 Classifying Edges . . . 87
10.10 Cycle Detection . . . 88
10.11 Topological Sorting . . . 90

1 Prerequisite: Asymptotic Notations

O, Ω, Θ are used to describe the asymptotic behavior of functions.

Definitions of Asymptotic Notations

(a) **Big-O (Upper Bound):** $f(n) = O(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(b) **Big-Omega (Lower Bound):** $f(n) = \Omega(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

(c) **Big-Theta (Tight Bound):** $f(n) = \Theta(g(n))$ means that there exist positive constants c_1, c_2 and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

$$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(d) **Not Comparable:** $f(n)$ and $g(n)$ are not comparable if $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ does not exist.

Asymptotic Growth Hierarchy (Memorize This Order)

constant $<$ logarithmic $<$ polynomial $<$ exponential

$$O(1) \subset O(\log\log n) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n\log n)$$
$$\subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!) \subset O(n^n)$$

Common Mistakes with Asymptotic Notation

- $O(g(n))$ describes an **upper bound**, not necessarily tight. $n = O(n^2)$ is technically true but not useful.
- $\Omega(g(n))$ describes a **lower bound**. Best-case $\neq$ lower bound!
- $\Theta(g(n))$ requires **both** upper and lower bounds to match.
- Always drop constant factors and lower-order terms: $3n^2 + 5n + 7 = \Theta(n^2)$, not $\Theta(3n^2)$.

2024–25 Spring, Problem 2 (Asymptotic complexity, 5 points)

Problem. For each case fill in **A** (algorithm A eventually always faster), **B** (algorithm B faster) or **U** (information insufficient).

Case	$T_A(n)$	$T_B(n)$	Faster	Reason
0	$\Theta(n)$	$\Theta(n^2)$	A	$n < n^2$
1	$\Theta\big(\sum_{i=1}^{n} i\sqrt{n}\big)$	$\Theta(n^2)$	B	$\sqrt{n} \cdot \frac{n(n+1)}{2} = \Theta(n^{2.5})$
2	$O\big((\log n)^{10}\big)$	$\Omega(n^{0.1})$	A	polylog beats any positive power
3	$\Theta(2^n)$	$\Theta(2^{n/2})$	A	ratio $2^{n/2} \to \infty$
4	$O\big(\sum_{i=1}^{n} 2^{-i}\big)$	$\Omega(\log n)$	A	geometric sum is $O(1)$
5	$\Theta\big(\sum_{i=1}^{n} i\sqrt{i}\big)$	$\Theta(n^{4/3})$	B	$\sum i^{1.5} = \Theta(n^{2.5})$

Solution. Cases 3 and 5 are the traps: (3) is exponential vs. exponential, and the bases decide — 2^n grows like $(2^{1/2})^{2\cdot}$ times faster; (5) needs the integral bound $\sum_{i=1}^{n} i^p = \Theta(n^{p+1})$ for $p > -1$. Case 2 is the one place where "O vs. Ω" is decisive: a polylogarithmic upper bound is eventually below any $\Omega(n^\epsilon)$ lower bound, so A is provably faster. Case 0–1 illustrate that O/Θ claims must be simplified before comparing. □

2025–26 Spring, Problem 1(a) (Asymptotics table, 5 points)

Problem. Fill in A / B / U.

Case	$T_A(n)$	$T_B(n)$	Answer	Reason
0	$\Theta(n)$	$\Theta(n^2)$	A	given as example
1	$\Theta\big(n^2/(\log n)^{10}\big)$	$\Theta(n^{1.5})$	B	$T_A/T_B = \frac{n^{0.5}}{(\log n)^{10}} \to \infty$
2	$\Theta\big(2^{\sqrt{n}}\big)$	$\Theta\big(10^{\sqrt[3]{n}}\big)$	B	$2^{n^{1/2}}$ vs. $2^{O(n^{1/3})}$
3	$O(\sqrt{\log n})$	$\Omega(\log\log n)$	U	O vs. Ω: no comparison possible
4	$\Theta(5^n)$	$\Theta(7^n)$	A	ratio $(5/7)^n \to 0$
5	$\Theta(n^2)$	$T_B(n) = 2T_B(n/2) + n$	B	$T_B = \Theta(n \log n)$

Solution. Case 1 rewards simplifying before comparing: a polylogarithmic factor never competes with any n^ϵ. Case 2 is exponential-vs-exponential in disguise — take $\log_2$ of both: $\sqrt{n}$ versus $(\log_2 10)\, n^{1/3}$, and $n^{1/2} > n^{1/3}$. Case 3 is the only U: one bound is an upper bound and the other a lower bound, so no ordering is justified (e.g. T_A could be constant). Case 5 needs the Master Theorem applied to B first. □

What the Midterms Repeatedly Test

- **Recursion trees**: always write the number of nodes per level, the cost per node, and the level sum. When the log contains the level index (e.g. $n \log_4(n/4^i)$), expand it into an arithmetic series before summing.

- **Lower bounds**: count the number of possible outputs ($\binom{n}{k}k!$, $n!$) and take the log. When one side is O and the other is Ω, answer "U" unless the bounds are provably separated.
- **Heaps**: insertions follow the bubble-up path; for merging m heaps and bottom-up build-up use $O(k2^{m-k})$ and $\sum_k k/2^{k+1} \leq 1$.
- **Randomization**: always split into geometric stages ($\mathbb{E}[\text{good tries}] = 2$) and sum a shrinking geometric series. Remember coupon collector $= \Theta(n)$ for $n/2$ coupons and $\Theta(n \log n)$ for all of them.
- **Sorted array with few distinct values**: binary-search for "the last position holding this value", giving $O(\log^2 n)$.

2 Divide and Conquer

Solve a problem of size n by breaking it into one or more smaller problems of size less than n. Solve the smaller problems recursively and combine their solutions to solve the large problem.

Three Steps of D&C

1. **Divide:** Split the problem into subproblems.
2. **Conquer:** Solve subproblems recursively (or directly if small enough).
3. **Combine:** Merge subproblem solutions into the original solution.

2.1 Binary Search

Given a sorted array A and a target value x, find the index of x (or -1 if not found).

Algorithm 1 Binary Search

```
function BinarySearch(A, x)
    low ← 0
    high ← length(A) − 1
    while low ≤ high do
        mid ← ⌊(low + high)/2⌋
        if A[mid] = x then
            return mid
        else if A[mid] < x then
            low ← mid + 1
        else
            high ← mid − 1
        end if
    end while
    return −1
end function
```

Binary Search Analysis

Let $T(n)$ be the number of comparisons on an array of size n.

- Recurrence: $T(n) \leq T\left(\frac{n}{2}\right) + 2$, with $T(1) = T(0) = 1$.

- Expand: $T(n) \le T\left(\frac{n}{2}\right) + 2 \le T\left(\frac{n}{4}\right) + 4 \le \cdots \le T(1) + 2\log_2 n = 1 + 2\log_2 n$.
- Therefore: $T(n) = O(\log n)$.

Algorithm 2 Binary Search (Recursive Version)

```
function BinarySearch(A, l, r, x)
    if l > r then
        return nil                                   ▷ x not present
    end if
    m ← ⌊(l + r)/2⌋
    if A[m] = x then
        return m
    else if x < A[m] then
        return BinarySearch(A, l, m − 1, x)
    else
        return BinarySearch(A, m + 1, r, x)
    end if
end function
```

Worked example: search for $x = 42$ in

$$A = [4, 7, 10, 15, 19, 20, 42, 54, 87, 90].$$

Call	Range	m	$A[m]$ **vs.** 42
1	$[1, 10]$	5	$19 < 42 \Rightarrow$ go right
2	$[6, 10]$	8	$54 > 42 \Rightarrow$ go left
3	$[6, 7]$	6	$20 < 42 \Rightarrow$ go right
4	$[7, 7]$	7	$A[7] = 42$ **found!**

Recursion-Tree View and the $\Omega(\log n)$ Lower Bound

The recursion tree is a single path: exactly one subproblem per level, costing 2 comparisons per internal level and 1 at the leaf, giving $2\log_2 n + 1$ comparisons — equivalent to expansion but more visual.

- Upper bound: each level halves the range, so at most $\lfloor \log_2 n \rfloor + 1$ levels $\Rightarrow T(n) = O(\log n)$.
- Lower bound: an element sitting at a deepest leaf (or a failed search) forces traversing the full path of length $\log_2 n$, so $T(n) = \Omega(\log n)$.

Hence $T(n) = \Theta(\log n)$. More generally, the decision-tree model proves no search algorithm on a sorted array can beat $\Omega(\log n)$ comparisons (there are $n + 1$ possible answers: n positions plus "not found").

2.1.1 Variant: Find the Boundary (lower_bound)

Consider the variant with **no equality test**:

Algorithm 3 Binary Search without Equality Test

```
function FindBoundary(A, l, r, x)
    if l = r then
        return l
    end if
    m ← ⌊(l + r)/2⌋
    if A[m] < x then
        return FindBoundary(A, m + 1, r, x)
    else
        return FindBoundary(A, l, m, x)
    end if
end function
```

What Does This Variant Return?

- It returns the index of the **first** element $\geq x$ — the insertion position even if x is absent (C++ `std::lower_bound`, Python `bisect.bisect_left`).
- Changing $A[m] < x$ to $A[m] \leq x$ returns the first element **strictly greater** than x (`upper_bound`, `bisect_right`); the two answers differ exactly when x occurs in A.
- Running time is still $O(\log n)$: every call halves a non-empty range.

Tail recursion into a loop. Because the recursive call is the very last action (tail recursion), it rewrites directly as a `while` loop:

```
l = 1; r = n;
while (l < r) {
    m = (l + r) / 2;
    if (A[m] < x) l = m + 1;
    else          r = m;
}
return l;
```

The iterative form avoids function-call overhead, uses $O(1)$ instead of $O(\log n)$ space, and cannot overflow the call stack — this is why library routines are written this way.

When to Use Binary Search

Binary search works whenever you can discard half the search space with one comparison. The array need not be literally sorted—it just needs a "monotonic" property that lets you decide which half contains the answer.

2.1.2 Exercise 1: Rotated Sorted Array

Let $A[1..n]$ be a sorted array of n distinct numbers that has been rotated $n - k$ steps for some unknown integer $k \in [1, n-1]$. That is, $A[1..k]$ is sorted increasing, $A[k+1..n]$ is sorted increasing, and $A[n] < A[1]$.

Example: $n = 16$, $k = 10$:

$$A = [9, 13, 16, 18, 19, 23, 28, 31, 37, 42, 0, 1, 2, 5, 7, 8]$$

(1) Find the rotation index k in $O(\log n)$. ($A[k]$ is the maximum element.)

Algorithm 4 Find Rotated Index

```
function FindRotatedIndex(A, l, r)
    mid ← ⌊(l + r)/2⌋
    if A[mid] > A[mid + 1] then
        return mid
    else if A[mid] ≥ A[l] then
        return FindRotatedIndex(A, mid + 1, r)
    else
        return FindRotatedIndex(A, l, mid − 1)
    end if
end function
```

(2) Search for x in $O(\log n)$.

Algorithm 5 Search in Rotated Sorted Array

```
function SearchRotated(A, x, n)
    k ← FindRotatedIndex(A, 1, n)
    if x ≥ A[1] then
        return BinarySearch(A[1..k], x)
    else
        return BinarySearch(A[k + 1..n], x)
    end if
end function
```

2.1.3 Exercise 2: Finding the Last 0

Array $A[1..n]$ contains 0s followed by 1s.

(1) Standard $O(\log n)$ solution.

Algorithm 6 Find Last 0

```
function FindLast0(A, l, r)
    mid ← ⌊(l + r)/2⌋
    if A[mid] = 0 and A[mid + 1] = 1 then
        return mid
    else if A[mid] = 0 then
        return FindLast0(A, mid + 1, r)
    else
        return FindLast0(A, l, mid − 1)
    end if
end function
```

(2) $O(\log k)$ solution when $k \ll n$. Exponential search (gallop): double the index until we overshoot, then binary search.

Algorithm 7 Find Last 0 with $O(\log k)$

```
i ← 1
while i < n ∧ A[i] = 0 do
    i ← min{2i, n}
end while
if A[i] = 0 then
    return i                                   ▷ all zeros: last 0 is n
else if i = 1 then
    return −1                                  ▷ no zero
else
    return FindLast0(A, ⌊i/2⌋, i)
end if
```

2024–25 Spring, Problem 6 (Searching in a sorted array with few distinct values, 20 points)

Problem. $A[1..n]$ is non-decreasing and contains $\Theta(\log n)$ distinct numbers, but the count is not given. Output every distinct value together with its frequency in $O(\log^2 n)$ time (a left-to-right scan costs $O(n)$ and is too slow). Example: $A =$ `10 10 12 12 12 15 15 20 20 30` $\Rightarrow$ `(10,2),(12,3),(15,2),(20,2),(30,1)`.

Solution (binary search for the last occurrence of each run). A decomposes into $m = \Theta(\log n)$ maximal constant runs $A[1..k_1], A[k_1+1..k_2], \ldots, A[k_{m-1}+1..n]$.

```
function LastOcc(A, p, r, x)   ▷ precondition: A[p] = x, returns the last index of x
    if p = r or A[r] = x then
        return r
    end if
    q ← ⌊(p + r)/2⌋
    if A[q] = x and A[q + 1] > x then
        return q
    end if
    if A[q] = x then
        return LastOcc(A, q + 1, r, x)
    else
        return LastOcc(A, p, q − 1, x)
    end if
end function
```

Start with $p = 1$; repeatedly $k_i \leftarrow \textsc{LastOcc}(A, p, n, A[p])$, output $(A[p], k_i - p + 1)$, then set $p \leftarrow k_i + 1$. Stop once $k_i = n$.

Correctness. Each call is given $A[p] = x$, so the returned k satisfies $A[k] = x$ and $A[k+1] > x$ (or $k = n$): the run of x is exactly $A[p..k]$, giving the correct frequency; the next run starts at $k + 1$, so no value is skipped or counted twice.

Running time. LastOcc halves the range each call — $O(\log n)$. There are $m = \Theta(\log n)$ runs, so the total is $O(\log^2 n)$. □

2025–26 Fall, Problem 4 (Divide and conquer: k-th of two sorted arrays, 10 points)

Problem. $A[1..m]$, $B[1..n]$ are sorted increasing with distinct values; given $k \in [1, m+n]$, find the k-th smallest of $A \cup B$ in $O(\log k)$ time. Example: $A = [2, 5, 7]$, $B = [3, 6]$, $k = 3 \Rightarrow 5$.

Solution. Search for the k-th element of $A[p..q] \cup B[r..s]$, maintaining the invariant that the answer is the k-th smallest of the current union.

- If $k = 1$, return $\min\{A[p], B[r]\}$.
- Otherwise let $h = \lfloor k/2 \rfloor \geq 1$ and compare $A[p+h-1]$ with $B[r+h-1]$ (a probe that runs past the end of an array is treated as $+\infty$). At most one probe overflows because $k \leq (q-p+1) + (s-r+1)$.

- If $A[p+h-1] < B[r+h-1]$: exactly h elements of A are $\le A[p+h-1]$, and at most $h-1$ elements of B are, so its rank is at most $2h-1 \le k-1$. Hence $A[p..p+h-1]$ contains no element of rank k and is discarded, with $k \leftarrow k-h$. Symmetrically, if $B[r+h-1] < A[p+h-1]$ discard $B[r..r+h-1]$.

Correctness. Each step removes exactly those elements that provably cannot be the k-th smallest, and reduces the rank by the number removed; the invariant "the answer is the k'-th smallest of the current union" is preserved until $k' = 1$.

Time. The rank goes $k \to k - \lfloor k/2 \rfloor = \lceil k/2 \rceil$, halving each step, so there are $O(\log k)$ calls of $O(1)$ work: $O(\log k)$.

Worked example $A = [2,5,7]$, $B = [3,6]$, $k = 3$: $h = 1$, compare $A[1] = 2$ with $B[1] = 3$; $2 < 3$, so discard $A[1]$ and set $k \leftarrow 2$. Now $A = [5,7]$, $B = [3,6]$, $k = 2$: $h = 1$, compare 5 with 3; $3 < 5$, discard $B[1]$ and set $k \leftarrow 1$. Finally $k = 1$ returns $\min\{5,6\} = 5$. □

2025–26 Spring, Problem 3 (Searching with $\Theta(\log n)$ distinct values, 20 points)

Problem. Identical to 2024–25 Spring Problem 6: output the distinct values of a non-decreasing array and their frequencies in $O(\log^2 n)$ time, without being given the number of distinct values.

Solution. Reuse the LastOcc divide-and-conquer from that problem: for each run start p, binary search the last index k with $A[k] = A[p]$ in $O(\log n)$, emit $(A[p], k-p+1)$, and continue at $k+1$. There are $m = \Theta(\log n)$ runs, so the total is $O(\log^2 n)$.

Why a plain binary search is not enough. Ordinary binary search finds an index of x; we need the right end of the run. The precondition $A[p] = x$ guarantees termination at $A[k+1] > x$, and the "$A[r] = x$" shortcut handles an all-equal suffix in $O(1)$. □

2.2 Mergesort & Inversion Count

2.2.1 Warm-up: Tower of Hanoi

Three pegs, n disks of decreasing size. Move all disks from source to target peg.

Hanoi Recurrence

Let $T(n)$ = minimum moves for n disks.

- Basis: $T(1) = 1$.
- Recursive step: move $n-1$ disks source→aux, move largest source→target, move $n-1$ aux→target.

$$T(n) = 2T(n-1) + 1$$

- Solution: $T(n) = 2^n - 1$. (Expand: $T(n) = 2^k T(n-k) + (2^k - 1)$, set $k = n-1$.)

Algorithm 8 Hanoi Tower

```
function Hanoi(n, A, B, C)
    if n = 1 then
        Move disk from A to C
    else
        Hanoi(n − 1, A, C, B)
        Move disk from A to C
        Hanoi(n − 1, B, A, C)
    end if
end function
```

2.2.2 Merge: Combining Two Sorted Halves

Given two sorted subarrays $A[p..q]$ and $A[q+1..r]$, merge them into $A[p..r]$ sorted.

Algorithm 9 Merge

```
function MERGE(A, p, q, r)
    n1 ← q − p + 1;    n2 ← r − q
    Create arrays L[1..n1 + 1], R[1..n2 + 1]
    for i = 1 to n1 do
        L[i] ← A[p + i − 1]
    end for
    for j = 1 to n2 do
        R[j] ← A[q + j]
    end for
    L[n1 + 1] ← ∞;    R[n2 + 1] ← ∞                    ▷ sentinels
    i ← 1;    j ← 1
    for k = p to r do
        if L[i] ≤ R[j] then
            A[k] ← L[i];    i ← i + 1
        else
            A[k] ← R[j];    j ← j + 1
        end if
    end for
end function
```

Merge Time Complexity

Merge runs in $\Theta(n)$ time on $n = r - p + 1$ elements: two copy loops ($n_1 + n_2 = n$) plus one comparison loop (n iterations). Sentinels avoid boundary checks — clean but not cache-friendly in practice.

Worked example: merge the sorted halves $L = [2, 4, 5, 7, \infty]$ and $R = [1, 2, 3, 6, \infty]$:

k	compare	winner	$A[p \ldots r]$ so far
1	$L[1] = 2$ vs $R[1] = 1$	$R[1]$	$[1]$
2	$L[1] = 2$ vs $R[2] = 2$	$L[1]$ ($\leq$ keeps it stable)	$[1, 2]$
3	$L[2] = 4$ vs $R[2] = 2$	$R[2]$	$[1, 2, 2]$
4	$L[2] = 4$ vs $R[3] = 3$	$R[3]$	$[1, 2, 2, 3]$
5	$L[2] = 4$ vs $R[4] = 6$	$L[2]$	$[1, 2, 2, 3, 4]$
6	$L[3] = 5$ vs $R[4] = 6$	$L[3]$	$[1, 2, 2, 3, 4, 5]$
7	$L[4] = 7$ vs $R[4] = 6$	$R[4]$	$[1, 2, 2, 3, 4, 5, 6]$
8	$L[4] = 7$ vs ∞	$L[4]$	$[1, 2, 2, 3, 4, 5, 6, 7]$

Each step advances the head pointer (i or j) of the list whose element was copied; exactly one element is placed per iteration.

2.2.3 Mergesort Algorithm

Algorithm 10 Mergesort

```
function MERGESORT(A, p, r)
    if p < r then
        q ← ⌊(p + r)/2⌋                          ▷ Divide
        MERGESORT(A, p, q)                       ▷ Conquer left
        MERGESORT(A, q + 1, r)                   ▷ Conquer right
        MERGE(A, p, q, r)                        ▷ Combine
    end if
end function
```

Mergesort Analysis

Recurrence for n elements (power of 2 for simplicity):

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Solution: $T(n) = \Theta(n \log n)$. (Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$, case 2.)

Worked example: $A = [5, 2, 4, 7, 1, 3, 2, 6]$.

Stage	Array state
input	$[5, 2, 4, 7 \mid 1, 3, 2, 6]$
after sorting left half recursively	$[2, 4, 5, 7 \mid 1, 3, 2, 6]$
after sorting right half recursively	$[2, 4, 5, 7 \mid 1, 2, 3, 6]$
after $\Theta(n)$ merge	$[1, 2, 2, 3, 4, 5, 6, 7]$

Solving the Recurrence by Expansion / Recursion Tree

Expanding directly:

$$\begin{aligned} T(n) &= 2T(n/2) + n = 2^2 T(n/4) + 2n = 2^3 T(n/8) + 3n \\ &= 2^i T(n/2^i) + in = nT(1) + n \log_2 n = n \log n + n. \end{aligned}$$

On the recursion tree, level i has 2^i nodes, each paying $n/2^i$, so the cost of every level is $2^i \cdot (n/2^i) = n$; with $\log_2 n + 1$ levels the total is $n(\log_2 n + 1)$.

Why $\Omega(n \log n)$ Too?

The merge step costs $\Theta(n)$ on every input — there is no lucky case that skips work. Replacing each O by Ω in the analysis is still valid, so $T(n) = \Omega(n \log n)$ and overall $T(n) = \Theta(n \log n)$: every input is a worst-case input for mergesort.

Mergesort Properties

- **Stable:** yes (preserves order of equal keys if $L[i] \leq R[j]$ uses $\leq$, not $<$).
- **In-place:** no (needs auxiliary arrays L and R; $\Theta(n)$ extra space).
- **Best / Worst / Average:** all $\Theta(n \log n)$ (unlike Quicksort).

2.2.4 Counting Inversions

An **inversion** is a pair (i, j) with $i < j$ and $A[i] > A[j]$. Count the total number of inversions.

Brute Force is Too Slow

Checking all $\binom{n}{2}$ pairs takes $\Theta(n^2)$ time. With D&C we can do $\Theta(n \log n)$ by **piggybacking on Mergesort**.

Algorithm 11 Count Inversions via Mergesort

```
function CountInv(A, p, r)
    if p ≥ r then
        return 0
    end if
    q ← ⌊(p + r)/2⌋
    left ← CountInv(A, p, q)
    right ← CountInv(A, q + 1, r)
    cross ← MergeCount(A, p, q, r)
    return left + right + cross
end function

function MergeCount(A, p, q, r)
    ... same as Merge, but whenever R[j] is copied before L[i] is exhausted:
    add (n_1 − i + 1) to cross (all remaining L[i..n_1] are > R[j])
    return cross
end function
```

Why Cross-Inversions Work

When copying $R[j]$ before $L[i]$ is empty, **every remaining element in L forms an inversion with $R[j]$** because L indices $< R$ indices (by array split) and $L[i..n_1] > R[j]$ (since we picked $R[j]$ first). This adds exactly $n_1 - i + 1$ inversions in $O(1)$.

Applications. The inversion number measures (i) how "sorted" an array is ($0 =$ sorted, $\binom{n}{2} =$ reverse sorted), and (ii) the similarity between two rankings: map one ranking to the identity permutation and count inversions in the other.

Worked example: $A = [4, 8, 10, 2, 1, 5, 12, 11, 3, 7, 6, 9]$.

Split into blue half $[4, 8, 10, 2, 1, 5]$ and green half $[12, 11, 3, 7, 6, 9]$:

- 5 blue-blue inversions and 8 green-green inversions (found recursively).
- 9 blue-green inversions: $(5, 3)$, $(4, 3)$, $(8, 6)$, $(8, 3)$, $(8, 7)$, $(10, 6)$, $(10, 9)$, $(10, 3)$, $(10, 7)$.
- Total: $5 + 8 + 9 = \mathbf{22}$.

Counting cross inversions during the merge. With 1-indexed arrays split at m, when $R[j]$ is copied while the L-pointer is at i, the blue elements already passed ($L[1..i-1]$) are $\le R[j]$; every remaining blue element $L[i..m]$ is $> R[j]$, so the contribution of $R[j]$ is

$$I[j] = m - i + 1 \quad \big(\text{or, with the slide's indexing, } m - l - i + 2\big).$$

Tiny trace: sorted halves $L = [1, 3, 5, 7]$, $R = [2, 4, 6, 8]$:

Element copied from R	pointer i	$L[i..]$ greater than it	$I[j]$
2	1 ($L[1] = 1 \le 2$ passed, then $i = 2$)	$3, 5, 7$	3
4	3	$5, 7$	2
6	4	7	1
8	past end	—	0

Cross inversions $= 3+2+1+0 = 6$; the merge simultaneously produces $[1, 2, 3, 4, 5, 6, 7, 8]$, preserving the sortedness invariant needed by the parent call.

Recursion-tree accounting. Each leaf returns 0; each internal node returns its cross count from the merge, and a node reports the sum of its two children plus its own cross count. For the 12-element example the two size-6 root children return 5 (blue half) and 8 (green half), and the root merge contributes 9:

root: $5 + 8 + 9 = 22$
left child `Sort-and-Count`$[4, 8, 10, 2, 1, 5] \to 5$
right child `Sort-and-Count`$[12, 11, 3, 7, 6, 9] \to 8$
(their values are sums of cross counts at lower levels; leaves contribute 0)

Since both the counting and the merging are linear scans, `Merge-and-Count` costs $\Theta(n)$ and the overall recurrence is again $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Algorithm 12 Merge-and-Count (precondition: $A[l..m]$ and $A[m+1..r]$ sorted)

copy $A[l..m]$ into $L[1..n_1]$, $A[m+1..r]$ into $R[1..n_2]$, append ∞ sentinels
$i \leftarrow 1$; $j \leftarrow 1$; $c \leftarrow 0$
for $k = l$ **to** r **do**
  if $L[i] \le R[j]$ **then**
    $A[k] \leftarrow L[i]$; $i \leftarrow i + 1$
  else
    $A[k] \leftarrow R[j]$; $j \leftarrow j + 1$
    $c \leftarrow c + (n_1 - i + 1)$ ▷ all remaining L elements invert $R[j]$
  end if
end for
return c

2025–26 Fall, Problem 1(a) (Recursion tree, 4 points)

Problem. For n a power of 4 and

$$T(n) = 4T(n/4) + n \log_4 n, \qquad T(1) = 1,$$

derive the exact asymptotic solution by the recursion tree method, showing for levels $0, 1, 2$ the level number, per-node cost and level sum, then the general level i, the number of leaves and the sum of leaf costs.

Solution. Level i has 4^i nodes of size $n/4^i$, each costing

$$\frac{n}{4^i} \log_4 \frac{n}{4^i} = \frac{n}{4^i}\big(\log_4 n - i\big).$$

Level	# nodes	Size per node	Cost per node	Level sum
0	1	n	$n \log_4 n$	$n \log_4 n$
1	4	$n/4$	$\frac{n}{4}(\log_4 n - 1)$	$n(\log_4 n - 1)$
2	4^2	$n/4^2$	$\frac{n}{4^2}(\log_4 n - 2)$	$n(\log_4 n - 2)$
i	4^i	$n/4^i$	$\frac{n}{4^i}(\log_4 n - i)$	$n(\log_4 n - i)$

The tree stops at level $L = \log_4 n$, so there are $4^L = n$ leaves, each costing $T(1) = 1$:

leaf total $= n$. Summing levels $i = 0, \ldots, L-1$:

$$T(n) = n + \sum_{i=0}^{L-1} n(L-i) = n + n\frac{L(L+1)}{2} = \Theta\big(n(\log n)^2\big),$$

i.e. $\alpha = 1$, $\beta = 2$. Note the contrast with mergesort: here the per-level cost decreases ($n \log_4 n$, $n(\log_4 n - 1)$, $\ldots$) and the leaves are the dominant term class, but the arithmetic series still contributes an extra $\log n$ factor. □

2.3 Maximum Subarray

Given an array $A[1..n]$, find indices i, j maximizing $\sum_{t=i}^{j} A[t]$ (the empty subarray is not allowed). (Useful for best-buy/best-sell stock problem.)

Motivating example. Yearly profits (M$) $[3, 2, 1, -7, 5, 2, -1, 3, -1]$: the best span is years 5–8 with $5 + 2 - 1 + 3 = 9$ M$.

Two Simpler Solutions First

1. **Brute force $\Theta(n^3)$:** enumerate all pairs (i, j) and re-sum $A[i..j]$ with an inner loop. The j- and k-loops average $n/2$ and $n/3$ iterations.
2. **Data reuse $\Theta(n^2)$:** exploit $V(i, j) = V(i, j-1) + A[j]$ — extend the running sum instead of restarting, removing the inner loop.

The D&C solution improves this to $\Theta(n \log n)$, and Kadane's DP reaches $\Theta(n)$.

D&C Strategy

For a subarray $A[low..high]$ with midpoint mid, the maximum subarray is **exactly one** of:

1. Entirely in $A[low..mid]$ (recurse left).
2. Entirely in $A[mid + 1..high]$ (recurse right).
3. **Crossing the midpoint:** one end in left half, one end in right half (compute directly).

Return the maximum of the three.

Algorithm 13 Maximum Subarray (D&C)

```
function MaxSubarray(A, low, high)
    if low = high then
        return (low, high, A[low])
    end if
    mid ← ⌊(low + high)/2⌋
    (left_lo, left_hi, left_sum) ← MaxSubarray(A, low, mid)
    (right_lo, right_hi, right_sum) ← MaxSubarray(A, mid + 1, high)
    (cross_lo, cross_hi, cross_sum) ← MaxCrossing(A, low, mid, high)
    if left_sum ≥ right_sum and left_sum ≥ cross_sum then
        return (left_lo, left_hi, left_sum)
    else if right_sum ≥ left_sum and right_sum ≥ cross_sum then
        return (right_lo, right_hi, right_sum)
    else
        return (cross_lo, cross_hi, cross_sum)
    end if
end function

function MaxCrossing(A, low, mid, high)
    left_sum ← −∞;    sum ← 0
    for i = mid downto low do
        sum ← sum + A[i]
        if sum > left_sum then
            left_sum ← sum;    max_left ← i
        end if
    end for
    right_sum ← −∞;    sum ← 0
    for j = mid + 1 to high do
        sum ← sum + A[j]
        if sum > right_sum then
            right_sum ← sum;    max_right ← j
        end if
    end for
    return (max_left, max_right, left_sum + right_sum)
end function
```

Max Subarray Analysis

Recurrence:

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ 2T(n/2) + \Theta(n) & n > 1. \end{cases}$$

Same as Mergesort: $T(n) = \Theta(n \log n)$.

Kadane's Algorithm (iterative DP, not D&C): $O(n)$ time, $O(1)$ space.

$$\text{ending_here}_i = \max\big(A[i],\ \text{ending_here}_{i-1} + A[i]\big),$$
$$\text{best}_i = \max\big(\text{best}_{i-1},\ \text{ending_here}_i\big).$$

Kadane trace on $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ (start $V = V_{\max} = -\infty$):

i	$A[i]$	$V = \max(V + A[i], A[i])$ **(best ending at i)**	$V_{\max}$	**best window so far**
1	-2	-2	-2	$[-2]$
2	1	1 (restart)	1	$[1]$
3	-3	-2	1	$[1]$
4	4	4 (restart)	4	$[4]$
5	-1	3	4	$[4]$
6	2	5	5	$[4, -1, 2]$
7	1	6	**6**	$[4, -1, 2, 1]$
8	-5	1	6	$[4, -1, 2, 1]$
9	4	5	6	$[4, -1, 2, 1]$

Answer: 6, achieved by $[4, -1, 2, 1]$. The "restart or extend" choice at every step is the DP decision.

2.3.1 Application: Maximizing Stock Profit

Given prices $p[1..n]$, buy once and sell later ($i < j$) to maximize $p[j] - p[i]$ (a trade is mandatory).

Direct D&C: split at m. The optimal pair is either (1) entirely in the left half, (2) entirely in the right half, or (3) **crossing**: buy at the minimum price in $p[l..m]$ and sell at the maximum price in $p[m+1..r]$, found by two linear scans. Same recurrence $2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Reduction to Maximum Subarray ($O(n)$ preprocessing)

Build the difference array of consecutive-day changes:

$$\text{Profit}[i] = p[i+1] - p[i].$$

Every interval telescopes:

$$\sum_{t=i}^{j-1} \text{Profit}[t] = \big(p[i+1] - p[i]\big) + \big(p[i+2] - p[i+1]\big) + \cdots + \big(p[j] - p[j-1]\big) = p[j] - p[i].$$

So the maximum-profit trade is exactly the maximum subarray of `Profit`, solvable by Kadane in $\Theta(n)$.

Example: prices $[3, 6, 10, 1, 8, 2, 6, 5, 10, 3, 1, 9]$ give differences

$$[3, 4, -9, 7, -6, 4, -1, 5, -7, -2, 8],$$

whose maximum subarray identifies the best buy/sell pair.

2.4 Multiplication and Master Theorem

2.4.1 Integer Multiplication (Karatsuba)

Given two n-bit integers x and y, compute $x \cdot y$.

Grade-School Multiplication

The standard pencil-and-paper algorithm is $\Theta(n^2)$: multiply each bit of y by x (shift & add), giving n rows of $O(n)$ bits each. Example: 163×97:

$$\begin{array}{rrr} & & 163 \\ \times & & 97 \\ \hline & 1141 & (163 \times 7) \\ + \quad 1467 & & (163 \times 9, \text{ shifted}) \\ \hline & & 15811 \end{array}$$

In binary, 10100011×01100001 similarly produces n shifted partial products: $\Theta(n^2)$ single-bit multiplications and $\Theta(n^2)$ binary additions.

2.4.2 First D&C Attempt: Four Half-Size Products

Split $a = 2^{n/2}a_1 + a_0$, $b = 2^{n/2}b_1 + b_0$ (high/low $n/2$ bits):

$$a \cdot b = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0.$$

Multiplication by 2^k is just a k-bit left shift (e.g. $12 = 00001100$, then $12 \cdot 2^3 = 96 = 01100000$), so the four products $a_1b_1, a_1b_0, a_0b_1, a_0b_0$ are computed recursively and combined with $\Theta(n)$ additions:

$$T(n) = 4T(n/2) + \Theta(n), \qquad T(1) = 1.$$

Analysis: The Naive D&C Is Still $\Theta(n^2)$

Expanding: $T(n) = 4^i T(n/2^i) + n(2^{i-1} + \cdots + 2 + 1)$; at $i = \log_2 n$,

$$T(n) = n^2 + n(2^{\log_2 n} - 1) = n^2 + n(n-1) = \Theta(n^2).$$

On the recursion tree, level i costs $4^i \cdot n/2^i = n2^i$, so costs double each level and the leaves dominate ($4^{\log_2 n} = n^2$). Essentially the method still multiplies every bit of a by every bit of b. The contrast with mergesort: one problem spawns **4** subproblems of size $n/2$, not 2.

Karatsuba Split

Write $x = a \cdot 2^{n/2} + b$, $y = c \cdot 2^{n/2} + d$ (split into high/low halves). Then:

$$x \cdot y = ac \cdot 2^n \; + \; \big[(a+b)(c+d) - ac - bd\big] \cdot 2^{n/2} \; + \; bd.$$

Instead of 4 half-size multiplications (ac, ad, bc, bd), we use **3**: ac, bd, and $(a+b)(c+d)$, subtracting to recover $ad + bc$.

Algorithm 14 Karatsuba Multiplication

```
function MULTIPLY(x, y, n)
    if n = 1 then
        return x · y
    end if
    a ← high n/2 bits of x;    b ← low n/2 bits of x
    c ← high n/2 bits of y;    d ← low n/2 bits of y
    ac ← MULTIPLY(a, c, n/2)
    bd ← MULTIPLY(b, d, n/2)
    ab_cd ← MULTIPLY(a + b, c + d, n/2)
    return ac ≪ n + (ab_cd − ac − bd) ≪ (n/2) + bd
end function
```

Karatsuba Recurrence

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

By Master Theorem case 1 ($\log_b a = \log_2 3 \approx 1.585 > 1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}),$$

asymptotically better than $\Theta(n^2)$.

Decimal illustration ($n = 2$ digits, no recursion needed). Multiply 23×14: $a_1 = 2, a_0 = 3$, $c_1 = 1, d_0 = 4$.

$$\begin{aligned} ac &= 2 \cdot 1 = 2, & bd &= 3 \cdot 4 = 12, \\ (a+b)(c+d) &= 5 \cdot 5 = 25, & ad + bc &= 25 - 2 - 12 = 11, \\ 23 \cdot 14 &= 2 \cdot 100 + 11 \cdot 10 + 12 \\ &= 200 + 110 + 12 = \mathbf{322}. \end{aligned}$$

Only **3** small multiplications $(2, 12, 25)$ instead of 4.

History and Practice

Karatsuba (1960) was the first algorithm to beat $\Theta(n^2)$. Splitting into r parts and using fewer than r^2 products gives the Toom–Cook family: Toom-3 $\Rightarrow \Theta(n^{\log_3 5}) = \Theta(n^{1.465})$. FFT-based methods reached $\Theta(n \log n \log\log n)$; a $\Theta(n \log n)$ integer-multiplication algorithm was proven by Harvey & van der Hoeven in 2019. Real big-integer libraries combine methods by input size: textbook for tiny inputs, Karatsuba/Toom for medium, FFT for very large.

2.4.3 Master Theorem for Recurrences

For recurrences of the form:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \quad a \geq 1,\ b > 1.$$

Compare $f(n)$ with $n^{\log_b a}$ (the "leaf work").

Master Theorem — Three Cases

1. **Case 1 (Leaves dominate):** If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$,
$$T(n) = \Theta(n^{\log_b a}).$$
2. **Case 2 (Balanced):** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$,
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$
3. **Case 3 (Root dominates):** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the **regularity condition** $a\,f(n/b) \leq c\,f(n)$ holds for some $c < 1$ and large n,
$$T(n) = \Theta(f(n)).$$

Common D&C Recurrences (Memorize!)

Algorithm	Recurrence	Solution
Binary search	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$ (Case 2, $k = 0$)
Mergesort	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$ (Case 2, $k = 0$)
Karatsuba	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3})$ (Case 1)
Strassen matrix	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7})$ (Case 1)
Bad D&C (naive 4-mult)	$T(n) = 4T(n/2) + \Theta(n)$	$\Theta(n^2)$ (Case 1)

Why the Master Theorem Works (Recursion-Tree Intuition)

For the simplified form $T(n) = aT(n/b)+n$: level i contains a^i subproblems of size n/b^i and costs $n(a/b)^i$; there are $\log_b n$ levels and the leaves alone contribute $a^{\log_b n} = n^{\log_b a}$.

Total:

$$T(n) = n^{\log_b a} + n \sum_{i=0}^{\log_b n - 1} \left(\frac{a}{b}\right)^i.$$

The geometric ratio a/b decides everything:

- $a > b$: increasing geometric series, **leaves dominate**, $T(n) = \Theta(n^{\log_b a})$.
- $a = b$: every term is 1, each level costs n, $T(n) = \Theta(n \log n)$.
- $a < b$: decreasing geometric series is $\Theta(1)$, **the root dominates**, $T(n) = \Theta(n)$.

2024–25 Spring, Problem 3 (Recurrence, 5 points)

Problem. Solve $T(n) = 2T(\sqrt{n}) + \log_2 n$, with $T(c) = \Theta(1)$ for any convenient constant $c \geq 1$.

Solution (substitution to fit the Master Theorem). Let $k = \log_2 n$, i.e. $n = 2^k$, and define $S(k) = T(2^k)$. Then $\sqrt{n} = 2^{k/2}$ and

$$S(k) = 2S\left(\frac{k}{2}\right) + k.$$

Master Theorem, case 2 ($a = 2$, $b = 2$, $f(k) = k = \Theta(k)$): $S(k) = \Theta(k \log k)$. Substituting back,

$$T(n) = \Theta\big(\log n \cdot \log\log n\big).$$

Sanity check by expansion: at depth i the argument is $n^{1/2^i}$ and there are 2^i nodes of cost $\log(n^{1/2^i}) = (\log n)/2^i$ each, so every level costs exactly $\log n$. The recursion stops when $n^{1/2^i} = 2$, i.e. $i = \log\log n$, giving $\log n \cdot \log\log n$ — the same answer without the Master Theorem. □

2025–26 Spring, Problem 1(b) (Recurrence by expansion, 5 points)

Problem. For n a power of 4, solve $T(n) = 64\,T(n/4) + n^3$, $T(1) = 1$, without the Master Theorem, and write the answer as $\Theta(n^\alpha (\log n)^\beta)$.

Solution. Since $64 = 4^3$, unrolling k times gives

$$T(n) = 4^{3k} T(n/4^k) + k\, n^3.$$

Indeed each level contributes n^3: level j has $64^j = 4^{3j}$ nodes of size $n/4^j$, total $4^{3j}(n/4^j)^3 = n^3$. The base case is reached at $k = \log_4 n$, where $4^{3k} = n^3$ and $T(1) = 1$:

$$T(n) = n^3 + (\log_4 n)\, n^3 = \Theta\big(n^3 \log n\big),$$

so $\alpha = 3$, $\beta = 1$. This is exactly Master Theorem case 2: $a = 64$, $b = 4$, $n^{\log_b a} = n^3$ matches $f(n) = n^3$. □

2.5 Matrix Multiplication and Strassen's Algorithm

Given two $n \times n$ matrices, compute $C = AB$ with $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$. The brute-force triple loop costs $\Theta(n^3)$ (n^2 entries, each a length-n dot product).

Naive D&C: partition each matrix into four $n/2 \times n/2$ blocks:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

requiring **8** block multiplications (e.g. $C_{11} = A_{11}B_{11} + A_{12}B_{21}$) and $\Theta(n^2)$ block additions:

$$T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$$

(Master case 1: $\log_2 8 = 3$, and $n^{\log_2 8} = n^3$ dominates n^2). No improvement.

Strassen's Idea: 7 Block Products

Strassen (1969) computes the 2×2 block product with only **7** recursive multiplications plus $\Theta(n^2)$ additions/subtractions:

$$P_1 = A_{11}(B_{12} - B_{22}), \quad P_2 = (A_{11} + A_{12})B_{22}, \quad P_3 = (A_{21} + A_{22})B_{11},$$
$$P_4 = A_{22}(B_{21} - B_{11}), \quad P_5 = (A_{11} + A_{22})(B_{11} + B_{22}),$$
$$P_6 = (A_{12} - A_{22})(B_{21} + B_{22}), \quad P_7 = (A_{11} - A_{21})(B_{11} + B_{12}),$$

then

$$C_{11} = P_5 + P_4 - P_2 + P_6, \quad C_{12} = P_1 + P_2,$$
$$C_{21} = P_3 + P_4, \quad C_{22} = P_5 + P_1 - P_3 - P_7.$$

Hence

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2.807\ldots}),$$

the first algorithm to beat cubic matrix multiplication. It is practical only for large dense matrices ($n \gtrsim 100$): the extra additions and recursion overhead make it slower on small matrices.

3 Randomized Algorithms

3.1 Why Randomization?

A randomized algorithm flips coins during execution. Worst-case analysis becomes **expected**-case analysis, often beating deterministic worst-case bounds.

Two Types of Randomized Algorithms

- **Las Vegas:** always correct; running time is a random variable (e.g. randomized Quicksort). We bound the expected runtime.
- **Monte Carlo:** always fast; has a small probability of being wrong (e.g. randomized min-cut / Karger). Repeat to reduce error.

How We Measure Randomized Running Time

We take the **expected running time on the worst-case input**: $\max_{I:|I|=n} \mathbb{E}_R[T(I,R)]$, where the expectation is only over the algorithm's internal randomness R (random pivots, random permutations, ...). Randomized algorithms are often simpler and faster in practice; whether randomization can give an asymptotic advantage over deterministic algorithms remains open for some problems (e.g. $O(1)$-time hash tables with $O(n)$ space).

3.2 Probability Review

For a discrete random variable X taking values with probabilities $\Pr[X = i]$:

$$\mathbb{E}[X] = \sum_i i \cdot \Pr[X = i].$$

Dice warm-ups:

- One fair 6-sided die: $\mathbb{E}[X] = \dfrac{1+2+\cdots+6}{6} = 3.5$.
- Sum of two dice: by linearity of expectation, $\mathbb{E}[X_1 + X_2] = 3.5 + 3.5 = \mathbf{7}$.
- Maximum of two dice: $\Pr[\max = k] = (k^2 - (k-1)^2)/36 = (2k-1)/36$, so

$$\mathbb{E}[\max] = 1 \cdot \tfrac{1}{36} + 2 \cdot \tfrac{3}{36} + 3 \cdot \tfrac{5}{36} + \cdots + 6 \cdot \tfrac{11}{36} \approx 4.47.$$

Two useful facts: $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ only when X, Y are **independent** (but linearity of expectation needs no independence — that is why it is so powerful). For a non-negative integer-valued variable:

$$\mathbb{E}[X] = \sum_{i \geq 1} \Pr[X \geq i]$$

(the "tail-sum" formula, handy for waiting times).

3.3 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (they need **not** be independent):

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

This is the single most useful tool for analyzing randomized algorithms.

Indicator Variables

To count "how many events happened", set:

$$X_i = \mathbf{1}\{\text{event } i \text{ occurs}\} = \begin{cases} 1 & \text{if event } i \text{ happens,} \\ 0 & \text{otherwise.} \end{cases}$$

Then $\mathbb{E}[X_i] = \Pr(\text{event } i)$, and $X = \sum X_i$ gives the total count with $\mathbb{E}[X] = \sum \Pr(\text{event } i)$.

3.4 Example: Hiring Problem

Interview candidates in random order; hire whenever candidate is better than all previous. How many hires (expected)?

Let $X_i = 1$ if candidate i is hired (i.e. best among first i candidates). In random order this occurs with probability $1/i$.

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{i} = H_n = \Theta(\log n).$$

So the expected number of hires is the n-th harmonic number, $\ln n + O(1)$.

3.5 More Indicator-Variable Examples

3.5.1 Guessing Shuffled Cards

Shuffle n cards and guess each card as it is revealed.

- **No memory:** each turn guess a uniformly random card from the full deck. Let $X_i = \mathbf{1}\{\text{guess } i \text{ correct}\}$; $\mathbb{E}[X_i] = 1/n$ regardless of other guesses, so $\mathbb{E}[X] = \sum_i 1/n = \mathbf{1}$.
- **With memory:** guess uniformly among the unseen cards. At turn i there are $n-i+1$ candidates, so $\mathbb{E}[X_i] = 1/(n-i+1)$ and

$$\mathbb{E}[X] = \frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{1} = H_n = \Theta(\log n).$$

Note that the events are dependent in both games — linearity of expectation applies anyway.

3.5.2 Hash Table with Chaining

A hash function $h : U \to \{0, \ldots, m-1\}$ maps n keys into a table of m slots; collisions are resolved by a linked list per slot. Under the **simple uniform hashing assumption** ($h(x)$ uniform and mutually independent across keys), define $I_y = \mathbf{1}\{h(y) = h(x)\}$ for each stored key $y \neq x$. Then $\mathbb{E}[I_y] = 1/m$ and

$$\mathbb{E}[\text{keys colliding with } x] = \sum_{y \neq x} \mathbb{E}[I_y] \leq \frac{n}{m}.$$

With load factor $\alpha = n/m$, insert/delete/query take $O(1+\alpha)$ expected time; choosing $m = \Theta(n)$ makes this $O(1)$. (This is the idea behind C++ `unordered_map`.)

3.5.3 Birthday Paradox

With k people and $n = 365$ equiprobable birthdays, let $X_{ij} = \mathbf{1}\{i, j \text{ share a birthday}\}$ ($\mathbb{E}[X_{ij}] = 1/n$):

$$\mathbb{E}[X] = \binom{k}{2}\frac{1}{n} = \frac{k(k-1)}{2n}.$$

Expecting ≥ 1 collision means $k(k-1)/(2n) \geq 1$, i.e. $k \gtrsim \sqrt{2n} + 1$ — about **28** people for birthdays, far below the intuitive $365/2$. (This threshold also dictates when an m-slot hash table starts seeing collisions.)

3.6 Generating a Uniform Random Permutation

The hiring analysis assumes a uniformly random candidate order; the Fisher–Yates shuffle produces one in place:

Algorithm 15 Fisher–Yates Shuffle

```
function RandomPermute(A)
    for i = 1 to n do
        swap A[i] ↔ A[Random(1, i)]
    end for
end function
```

$O(n)$ time and $O(1)$ extra space.

Correctness: Every Permutation Has Probability $1/n!$

Induct on i: after iteration i, every permutation of $A[1..i]$ occurs with probability $1/i!$. For target $(a_1, \ldots, a_i)$, let j be the position of the original $A[i]$ in this target. The target appears iff (i) after iteration $i-1$ the other $i-1$ elements had the required order (probability $1/(i-1)!$ by induction), and (ii) Random$(1, i) = j$ (probability $1/i$). Multiplying: $1/(i-1)! \cdot 1/i = 1/i!$.

3.7 Geometric Distribution and Coupon Collector

Waiting for a success. If each coin flip succeeds with probability p, the number X of flips to the first head is geometric:

$$\Pr[X = j] = (1-p)^{j-1}p, \qquad \mathbb{E}[X] = \sum_{i \geq 1}\Pr[X \geq i] = \sum_{i \geq 1}(1-p)^{i-1} = \frac{1}{p}.$$

Coupon collector. Each cereal box contains one of n coupon types uniformly at random; how many boxes until all types are seen? Split the process into stages: after collecting i distinct coupons, the chance the next box is new is $(n-i)/n$, so stage i is geometric with mean $n/(n-i)$:

$$\mathbb{E}[X] = \sum_{i=0}^{n-1}\frac{n}{n-i} = n\sum_{j=1}^{n}\frac{1}{j} = nH_n = \Theta(n \log n).$$

2024–25 Spring, Problem 5 (Randomization: coupon collector variants, 15 points)

Problem. Each cereal box holds one of n coupon types uniformly at random. (a)

Find the expected number of boxes needed to collect $n/2$ distinct coupons, in the form $\Theta(n^{\alpha}(\log n)^{\beta})$, with both bounds. (b) After opening n boxes, how many distinct coupons do we expect to have? Give an exact formula (no asymptotics).

Solution (a). Let X_i be the number of boxes to go from i to $i+1$ distinct coupons. When i are collected, a new coupon appears with probability $(n-i)/n$, so X_i is geometric and $\mathbb{E}[X_i] = \dfrac{n}{n-i}$. Hence

$$\mathbb{E}\Big[\sum_{i=0}^{n/2-1} X_i\Big] = n\Big(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{n/2+1}\Big).$$

Bound the bracket:

$$\underbrace{\frac{1}{n} + \cdots + \frac{1}{n/2+1}}_{\text{each term } \geq\ 1/n} \geq \frac{1}{2}, \qquad \frac{1}{n} + \cdots + \frac{1}{n/2+1} \leq \frac{n/2}{n/2} = 1.$$

So the bracket is $\Theta(1)$ and the answer is $\Theta(n)$ — i.e. $\alpha = 1$, $\beta = 0$. (Compare: collecting all n coupons costs $nH_n = \Theta(n \log n)$.)

Solution (b). Let $X_i = \mathbf{1}\{\text{coupon } i \text{ is never seen}\}$. The i-th type is missed iff all n boxes contain something else, so by independence

$$\mathbb{E}[X_i] = \Pr[X_i = 1] = \Big(1 - \frac{1}{n}\Big)^n.$$

Linearity (no independence needed between the X_i) gives the expected number missed $= n(1-1/n)^n$, hence

$$\boxed{\mathbb{E}[\text{distinct}] = n - n\Big(1 - \frac{1}{n}\Big)^n}$$

which is $\approx (1-1/e)n \approx 0.63n$ for large n. □

2025–26 Fall, Problem 3 (Randomization: searching with random-access queries, 10 points)

Problem. A database of n records with distinct floating-point keys supports

$$\text{RANDOM-ACCESS}(l, r) :$$

return a uniformly random record whose key lies in (l, r) (null if none), at cost equal to the number of keys inside (l, r); $l = -\infty$, $r = +\infty$ are allowed. Retrieve the record with key x (or report absence) and bound the expected total cost.

Solution (randomized binary search). Keep $l = -\infty$, $r = +\infty$. Repeat: $R \leftarrow$ RANDOM-ACCESS(l, r); if R = null report "not found"; if key$(R) = x$ return R; if key$(R) < x$ set $r \leftarrow$ key(R), else set $l \leftarrow$ key(R).

Expected cost $O(n)$. Call a query **good** if the returned key lies between the 25th and 75th percentile of the current range — probability $1/2$, and then at least $1/4$ of the candidates are discarded forever. After i good calls the current range has at most $(3/4)^i n$ keys, and the cost of a call is at most the current range size. Let X_i be the number of calls needed to obtain the $(i+1)$-st good call; X_i is geometric with $\mathbb{E}[X_i] = 2$. Then

$$\mathbb{E}[\text{total cost}] \leq \sum_{i \geq 0} \mathbb{E}[X_i]\Big(\tfrac{3}{4}\Big)^i n = 2n \sum_{i \geq 0} \Big(\tfrac{3}{4}\Big)^i \leq 8n = O(n).$$

Note the query cost is not $O(1)$: a single call can inspect $\Theta(n)$ keys, so the argument

must charge each call against the current range size — exactly the "geometric stages" template of Quickselect. □

2025–26 Spring, Problem 2 (Randomization: coupon collector, 10 points)

Problem. (a) Expected boxes to collect $n/2$ distinct coupons, as $\Theta(n^{\alpha}(\log n)^{\beta})$, with both bounds. (b) Expected number of distinct coupons after n boxes, exact formula.

Solution. (a) With X_i geometric, $\mathbb{E}[X_i] = n/(n-i)$, the total is $n\big(\frac{1}{n}+\cdots+\frac{1}{n/2+1}\big)$, and since each of the $n/2$ terms lies in $[\frac{1}{n}, \frac{2}{n}]$ the bracket is in $[\frac{1}{2}, 1]$: the answer is $\Theta(n)$. (b) By indicators, $\mathbb{E}[\text{missed}] = n(1-1/n)^n$, so $\mathbb{E}[\text{distinct}] = n - n(1-1/n)^n \approx 0.63n$.
This is the same paper question as 2024–25 Spring Problem 5; see the full derivation there. The examinable difference: part (a) is $\Theta(n)$ because the harmonic tail saturates after $n/2$ terms, while collecting all n coupons is $\Theta(n \log n)$ — the $\log n$ comes only from the last few rare coupons. □

3.8 Case Study: The Majority Element

Find the element appearing $> n/2$ times, assuming it exists.

Method	Time	Space	Idea
Sort and count	$O(n \log n)$	$O(1)$	majority, if present, occupies the middle position
Divide and conquer	$O(n \log n)$	$O(\log n)$	winners of halves, equality tests only
Hashing	$O(n)$ expected	$O(n)$	frequency map
Try-and-check	$O(n)$ expected	$O(1)$	sample a random element, verify by counting; retry
Boyer–Moore vote	$O(n)$ worst-case	$O(1)$	one cancellation pass, equality tests only

Algorithm 16 Boyer–Moore Majority Vote

```
c ← 0
for i = 1 to n do
    if c = 0 then
        m ← A[i];   c ← 1
    else if m = A[i] then
        c ← c + 1
    else
        c ← c − 1                ▷ cancel A[i] against one stored copy of m
    end if
end for
return m
```

Why the Vote Works

Invariant: after processing $A[1..i]$, the prefix equals "c copies of m plus elements already cancelled in pairs." Each cancellation removes two different items, so any single value can lose at most $n/2$ copies. A true majority ($> n/2$ copies) can never be completely cancelled — it must survive as m. (If a majority is not guaranteed, a second $O(n)$ verification pass is required.)

4 Quicksort

4.1 Deterministic Quicksort (Lomuto Partition)

Pick $A[r]$ as pivot; partition into $<$ pivot (left), $=$ pivot (middle), $>$ pivot (right).

Algorithm 17 Quicksort

```
function QUICKSORT(A, p, r)
    if p < r then
        q ← PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
    end if
end function

function PARTITION(A, p, r)
    pivot ← A[r]
    i ← p − 1
    for j = p to r − 1 do
        if A[j] ≤ pivot then
            i ← i + 1
            swap A[i] ↔ A[j]
        end if
    end for
    swap A[i + 1] ↔ A[r]
    return i + 1
end function
```

Partition Invariants

At the end of each j-iteration in Partition:

- $A[p \dots i] \le pivot$,
- $A[i+1 \dots j] > pivot$,
- $A[j+1 \dots r-1]$ are unprocessed,
- $A[r] = pivot$.

Maintain with swaps; runtime is $\Theta(r - p + 1)$ on subarray size n.

Worked example: Partition $[2, 8, 7, 1, 3, 5, 6, 4]$ with pivot $A[r] = 4$ (Lomuto; bold marks $A[i]$ boundary):

j	$A[j] \le 4$?	i after step	array
start	—	$p-1$	$[2, 8, 7, 1, 3, 5, 6, \mathbf{4}]$
p ($A[j]=2$)	yes	p	$[\mathbf{2}, 8, 7, 1, 3, 5, 6, 4]$
$p+1$ (8)	no	p	$[2, 8, 7, 1, 3, 5, 6, 4]$
$p+2$ (7)	no	p	$[2, 8, 7, 1, 3, 5, 6, 4]$
$p+3$ (1)	yes	$p+1$	$[2, \mathbf{1}, 7, 8, 3, 5, 6, 4]$
$p+4$ (3)	yes	$p+2$	$[2, 1, \mathbf{3}, 8, 7, 5, 6, 4]$
$p+5, p+6$ (5, 6)	no	$p+2$	unchanged
final swap	—	$q = p+3$	$[2, 1, 3, \mathbf{4}, 7, 5, 6, 8]$

Pivot lands at its final sorted position; quicksort then recurses on $[2, 1, 3]$ and $[7, 5, 6, 8]$ independently.

4.2 Quicksort Analysis

Deterministic Quicksort Worst-Case

If the input is **already sorted** and pivot is always the **last** (or first) element:

$$T(n) = T(n-1) + \Theta(n) \quad \Longrightarrow \quad T(n) = \Theta(n^2).$$

Same as insertion sort! This is why we randomize.

Best case: perfectly balanced splits give $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

"Balanced Enough" Splits: 1/4–3/4

Suppose every split were a **worst balanced case**, sizes $n/4$ and $3n/4$:

$$T(n) = T(n/4) + T(3n/4) + cn.$$

The recursion tree has a short left spine (depth $\log_4 n$) and a long right spine (depth $\log_{4/3} n$); every level does at most cn work, so even here

$$T(n) \le cn \cdot \log_{4/3} n = O(n \log n).$$

With a random pivot, $\Pr[\text{pivot rank lies in middle half } [n/4, 3n/4]] = 1/2$ — so on average every other split is good, which is already enough for $\Theta(n \log n)$ expected depth.

Randomized Quicksort Expected Time

Pick pivot **uniformly at random** from $A[p..r]$ (swap it to position r before Partition). Let X = total number of **comparisons** made. Use indicator variables $X_{ij} = 1$ if elements of ranks i and j $(i < j)$ are ever compared.
Key insight: ranks i and j are compared iff **the first pivot chosen from within** $\{i, i+1, \ldots, j\}$ **is either** i **or** j. That happens with probability $\dfrac{2}{j-i+1}$.

$$\mathbb{E}[X] = \sum_{1 \le i < j \le n} \frac{2}{j-i+1} = \sum_{k=2}^{n} \frac{2}{k} \cdot (n-k+1) = 2(n+1)H_n - 4n.$$

So $\mathbb{E}[X] = \Theta(n \log n)$, and therefore expected runtime is $\Theta(n \log n)$.

Alternative View: Pivots Form a Random BST

Imagine inserting elements into a BST in the order in which they become pivots (the first pivot is the root, left-subarray pivots are its left subtree, ...). Two elements are compared exactly when one is an ancestor of the other — the same indicator argument, reinterpreted. The expected depth of the node ranked i is

$$H_i + H_{n-i+1} - 2 \le 2H_n = O(\log n),$$

so the expected total comparisons are $\sum_i O(\log n) = O(n \log n)$. Equivalently, arrange the pair-comparison probabilities in an upper-triangular matrix: every row is a truncated harmonic series of $O(\log n)$ "large" terms $(2/k)$, and all n rows together sum to $O(n \log n)$.

Quicksort Properties

- **Stable:** no (Lomuto and Hoare partitions swap non-adjacent elements).
- **In-place:** yes (only $O(\log n)$ recursion stack; no auxiliary array).

- **Practical speed:** usually faster than Mergesort or Heapsort despite the same $\Theta(n \log n)$ bound, thanks to better cache locality and low constants.

Engineering Quicksort (What Real Libraries Do)

- **Insertion-sort cutoff:** for subarrays of size $\lesssim 16$, switch to insertion sort, which beats recursive quicksort on tiny nearly-sorted ranges.
- **Median-of-three pivot:** choose the median of first/middle/last elements — cheaply defeats already-sorted and reverse inputs.
- **Introsort:** switch to Heapsort after recursion depth exceeds $2\log_2 n$, retaining Quicksort's average speed with a guaranteed $O(n \log n)$ worst case (used by C++ `std::sort`).
- **3-way partition (Dutch national flag):** group equal keys together; without it, arrays with many duplicate keys degenerate toward $\Theta(n^2)$.

4.3 Randomized Selection (Quickselect)

Find the k-th smallest element of an unsorted array (median is the $k = \lceil n/2 \rceil$ case). Reuse Partition: after partitioning on a random pivot, the pivot is at its final rank $k = m - l + 1$ inside the current segment; only **one** recursive call is needed.

Algorithm 18 RandomizedSelect(A, l, r, k) — return the k-th smallest of $A[l..r]$

```
if l = r then
    return A[l]
end if
pick pivot uniformly at random, swap it to r
m ← PARTITION(A, l, r)
pos ← m − l + 1                          ▷ rank of pivot within A[l..r]
if k = pos then
    return A[m]
else if k < pos then
    return RANDOMIZEDSELECT(A, l, m − 1, k)
else
    return RANDOMIZEDSELECT(A, m + 1, r, k − pos)        ▷ relative rank
end if
```

Worked example: find the 4-th smallest of $[7, 2, 9, 1, 4, 8, 3]$ (sorted: $1, 2, 3, \mathbf{4}, 7, 8, 9$).

Segment	k	pivot	result
$[7, 2, 9, 1, 4, 8, 3]$	4	3	partition $\to [2, 1, 3 \mid 7, 4, 8, 9]$, pivot rank $3 < 4$, go right
$[7, 4, 8, 9]$	1	9	rank $4 > 1$, go left
$[7, 4, 8]$	1	8	rank $3 > 1$, go left
$[7, 4]$	1	4	partition $\to [4 \mid 7]$, rank $1 = k$, **return** 4

Expected Time $\Theta(n)$

Call a pivot **good** when its rank lies in the middle half $[n/4, 3n/4]$; this happens with probability $1/2$. After a good pivot the surviving subproblem has size $\leq 3n/4$; bad pivots only add linear partitioning cost. Stages between good pivots are geometric with expected count 2, so the expected total work is

$$2cn + 2c\tfrac{3}{4}n + 2c\left(\tfrac{3}{4}\right)^2 n + \cdots = 2cn \sum_{i \geq 0} \left(\tfrac{3}{4}\right)^i = 8cn = \Theta(n).$$

Worst case remains $\Theta(n^2)$ (probability tiny); the deterministic **median-of-medians** algorithm achieves worst-case $O(n)$ by splitting into groups of 5, whose pivot is guaranteed to have at least $3n/10$ elements on each side: $T(n) = T(n/5)+T(7n/10)+\Theta(n) = O(n)$.

Bonus: k-th of Two Sorted Arrays in $\Theta(\log k)$

Given sorted arrays A, B and rank k, compare $A[\lfloor k/2 \rfloor]$ and $B[\lfloor k/2 \rfloor]$ and discard the smaller half (none of its elements can be the k-th), then recurse with rank $k - \lfloor k/2 \rfloor$. Every step halves k; binary-search variants achieve $O(\log \min(m, n))$.

4.4 Hoare's Partition (Alternative)

Two pointers moving toward each other from opposite ends; swap inversions. Does slightly fewer swaps than Lomuto. Pivot is not guaranteed to end at index q; recur on $A[p..q]$ and $A[q+1..r]$, **skipping** q.

5 Heapsort

5.1 Priority Queue Motivation

A **priority queue** stores n elements, each with a priority (key), and supports `Insert` and `Extract-Max` (min version: negate priorities). Concrete uses: OS process scheduling (the `top` table — new/preempted processes are inserted; when the CPU is free, the lowest-PR process is extracted), event-driven simulation, Dijkstra's shortest paths, and streaming top-k.

Implementation	Insert	Extract-Max	Notes
Unsorted array/list	$O(1)$	$O(n)$	extract must scan everything
Sorted array/list	$O(n)$	$O(1)$	insert requires a shift
Balanced BST	$O(\log n)$	$O(\log n)$	correct bound but large pointer/balance overhead
Binary heap	$O(\log n)$	$O(\log n)$	compact array, no pointers, small constants

5.2 Heap Data Structure

A (binary) **max-heap** is a complete binary tree satisfying:

$$A[\text{Parent}(i)] \geq A[i] \quad \forall i.$$

(Min-heap: $\leq$ instead.)

Array Representation of a Heap

Given 1-indexed array $A[1..n]$:

$$\begin{aligned} \text{Parent}(i) &= \lfloor i/2 \rfloor, \\ \text{Left}(i) &= 2i, \\ \text{Right}(i) &= 2i+1. \end{aligned}$$

A complete binary tree on n nodes has height $\lfloor \lg n \rfloor$.

Height derivation. A complete tree of height h has all levels $0, \ldots, h-1$ full ($2^h - 1$ nodes) and at least one node on level h; hence

$$2^0 + 2^1 + \cdots + 2^{h-1} = 2^h - 1 < n \quad \Longrightarrow \quad n \leq 2^0 + 2^1 + \cdots + 2^h = 2^{h+1} - 1,$$

so $2^h \leq n < 2^{h+1}$ and $h = \lfloor \lg n \rfloor$. This $\Theta(\log n)$ height is why every heap operation is fast.

2025–26 Fall, Problem 2(a) (Min-heap insertions, 3 points)

Problem. Insert $4, 3, 8, 2, 5, 6, 7, 9$ in this order into an initially empty **min**-heap, drawing the tree after each insertion.
Solution. Appending then bubbling up (array form shown; identical to the tree):

Insert	Array	Action	Bubble-up path
4	$[4]$	root	—
3	$[3, 4]$	swap with parent 4	3 beats 4
8	$[3, 4, 8]$	parent $4 < 8$	none
2	$[2, 3, 8, 4]$	swap with 4, then with 3	$2 < 4$, $2 < 3$
5	$[2, 3, 8, 4, 5]$	parent $3 < 5$	none
6	$[2, 3, 6, 4, 5, 8]$	swap with 8	$6 < 8$, then $3 < 6$
7	$[2, 3, 6, 4, 5, 8, 7]$	parent $6 < 7$	none
9	$[2, 3, 6, 4, 5, 8, 7, 9]$	parent $4 < 9$	none

Each insertion is $O(\log n)$ (one root-to-leaf path). Final tree: 2 over $\{3,6\}$, 3 over $\{4,5\}$, 6 over $\{8,7\}$, 4 over $\{9\}$. □

2025–26 Spring, Problem 4(a) (Min-heap insertions, 8 points)

Problem. Insert $5,8,1,4,7,3,2,6$ in this order into an initially empty min-heap, drawing the tree after each insertion.
Solution.

Insert	Array	Action	Bubble-up path
5	$[5]$	root	—
8	$[5,8]$	parent $5<8$	none
1	$[1,8,5]$	swap with 5	$1<5$
4	$[1,4,5,8]$	swap with 8	$4<8$, then $1<4$
7	$[1,4,5,8,7]$	parent $4<7$	none
3	$[1,4,3,8,7,5]$	swap with 5	$3<5$, then $1<3$
2	$[1,4,2,8,7,5,3]$	swap with 3	$2<3$, then $1<2$
6	$[1,4,2,6,7,5,3,8]$	swap with 8	$6<8$, then $4<6$

Final tree: 1 over $\{4,2\}$; 4 over $\{6,7\}$; 2 over $\{5,3\}$; 6 over $\{8\}$. □

5.3 Max-Heapify (Bubble-Down)

Assume children of node i are heaps, but $A[i]$ may be too small. Fix by swapping $A[i]$ with the **larger** child, recursing down.

Algorithm 19 Max-Heapify

```
function MaxHeapify(A, i, heap_size)
    L ← 2i;    R ← 2i + 1
    largest ← i
    if L ≤ heap_size and A[L] > A[largest] then
        largest ← L
    end if
    if R ≤ heap_size and A[R] > A[largest] then
        largest ← R
    end if
    if largest ≠ i then
        swap A[i] ↔ A[largest]
        MaxHeapify(A, largest, heap_size)
    end if
end function
```

Time: $O(\log n)$ per call (proportional to tree height).

5.4 Build Max-Heap

Naive approach: run Max-Heapify top-down from the **last non-leaf** ($\lfloor n/2 \rfloor$) *down to* root 1. Leaves are trivially heaps of size 1.

Algorithm 20 Build Max-Heap

```
function BuildMaxHeap(A, n)
    for i = ⌊n/2⌋ downto 1 do
        MaxHeapify(A, i, n)
    end for
end function
```

Build-Max-Heap Cost is $O(n)$, Not $O(n \log n)$

Height-h nodes are $\le n/2^{h+1}$; each MaxHeapify on a height-h node costs $O(h)$:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(2n) = O(n).$$

(Building from the leaves up is much cheaper than building from the root down.)

Per-level bookkeeping (take $n = 2^{h+1} - 1$, a perfectly full tree):

Level (root = 0)	# nodes	heapify cost per node	total for level
0	1	h	h
1	2	$h-1$	$2(h-1)$
2	4	$h-2$	$4(h-2)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$h-i$	$2^i(h-i)$
h	2^h	0	0

Summing with $j = h - i$:

$$\sum_{i=0}^{h-1} 2^i(h-i) = 2^h \sum_{j=1}^{h} \frac{j}{2^j} = 2^h\left(2 - \frac{h+2}{2^h}\right) = 2^{h+1} - h - 2 < n,$$

using $\sum_{j=1}^{h} j/2^j = 2 - (h+2)/2^h$. Nearly all the work is at the bottom levels; the root level costs only h.

2025–26 Fall, Problem 2(b) (Merging equal-sized heaps, 3 points)

Problem. Given 2^{m-k} min-heaps $H_1, \ldots, H_{2^{m-k}}$, each with $2^k - 1$ elements and stored with parent/child pointers, plus 2^{m-k-1} new numbers in nodes $v_1, \ldots, v_{2^{m-k-1}}$, merge everything into 2^{m-k-1} min-heaps of size $2^{k+1} - 1$ in $O(k\,2^{m-k-1})$ time.

Solution. For each i, make v_i the parent of the roots of H_{2i-1} and H_{2i}, which yields a binary tree of size $1 + 2(2^k - 1) = 2^{k+1} - 1$.

1. Compare $x = \text{value}(v_i)$ with the two children roots; if x is no larger, the subtree is already a min-heap.
2. Otherwise swap x with the smaller of the two child roots and continue downward exactly as MinHeapify: at each level compare with the two children and swap into the smaller one.
3. Stop when x is $\le$ both children or x has reached a leaf.

Correctness. The only violation of the heap order is at the node holding x; all other subtrees are untouched valid heaps, so one bubble-down fixes everything (the standard Heapify precondition).

Running time. Each level costs $O(1)$ and there are $O(k)$ levels, so $O(k)$ per merge; over 2^{m-k-1} merges, $O(k\,2^{m-k-1})$. □

2025–26 Fall, Problem 2(c) (Linear-time build-heap from scratch, 4 points)

Problem. Let $n = 2^m - 1$, $m \ge 2$. Using (b), build a min-heap of n numbers in $O(n)$ time, proving the bound from scratch.

Solution. Start with the first 2^{m-1} numbers as 2^{m-1} heaps of size $2^1 - 1 = 1$. For $k = 1, 2, \ldots, m-1$: suppose 2^{m-k} heaps of size $2^k - 1$ hold the first $2^{m-1} + 2^{m-2} + \cdots + 2^{m-k}$

numbers; take the next 2^{m-k-1} numbers as the nodes $v_1, \ldots$ and apply (b) to obtain 2^{m-k-1} heaps of size $2^{k+1}-1$. After the last iteration ($k = m-1$) one heap holding all n numbers remains.

Time. The total is

$$\sum_{k=1}^{m-1} k\, 2^{m-k-1} = 2^m \sum_{k=1}^{m-1} \frac{k}{2^{k+1}} \;\le\; 2^m = n+1 = O(n),$$

because for $|x| < 1$, $S(x) = \sum_{k\ge 1} x^k = \dfrac{x}{1-x}$ gives $S'(x) = \sum_{k\ge 1} k x^{k-1} = \dfrac{1}{(1-x)^2}$, so $\sum_{k\ge 1} \dfrac{k}{2^{k+1}} = \dfrac{1}{4} S'(\tfrac{1}{2}) = 1$. Hence $O(n)$ — the same $O(n)$ as the array-based BUILD-MAX-HEAP, but here the tree is pointer-based and we never need random access. □

5.5 Heapsort Algorithm

(1) Build max-heap on $A[1..n]$. (2) Repeatedly: swap max (root $A[1]$) with last element, shrink heap, MaxHeapify root.

Algorithm 21 Heapsort

```
function HEAPSORT(A, n)
    BUILDMAXHEAP(A, n)
    for i = n downto 2 do
        swap A[1] ↔ A[i]
        heap_size ← i − 1
        MAXHEAPIFY(A, 1, heap_size)
    end for
end function
```

Heapsort Analysis

- Build: $O(n)$.
- $n-1$ swaps each followed by MaxHeapify: $(n-1)\,O(\log n) = O(n \log n)$.
- **Total:** $O(n \log n)$ (best = worst = average, unlike Quicksort).
- **Stable?** No.
- **In-place?** Yes (only $O(1)$ extra storage, no recursion stack if MaxHeapify is written iteratively).

5.6 Priority Queue Operations (Applications of Heaps)

Max-priority queue supports: `Maximum`, `ExtractMax`, `IncreaseKey`, `Insert`.

Algorithm 22 Priority Queue Operations

```
function ExtractMax(A, heap_size)
    max ← A[1]
    A[1] ← A[heap_size];    heap_size ← heap_size − 1
    MaxHeapify(A, 1, heap_size)
    return max
end function

function IncreaseKey(A, i, key)
    if key < A[i] then error
    end if
    A[i] ← key
    while i > 1 and A[Parent(i)] < A[i] do                ▷ bubble up
        swap A[i] ↔ A[Parent(i)]
        i ← Parent(i)
    end while
end function

function Insert(A, key, heap_size)
    heap_size ← heap_size + 1
    A[heap_size] ← −∞
    IncreaseKey(A, heap_size, key)
end function
```

Each operation runs in $O(\log n)$ time.

Extract-Max trace on $[42, 29, 18, 14, 7, 6]$:

initial	save $m = 42$, move 6 to root	$[\mathbf{6}, 29, 18, 14, 7]$
bubble down	$6 < 29$, swap with larger child	$[\mathbf{29}, 6, 18, 14, 7]$
bubble down	$6 < \max(14, 7) = 14$, swap with 14	$[29, \mathbf{14}, 18, 6, 7]$

Returned 42; heap is now $[29, 14, 18, 6, 7]$.

Insert trace (bubble up): insert 20 into $[29, 14, 18, 6, 7]$:

append	$[29, 14, 18, 6, 7, \mathbf{20}]$	parent of 20 is 18
swap up	$20 > 18$	$[29, 14, \mathbf{20}, 6, 7, 18]$
stop	parent is $29 > 20$	done

2024–25 Spring, Problem 4 (Heaps, 10 points)

Problem. Maintain m sets $S_1, \ldots, S_m$ supporting
- FindMin: return $\min(S_1 \cup \cdots \cup S_m)$, or null if empty;
- Add(x, i): insert x into S_i.

Each $n_i = |S_i| \le N$. Achieve FindMin in $O(1)$ and Add in $O(\log m + \log n_i)$.

Solution (heap of heap-minima + position map). Keep one min-heap H (as array $B[1..m]$) holding the pair (a_i, i) where $a_i = \min S_i$, ordered by a_i. Store each set as a pair (A_i, p_i): A_i is an array of length N holding the elements of S_i as a heap, and $p_i \in [1, m]$ is the position in H where the pair (a_i, i) currently sits.
- FindMin: if H is empty return null, else return the key stored at $B[1]$ — $O(1)$.
- Add(x, i): let $y = A_i[1]$ (the current minimum of S_i). Insert x into the heap A_i in $O(\log n_i)$. If $x < y$ the minimum changed: overwrite $B[p_i] \leftarrow (x, i)$ and **bubble up** while x is smaller than its parent. During each swap, if a pair (a_j, j) moves to position $B[k]$, update $p_j \leftarrow k$. There are $O(\log m)$ levels.

Total: $O(\log n_i) + O(\log m)$ per Add. The subtlety the marker looks for: the position

map must be repaired on every swap in H, otherwise ADD would have to search for (a_i, i) and the bound would break. □

2025–26 Spring, Problem 4(b) (Largest $\lceil \log_2 n \rceil$ elements in $O(n \log \log n)$, 12 points)

Problem. $A[1..n]$ is unsorted and distinct. Return its largest $\lceil \log_2 n \rceil$ elements in $O(n \log \log n)$ time, using any algorithm taught in class.

Solution (block sort + heap of block maxima). Let $L = \log_2 n$.

1. Split A into n/L blocks of L consecutive elements and sort each block increasing in $O(L \log L) = O(\log n \log \log n)$; total
$$\frac{n}{L} \cdot O(L \log L) = O(n \log \log n).$$
2. Build a max-heap H containing the last (largest) element of each block, together with its block index; each block keeps a pointer to its current position. Heap construction costs $O(n/L)$ insertions of $O(\log(n/L)) = O(\log n) \Rightarrow O(n)$.
3. Repeat L times: extract the maximum v from H (it is the next largest overall), output v, move that block's pointer one step left and insert the new block maximum. Each step is $O(\log(n/L)) = O(\log n)$, so this stage costs $O(L \log n) = O(\log^2 n)$.

The first stage dominates: $O(n \log \log n)$ total. **Why the pointers matter:** the heap must return the largest remaining value of a block, so after consuming one element we refill from the same block — the standard "merge k sorted lists" trick with $k = n/L$. □

5.7 Applications

5.7.1 Merging k Sorted Arrays

Combine k sorted arrays of length n into one sorted sequence of length kn.

Solution	Time	Idea
Sort everything	$O(kn \log(kn))$	ignore existing order
Divide & conquer	$O(kn \log k)$	pairwise 2-way merges, $\log_2 k$ rounds of kn work
Min-heap	$O(kn \log k)$	keep one current head per array in a heap of size k

Heap method: insert the first element of each array (tagged with its source array) into a min-heap; repeatedly extract-min and insert the next element from the same source array. Each of the kn elements costs one insert + one extract, each $O(\log k)$. Uses only $O(k)$ extra space, and it works for online/streaming inputs and huge files (external merge sort).

5.7.2 Delete and Update-Priority

Store `(key, priority, payload)` tuples, where `key` uniquely identifies the record.

- `Delete(key)`: locate its heap index k, swap $A[k]$ with $A[n]$, shrink, then **bubble up or bubble down** from k (either direction may be needed) — $O(\log n)$ once located.
- `UpdatePriority(key, p)`: set the new priority and bubble in the appropriate direction.

Locating an element by key requires a separate **position map** (hash table: key → heap index), updated on every swap. It preserves the $O(\log n)$ bounds but adds real overhead,

which is why C++/Python standard heaps do not support it directly (`std::set` or lazy deletion are common workarounds).

5.7.3 The Skyline Problem

Given buildings (x_L, x_R, y), output the skyline key points (positions where the maximum height changes).

- **D&C solution:** recursively get the skylines of two halves; merge them like merge-sort while tracking current heights h_1, h_2 of each side. At each next boundary $x = \min(x_1, x_2)$, advance that side and append $(x, \max(h_1, h_2))$ only when it differs from the last emitted height. An $O(n)$ merge gives $T(n) = 2T(n/2) + \Theta(n) = O(n \log n)$.
- **Heap (sweep-line) solution:** sort all vertical boundaries $(x, y, \text{id}, L/R)$; sweep left to right, inserting a building at its left edge and deleting it at its right edge from a max-heap of live heights. The heap top is the current skyline height; emit a key point whenever it changes. $O(n \log n)$.

6 Linear-Time Sorting

All comparison-based sorts have *worst-case* $\Omega(n \log n)$ lower bound (decision-tree height). The following beat the bound by **not comparing keys directly**.

When Can We Beat $\Omega(n \log n)$?

We need extra information about the keys: their values lie in a **small integer range**, or they have structure that allows *digit-wise* processing.

6.1 The Decision-Tree Model and the Sorting Lower Bound

A **comparison-based** algorithm makes decisions only from pairwise comparisons ("$a_i \le a_j$?"); its behavior depends solely on the relative order, never on actual values. Merge, heap and quick sort are comparison-based; counting/radix/bucket sort are not.

Represent any execution as a **decision tree**: each internal node is one comparison with two outcomes (left/right branch); each leaf is one possible output. The **worst-case number of comparisons is the height** of the tree, and a binary tree with N leaves has height at least $\lceil \log_2 N \rceil$.

Warm-up: Lower Bound for Binary Search

Searching a sorted array of size n has $n+1$ distinct outputs — one per element, plus "not found." Hence the decision tree needs $\ge n+1$ leaves and height $\ge \log_2(n+1) = \Omega(\log n)$. Binary search matches this bound, so it is asymptotically optimal.

Exercise: The Defective Coin Balance

Among n coins, at most one is too heavy or too light (unknown which); a three-way balance reports left-heavy / balanced / right-heavy.

- **Lower bound:** there are $2n+1$ outcomes (n heavy, n light, all genuine), so the ternary decision tree has $\ge 2n+1$ leaves: height $\ge \log_3(2n+1)$ weighings.
- **Algorithm:** split into three equal groups C_1, C_2, C_3. Two weighings (C_1 vs C_2, then C_1 vs C_3) distinguish the 7 outcomes $C_1H, C_1L, C_2H, C_2L, C_3H, C_3L$, "none"; recurse into the defective group:

$$T(n) = T(n/3) + 2 = 2\log_3 n = O(\log n),$$

matching the lower bound up to constants.

Theorem: Sorting Requires $\Omega(n \log n)$ Comparisons

A sorting algorithm must be able to output any of the $n!$ input permutations, so its decision tree has $\ge n!$ leaves and height

$$h \ge \log_2(n!) = \sum_{i=1}^{n} \log_2 i \ge \sum_{i=\lceil n/2 \rceil}^{n} \log_2 i \ge \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$$

(equivalently use Stirling: $n! \ge (n/e)^n$, so $\log_2 n! = \Theta(n \log n)$). Therefore **every** comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case — Merge Sort and Heapsort are thus asymptotically optimal among comparison sorts. To do better we must leave the comparison model.

Algorithm vs problem lower bounds. "Insertion sort is $\Omega(n^2)$" is an algorithm bound (a particular algorithm's cost); "sorting is $\Omega(n \log n)$" is a problem bound (no comparison-based algorithm can beat it, proved independently of how clever the algorithm is). Problem lower bounds are the rarer and more valuable kind — they tell us when to

stop improving and point toward new models (non-comparison methods, randomization, average-case assumptions).

2024–25 Spring, Problem 1 (Decision tree, 5 points)

Problem. The following recursive algorithm returns the minimum of $A[1..n]$ of distinct numbers; the top-level call is $\textsc{Smallest}(A, 1, n)$.

function $\textsc{Smallest}(A, p, r)$
  if $p = r$ **then**
    return $A[p]$
  end if
  $q \leftarrow \lfloor (p + r)/2 \rfloor$
  $x \leftarrow \textsc{Smallest}(A, p, q)$; $y \leftarrow \textsc{Smallest}(A, q + 1, r)$
  return $\min\{x, y\}$
end function

Draw the decision tree of this algorithm for four distinct inputs a_1, a_2, a_3, a_4.

Solution. The recursion splits $\{a_1, a_2, a_3, a_4\}$ into $\{a_1, a_2\}$ and $\{a_3, a_4\}$; each half is resolved with **one** comparison, and the two winners are compared once more. Every root–leaf path therefore contains exactly 3 comparisons:

Level	Comparison made	Branching
1	a_1 vs. a_2	winner is a_1 or a_2
2	a_3 vs. a_4	winner is a_3 or a_4
3	winner of left vs. winner of right	4 combinations: a_1:a_3, a_1:a_4, a_2:a_3, a_2:a_4

The tree has 4 leaves (one per candidate minimum) and height 3, so the worst case is $3 = n - 1$ comparisons. This matches the information-theoretic bound: to certify a minimum one must "eliminate" the other $n - 1$ elements, each elimination costing one comparison. □

2025–26 Fall, Problem 1(b) (Lower bound for the k smallest, 4 points)

Problem. $A[1..n]$ holds n distinct integers and $1 \le k \le n/2$. Prove that outputting the k smallest numbers in increasing order requires $\Omega(k \log n)$ comparisons in the worst case.

Solution (decision tree / counting outputs). A valid output is a sequence of k distinct values that are the k smallest. Choose the set: $\binom{n}{k}$ ways; choose their increasing order: $k!$ ways. So the number of possible outputs is

$$\binom{n}{k} k! = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1) \;\ge\; (n-k+1)^k.$$

Any binary decision tree with N leaves has height $\ge \log_2 N$, hence

$$h \ge \log_2 (n-k+1)^k = k \log_2 (n-k+1).$$

Since $k \le n/2$ we have $n - k + 1 > n/2$, so $h \ge k(\log_2 n - 1) = \Omega(k \log n)$. Why the order matters: dropping "in increasing order" would remove the $k!$ factor and destroy the bound — the multiset alone can be certified with far fewer comparisons. □

6.2 Counting Sort

Assume each of n input keys is an integer in $\{0, 1, \ldots, k\}$.

Algorithm 23 Counting Sort

```
function CountingSort(A, B, n, k)
    for i = 0 to k do
        C[i] ← 0
    end for
    for j = 1 to n do
        C[A[j]] ← C[A[j]] + 1                          ▷ count frequencies
    end for
    for i = 1 to k do
        C[i] ← C[i] + C[i − 1]                         ▷ prefix sums = "how many ≤ i"
    end for
    for j = n downto 1 do
        B[C[A[j]]] ← A[j]
        C[A[j]] ← C[A[j]] − 1
    end for
end function
```

Counting Sort Properties

- Runtime: $\Theta(n+k)$. If $k = O(n)$ then $\Theta(n)$ overall.
- **Stable:** yes (fill B *right-to-left* so equal keys preserve input order). This is critical for being used as a subroutine in Radix Sort.
- **In-place:** no (uses arrays C of size $k+1$ and output B of size n).

Worked example: $A = [2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$.

i (key value)	0	1	2	3	4	5
frequencies C	2	0	2	3	0	1
prefix sums C	2	2	4	7	7	8

Placing right-to-left (equal keys keep their order): $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

6.3 Radix Sort

Sort d-digit keys by applying a **stable** sort digit-by-digit, least significant digit first (LSD). Each digit sort is Counting Sort on that digit.

Algorithm 24 Radix Sort

```
function RadixSort(A, d, n)
    for i = 1 to d do
        Use a stable sort to sort A on digit i (the i-th least significant)
    end for
end function
```

Radix Sort Correctness & Complexity

- Why LSD works: after pass i, numbers are sorted correctly w.r.t. their low i digits; next pass (higher digit) is stable so ties keep the lower-order ordering.
- Each digit in $\{0, \ldots, k-1\}$ sorted by Counting Sort in $\Theta(n+k)$.
- Total: $\Theta(d(n+k))$. If d is constant and $k = O(n)$, this is $\Theta(n)$.
- **Stable:** yes (because inner sort is stable).

Radix Sort Gotchas

- Must process digits **from LSD to MSD** (not the other way around).

- Inner sort **must be stable**; otherwise equal-digit elements lose their previously-computed order.

Correctness by Induction on Digit Position

Claim: after pass i, keys are sorted by their low-order i digits. Base $i = 0$ vacuous. Assume sorted by low $i-1$ digits; after stably sorting on digit i:

- two keys differing in digit i are correctly ordered by that more significant digit;
- two keys equal in digit i keep their pre-pass order by **stability**, which is correct ordering by the low $i-1$ digits (induction hypothesis).

Both cases give correctness by low i digits; after d passes the keys are fully sorted.

Worked example (3 LSD passes):

input	$329, 457, 657, 839, 436, 720, 355$
pass 1 (units)	$720, 355, 436, 457, 657, 329, 839$
pass 2 (tens)	$720, 329, 436, 839, 355, 457, 657$
pass 3 (hundreds)	$329, 355, 436, 457, 657, 720, 839$

Bit-Parameter Analysis (w-bit Integers)

Each "digit" spans $\log_2 k$ bits, so the number of passes is $d = \lceil w / \log k \rceil$ and total time is

$$\Theta\Big(\lceil \tfrac{w}{\log k} \rceil (n + k)\Big).$$

The $(n + k)$ factor is minimized at $k = n$ (equal split between pass count and bucket overhead), giving $\Theta(\lceil w/\log n \rceil n)$. In the standard word-RAM regime $w = O(\log n)$ (pointers/indices fit in a machine word; all machines from 8-bit to 64-bit satisfy this for realistic RAM sizes), radix sort runs in $\Theta(n)$. It also extends to fixed-length strings and to floating-point keys (their IEEE-754 bit order is monotone after a sign flip).

2025–26 Fall, Problem 1(c) (Radix sort passes, 4 points)

Problem. Run radix sort (LSD first) on the five 4-digit numbers below and give the array after each pass:

$$9341, \quad 6275, \quad 4194, \quad 7819, \quad 3532.$$

Solution. Each pass is a stable counting sort on one digit.

Pass	Digit sorted	Order of that digit	Array afterwards
1 (units)	$1, 5, 4, 9, 2$	$1, 2, 4, 5, 9$	9341, 3532, 4194, 6275, 7819
2 (tens)	$4, 3, 9, 7, 1$	$1, 3, 4, 7, 9$	7819, 3532, 9341, 6275, 4194
3 (hundreds)	$8, 5, 3, 2, 1$	$1, 2, 3, 5, 8$	4194, 6275, 9341, 3532, 7819
4 (thousands)	$4, 6, 9, 3, 7$	$3, 4, 6, 7, 9$	3532, 4194, 6275, 7819, 9341

The fourth pass yields the sorted order. Check the stability requirement: in pass 2, 9341 and 4194 both have tens digit 4 and 9; their relative order inherited from pass 1 must be preserved — a non-stable inner sort would silently break correctness. Total time $\Theta(d(n + k))$ with $d = 4$, $k = 10$. □

6.4 Bucket Sort

Assume input is drawn **uniformly at random** from $[0, 1)$. Distribute into n equal-sized buckets, sort each bucket (e.g. insertion sort), concatenate.

Algorithm 25 Bucket Sort

```
function BucketSort(A, n)
    Create B[0..n − 1] (each an empty list)
    for i = 1 to n do
        insert A[i] into B[⌊n · A[i]⌋]
    end for
    for i = 0 to n − 1 do
        sort list B[i] (e.g. insertion sort)
    end for
    Concatenate B[0], B[1], . . . , B[n − 1] in order
end function
```

Bucket Sort Expected Time

With uniform input, each bucket has $O(1)$ elements in expectation; sorting each by insertion sort is $O(1)$ per bucket.

$$\mathbb{E}[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(1) = \Theta(n).$$

Worst case (all elements fall in one bucket): $\Theta(n^2)$ if inner sort is insertion sort.

Why the Expected Cost Is Linear: $\mathbb{E}[X_0^2] = O(1)$

Let X_i be the number of elements in bucket i and $Y_j = \mathbf{1}\{A[j] \text{ lands in bucket } 0\}$, so $X_0 = \sum_{j=1}^{n} Y_j$. Insertion sort on bucket i costs $\Theta(X_i^2)$; by identical distributions,

$$\mathbb{E}\left[\sum_{i=0}^{n-1} X_i^2\right] = n\,\mathbb{E}[X_0^2].$$

Expand the square and use independence of distinct Y_j ($\Pr[Y_j = 1] = 1/n$):

$$\mathbb{E}[X_0^2] = \sum_j \mathbb{E}[Y_j^2] + 2\sum_{j<\ell} \mathbb{E}[Y_j Y_\ell] = n \cdot \frac{1}{n} + 2\binom{n}{2}\frac{1}{n^2} = 1 + 1 - \frac{1}{n} = 2 - \frac{1}{n} = O(1).$$

So bucket sorting costs $O(n)$ expected time total (plus the $O(n)$ distribution/concatenation passes). If inputs are not in $[0, 1)$, normalize first; with non-uniform distributions a custom bucket partition is needed.

Sorting Algorithm Cheat Sheet

Algorithm	Worst	Avg.	Space	Comp.	Stab.	Cache	Par.
Insertion	n^2	n^2	1	yes	yes	good	no
Mergesort	$n \log n$	$n \log n$	n	yes	yes	good	good
Quicksort	n^2	$n \log n$	$\log n$	yes	no	good	good
Heapsort	$n \log n$	$n \log n$	1	yes	no	bad	no
Counting	$n + k$	$n + k$	$n + k$	no	yes	bad	no
Radix	$\frac{w}{\log n} n$	same	n	no	yes	bad	good
Bucket	n^2	n (exp.)	n	no	yes	bad	good

2025–26 Fall, Problem 1(d) (Choosing the sorting algorithm, 3 points)

Problem. Among selection/insertion/merge/heap/quick/counting/radix sort, which gives the smallest worst-case running time, and what is it?

(i) n integers in $[1, n]$;

(ii) n floating-point numbers with $O(\sqrt{\log n})$ inversions;

(iii) n arbitrary floating-point numbers.

Solution.

Case	Structure exploited	Best algorithm	Worst case
(i)	keys are integers in a size-n range	counting sort	$O(n)$
(ii)	nearly sorted (few inversions)	insertion sort	$O(n\sqrt{\log n})$
(iii)	comparison model only (no small range)	merge sort or heapsort	$O(n \log n)$

Reasoning: (i) counting sort runs in $\Theta(n + k)$ with $k = n$. (ii) insertion sort costs $\Theta(n + I)$ where I is the inversion count, and no comparison sort can beat $\Omega(n + I)$; quicksort/mergesort ignore the structure and stay $\Theta(n \log n)$. (iii) floating-point values are not restricted to a small integer range, so the $\Omega(n \log n)$ comparison lower bound applies and must be met (quicksort is only $O(n \log n)$ average). □

7 Greedy Algorithms

A **greedy algorithm** always makes the choice that looks best at the moment and adds it to the current partial solution. It builds a solution to an optimization problem through a sequence of choices that are:

- **Feasible:** satisfying the given constraints.
- **Locally optimal:** make the choice that looks best right now, without considering the future.
- **Irrevocable:** once a choice is made it is never undone (no backtracking).

When Does Greedy Work?

- A greedy algorithm is correct when the problem has the **greedy-choice property** (a locally optimal choice can be part of some global optimum) together with **optimal substructure** (an optimal solution contains optimal solutions to subproblems).
- When they work, greedy algorithms are usually the simplest and most efficient algorithms available. A sorting step often dominates the cost, giving $\Theta(n \log n)$.
- When they fail, they may still give fast **approximations**.

Greedy Correctness Must Be Proven!

"Picking the locally best option" is not automatically optimal. The correctness (optimality) of a greedy algorithm is usually non-obvious. Two standard proof techniques:

1. **Exchange argument:** transform any optimal solution into the greedy solution one choice at a time, without worsening it (Interval Scheduling, Fractional Knapsack, Huffman).
2. **Greedy stays ahead / match a lower bound:** show the greedy solution reaches a universal lower bound (Interval Partitioning vs. depth).

7.1 Warm-up: Change-Making

Input: amount n and coin denominations $d_1 > d_2 > \cdots > d_m$. Output: the minimum number of coins summing to n.

Greedy rule: give as many coins as possible of the largest denomination, then the second largest, and so on.

Example (Hong Kong coins): $n = 19$, denominations $10, 5, 2, 1$.

$$19 = 10 + 5 + 2 + 2 \quad \Longrightarrow \quad 4 \text{ coins, and this is optimal.}$$

Greedy Change-Making Is Not Always Optimal

British coinage before 1971 used denominations $1, 3, 6, 12, 24, 30, 60$. For $n = 48$:

- Greedy: $48 = 30 + 12 + 6$ (3 coins).
- Optimal: $48 = 24 + 24$ (2 coins).

The greedy choice (take 30) "steals" capacity that two better coins could have used. For arbitrary denominations, change-making requires **dynamic programming**.

7.2 Warm-up: Max Sum of Non-Adjacent Elements

Given n positive numbers $A[1], \ldots, A[n]$, pick a subset of **pairwise non-adjacent** elements with maximum total sum. Example: $A = (1, 8, 6, 3, 6) \Rightarrow$ pick $A[2], A[5]$ with sum $8 + 6 = 14$.

Greedy attempt: repeatedly take the largest remaining element, then delete it and its two neighbors.

Algorithm 26 Greedy Non-Adjacent Selection (Not Optimal!)

```
S ← ∅
while some elements remain in A do
    pick index i with the largest remaining A[i]
    S ← S ∪ {A[i]}
    mark A[i − 1], A[i], A[i + 1] as deleted
end while
return S
```

Counterexample: $(1, 4, 6, 4)$

- Greedy takes the maximum 6, which forces deleting both neighbors 4 and 4, then takes 1: sum $= 6 + 1 = 7$.
- Optimal: take the two 4's (positions 2 and 4): sum $= 8 > 7$.

The optimal solution is given by DP: $\mathrm{OPT}(i) = \max\big(\mathrm{OPT}(i-1),\ \mathrm{OPT}(i-2) + A[i]\big)$.

7.3 Interval Scheduling

Job j starts at s_j and finishes at f_j. Two jobs are **compatible** if their intervals do not overlap. Goal: select a **maximum-size** subset of mutually compatible jobs.

7.3.1 Three Tempting Greedy Rules (All Fail)

Counterexamples

1. **Earliest start time.** One long job $e = [0, 10)$ starting first can block four short jobs $[1, 2), [2, 3), [3, 4), [4, 5)$: greedy chooses $\{e\}$ (size 1) instead of $\{a, b, c, d\}$ (size 4).
2. **Shortest interval.** Let $a = [0, 4)$, $b = [4, 8)$, and a very short $c = [3.5, 4.5)$ overlapping both. Shortest-first chooses $\{c\}$ (size 1) instead of $\{a, b\}$ (size 2).
3. **Fewest conflicts.** Count each job's overlapping jobs c_j and schedule in ascending c_j. A job conflicting with only a few others can still sit in the middle of a long compatible chain and, once chosen, force several of them out — the slide example forces $\{a, f, d\}$ (size 3) instead of $\{a, b, c, d\}$ (size 4). The number of conflicts is a global statistic and does not measure local opportunity cost.

7.3.2 The Correct Rule: Earliest Finish Time

Greedy algorithm: consider jobs in increasing order of finish time; take a job whenever it is compatible with the jobs already chosen. Intuition: finishing early **leaves the maximum remaining time** for all later jobs.

Algorithm 27 Interval Scheduling (Earliest Finish Time)

```
Sort jobs so that f_1 ≤ f_2 ≤ ··· ≤ f_n
A ← ∅;   last ← 0                          ▷ finish time of the last selected job
for j = 1 to n do
    if s_j ≥ last then                     ▷ compatible: ends are inclusive, so ≥ is fine
        A ← A ∪ {j};   last ← f_j
    end if
end for
return A
```

Running time: $\Theta(n \log n)$, dominated by sorting; the scan itself is $\Theta(n)$.

Worked trace. Jobs sorted by finish time:

$$j_1 = [0, 2),\ j_2 = [1, 4),\ j_3 = [3, 5),\ j_4 = [4, 6),\ j_5 = [5, 7).$$

- Select j_1 (earliest finish), set $last = 2$.
- Skip j_2: $s_2 = 1 < 2$, overlaps j_1.
- Select j_3: $s_3 = 3 \geq 2$, set $last = 5$.
- Skip j_4: $s_4 = 4 < 5$.
- Select j_5: $s_5 = 5 \geq 5$.

Greedy returns $\{j_1, j_3, j_5\}$, size 3 — the maximum possible.

7.3.3 Correctness Proof (Exchange Argument)

Theorem: Earliest-Finish-Time Greedy Is Optimal

Let $G = (i_1, i_2, \ldots, i_k)$ be the jobs selected by greedy and $O = (j_1, j_2, \ldots, j_m)$ be any optimal schedule, both in selection order.

1. Suppose $G \neq O$. Find the largest r such that $i_1 = j_1, \ldots, i_r = j_r$ (the common prefix; $r = 0$ if they differ immediately).
2. By the greedy rule, i_{r+1} is the compatible job with the **earliest finish**, so $f_{i_{r+1}} \leq f_{j_{r+1}}$.

3. Build O^* from O by replacing j_{r+1} with i_{r+1}. Because i_{r+1} finishes no later than j_{r+1}, it is still compatible with $j_{r+2}, \ldots, j_m$; hence O^* is a feasible schedule with the same size as O, so O^* is also optimal.
4. O^* now shares the first $r+1$ jobs with G. Repeat the replacement; the common prefix grows each time, so eventually O is transformed into G with its size (value) unchanged.

Therefore $|G| = |O|$, i.e. greedy is optimal. Equivalently, the greedy schedule "stays ahead": the r-th greedy job finishes no later than the r-th job of any schedule, so greedy can always fit at least as many jobs.

7.4 Exercise: Interval Covering

Given n intervals that together cover $[0, 1]$; interval x has start $x.s$ and finish $x.f$. Find the **minimum** number of intervals whose union covers $[0, 1]$. (Assume all endpoints are distinct.)

Greedy idea: maintain the rightmost covered point x. Among all intervals covering x ($I.s \le x < I.f$), pick the one extending **furthest to the right**; advance x to its finish and repeat until $x \ge 1$.

Algorithm 28 Greedy Interval Covering

```
x ← 0;   G ← ∅
while x < 1 do
    pick interval I with I.s ≤ x that maximizes I.f
    G ← G ∪ {I};   x ← I.f
end while
return G
```

Worked trace. Intervals $A = [0, 0.3]$, $B = [0.2, 0.6]$, $C = [0.5, 0.8]$, $D = [0.75, 1]$:

- $x = 0$: only A covers 0 $\Rightarrow$ take A, $x = 0.3$.
- $x = 0.3$: B covers 0.3 and extends furthest $\Rightarrow$ take B, $x = 0.6$.
- $x = 0.6$: C covers 0.6 (D starts at 0.75) $\Rightarrow$ take C, $x = 0.8$.
- $x = 0.8$: take D, $x = 1$. Done with 4 intervals.

Optimality Sketch & Implementation

Let $O = (o_1, \ldots, o_m)$ be optimal and $G = (g_1, \ldots, g_k)$ greedy, ordered by start time. Let i be the first index with $o_i \ne g_i$. The greedy choice extends at least as far: $g_i.f \ge o_i.f$. Replacing o_i by g_i keeps the cover valid; iterating converts O into G, so $k \le m$ and greedy is optimal. Sorting plus a single sweep (among intervals already started, track the maximum finish) gives $O(n \log n)$.

7.5 Fractional Knapsack

Input: n items, item i has weight w_i and value v_i; knapsack capacity W. Choose fractions $0 \le x_i \le 1$ maximizing $\sum_i x_i v_i$ subject to $\sum_i x_i w_i \le W$.

- **0/1 knapsack:** $x_i \in \{0, 1\}$ (take the whole item or nothing) — greedy fails; needs DP.
- **Fractional knapsack:** $x_i \in [0, 1]$ (items are divisible, e.g. liquids, grain) — greedy is optimal.

Greedy rule: sort items by **value per unit weight** v_i/w_i decreasing; take as much as possible of each item in that order. Running time $\Theta(n \log n)$.

Algorithm 29 Fractional Knapsack (Greedy)

Sort items so that $\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \cdots \geq \frac{v_n}{w_n}$
$w \leftarrow W$ ▷ remaining capacity
for $i = 1$ **to** n **do**
  if $w_i \leq w$ **then**
    $x_i \leftarrow 1$; $w \leftarrow w - w_i$
  else
    $x_i \leftarrow w/w_i$; $w \leftarrow 0$; **return**
  end if
end for

Worked example. $W = 50$; items $(w_i, v_i) = (10, 60), (20, 100), (30, 120)$, with ratios $6, 5, 4$.

1. Take all of item 1: used weight 10, value 60.
2. Take all of item 2: used weight 30, value 160.
3. Fill the remaining 20 units with a fraction $20/30 = 2/3$ of item 3: added value $\frac{2}{3} \cdot 120 = 80$.

Optimal total: $60 + 100 + 80 = \mathbf{240}$, weight exactly 50.

Why the Same Greedy Fails for 0/1 Knapsack

On the example above, "take by ratio while the whole item fits" grabs items 1 and 2 (weight 30, value 160) and then item 3 does not fit — leaving 20 units of capacity wasted. The 0/1 optimum takes items 2 and 3: weight 50, value $220 > 160$. Being forced to leave capacity unused is exactly what divisibility (the fractional version) avoids.

Correctness Proof (Exchange Argument)

Assume $\sum_i w_i \geq W$ (otherwise take everything; trivially optimal), so both greedy $G = (x_1, \ldots, x_k, 0, \ldots, 0)$ (where $x_i = 1$ for $i < k$ and $0 \leq x_k \leq 1$) and any optimum $O = (y_1, \ldots, y_n)$ fill the knapsack exactly.

1. Let i be the first index where $x_i \neq y_i$. By the greedy ordering and the prefix agreement, greedy takes at least as much of the best remaining item: $x_i \geq y_i$.
2. Increase y_i to x_i and compensate by removing total weight $(x_i - y_i)w_i$ from items $i+1, \ldots, n$ (enough such weight exists because both solutions pack exactly W).
3. Value cannot decrease: every item $j > i$ has ratio $v_j/w_j \leq v_i/w_i$, so swapping weight onto item i replaces "cheaper" weight with "dearer" weight.
4. Since O was optimal, its value also cannot increase, so it stays exactly the same: the modified O is still optimal and now agrees with G up to index $i+1$.

Repeating converts O into G with unchanged value $\Rightarrow$ greedy is optimal.

7.6 Interval Partitioning

Lecture j occupies $[s_j, f_j)$. Goal: schedule all lectures using the **minimum number of classrooms** so that no two lectures overlap in the same room.

Greedy algorithm: sort lectures by increasing **start** time; assign each lecture to any classroom whose last lecture has finished; if none exists, open a new classroom.

Algorithm 30 Interval Partitioning (Greedy by Start Time)

```
Sort lectures so that s_1 ≤ s_2 ≤ ··· ≤ s_n
H ← empty min-heap of classroom finish times
for j = 1 to n do
    if H ≠ ∅ and min(H) ≤ s_j then
        ExtractMin(H)                          ▷ reuse that classroom
    end if
    Insert(H, f_j)                             ▷ new room if none was reused
end for
return |H|                                     ▷ number of classrooms used
```

Worked trace. Lectures (sorted by start): $A = [1, 4)$, $B = [2, 6)$, $F = [3, 3.5)$, $C = [4, 5)$, $D = [5, 7)$, $E = [6, 8)$.

Lecture	Decision	Heap of finish times	Overlapping rooms
$A[1, 4)$	open room 1	$\{4\}$	1
$B[2, 6)$	$4 > 2$, open room 2	$\{4, 6\}$	2
$F[3, 3.5)$	$4 > 3$, open room 3	$\{3.5, 4, 6\}$	3
$C[4, 5)$	$3.5 \leq 4$, reuse	$\{4, 5, 6\}$	3
$D[5, 7)$	$4 \leq 5$, reuse	$\{5, 6, 7\}$	3
$E[6, 8)$	$5 \leq 6$, reuse	$\{6, 7, 8\}$	3

Three rooms are used; at time 3 lectures A, B, F overlap simultaneously, so 3 is unavoidable.

Depth Lower Bound and Correctness

The **depth** of a set of intervals is the maximum number of intervals covering any single point in time.

- **Lower bound:** every schedule needs at least depth classrooms — at that instant all those lectures are running and each needs its own room.
- **Greedy matches it:** suppose greedy opens its d-th classroom for lecture j. At that moment all $d - 1$ existing rooms have finish time $> s_j$; by the start-time ordering their lectures started at or before s_j, so together with j there are d lectures covering time s_j. Hence depth $\geq d$.

Greedy uses exactly (no more than) the depth $\Rightarrow$ optimal.

Running Time

- Naively scanning all rooms for compatibility: $O(n^2)$.
- Key observation: if j is incompatible with the room whose last lecture finishes **earliest**, it is incompatible with every room. So one min-heap keyed by room finish times suffices: each lecture does one `Min` check ($O(1)$) and at most one `ExtractMin`/`Insert` ($O(\log n)$ each).
- Total: $\Theta(n \log n)$ including the sort.

Sorting by Finish Time with Arbitrary Room Assignment Fails

Intervals $(1, 3), (2, 4), (4, 5), (3, 6)$, processed by increasing finish time:

1. $(1, 3) \rightarrow$ room 1 (finishes 3).
2. $(2, 4)$: no room free $\rightarrow$ room 2 (finishes 4).
3. $(4, 5)$: both rooms are free; **arbitrarily** choose room 1 (finish 3) $\rightarrow$ room 1 now finishes 5.
4. $(3, 6)$: neither room is free $(5, 4 > 3) \rightarrow$ room 3.

This uses 3 rooms although depth is 2 (optimal: room 1 gets $(1, 3), (3, 6)$; room 2 gets

$(2,4),(4,5)$). **Fix (tie-breaking):** if several rooms are available, reuse the one with the **latest** finish time (best fit). Implement with a BST of rooms ordered by finish time: take the predecessor with finish $\leq s_j$ and update it to f_j; total $O(n \log n)$.

8 Huffman Coding

Huffman coding is another greedy algorithm: it builds an optimal **prefix-free binary code** for a file whose character frequencies are known.

8.1 Motivation: Variable-Length Encoding

There are over 10^6 characters across all languages: a fixed-length code would need ≥ 21 bits per character, while English needs only 7 (ASCII). Internet traffic is predominantly English, so **variable-length** codes — short codewords for frequent characters, long codewords for rare ones — save many bits in expectation.

8.2 Prefix-Free Codes

A code is a **prefix-free code** ("prefix code") if no codeword is a prefix of any other codeword.

Prefix-Free ⇒ Unique Decodability

Theorem. Every message encoded by a prefix-free code is uniquely decodable: reading left to right, the bits can match only one codeword (it cannot be a prefix of a longer one), so peel it off and continue.

Examples.

- $C_1 = \{a = 00, b = 01, c = 10, d = 11\}$ (fixed-length): $010011 = 01\,|\,00\,|\,11 \rightarrow$ "bad".
- $C_2 = \{a = 0, b = 110, c = 10, d = 111\}$ (prefix-free): $1100111 = 110\,|\,0\,|\,111 \rightarrow$ "bad".
- $C_3 = \{a = 1, b = 110, c = 10, d = 111\}$ (not prefix-free, since 1 is a prefix of 110 and 111): 1101111 can be parsed as $110\,|\,1\,|\,111$ = "bad" or $1\,|\,10\,|\,1\,|\,111$ = "acad" — ambiguous!

Why Prefix Codes in Practice?

UTF-8 marks continuation bytes with the prefix `10`. If bytes are corrupted or dropped, the decoder self-synchronizes: skip ahead to the next byte not beginning with `10` rather than losing the rest of the file.

8.3 The Encoding Problem

Given characters with known frequencies, find a prefix code minimizing the total encoded length (compression).

Example file: 100,000 characters over $\{a, b, c, d, e, f\}$ with frequencies in thousands:

	a	b	c	d	e	f	Total (bits)
Frequency ($\times 1000$)	45	13	12	16	9	5	100,000 chars
Fixed-length code	000	001	010	011	100	101	$3 \cdot 100{,}000 = 300{,}000$
Variable-length code	0	101	100	111	1101	1100	224,000

The variable code costs

$(45{\cdot}1+13{\cdot}3+12{\cdot}3+16{\cdot}3+9{\cdot}4+5{\cdot}4){\cdot}1000 = (45+39+36+48+36+20){\cdot}1000 = 224{,}000$ bits,

a 25% reduction.

8.4 Tree Correspondence and Weighted External Path Length

Prefix Codes ↔ Binary Trees

Represent a prefix code by a binary tree: label the left edge 0 and the right edge 1; the codeword of a character is the bit string on the root-to-leaf path.

- Every **leaf** stores one character; an internal node has two children (a "full" tree — no leaf codeword can be a prefix of another).
- Leaf depth $d(a_i)$ equals codeword length.
- Set leaf weight to the character frequency $f(a_i)$. The **weighted external path length**

$$B(T) = \sum_{i=1}^{n} f(a_i)\, d(a_i)$$

is exactly the total encoded length. Finding the minimum-cost prefix code = finding a minimum-weighted-external-path-length binary tree!

In the example, the fixed-length tree (depth 3 for all six leaves) costs $(45{+}13{+}12{+}16{+}9{+}5){\cdot}3 = 300$ (thousand bits); the variable tree costs $45{\cdot}1{+}(12{+}13{+}16){\cdot}3{+}(9{+}5){\cdot}4 = 224$.

8.5 Huffman's Greedy Algorithm

Greedy idea (a "merge"):

1. Pick the two characters x, y with the **smallest** weights.
2. Join them as sibling leaves under a new internal node z with weight $f(z) = f(x)+f(y)$.
3. Remove x, y from the alphabet and insert z.
4. Repeat until one node (the root) remains.

Algorithm 31 Huffman

```
function HUFFMAN(A)
    build a min-priority queue Q on A, keyed by weight
    for i = 1 to n − 1 do
        allocate a new node z
        x ← EXTRACTMIN(Q)
        y ← EXTRACTMIN(Q)
        z.left ← x;    z.right ← y
        z.weight ← x.weight + y.weight
        INSERT(Q, z)
    end for
    return EXTRACTMIN(Q)                    ▷ root of the Huffman tree
end function
```

Running time: $O(n \log n)$ with a binary min-heap ($n-1$ merge rounds, each with two extract-mins and one insert, each $O(\log n)$).

8.6 Worked Example

Initial alphabet (weights): f:5, e:9, c:12, b:13, d:16, a:45.

Round	Merge	Priority queue afterwards
0	—	$\{5f,\ 9e,\ 12c,\ 13b,\ 16d,\ 45a\}$
1	$5f + 9e = 14$	$\{12c,\ 13b,\ 14,\ 16d,\ 45a\}$
2	$12c + 13b = 25$	$\{14,\ 16d,\ 25,\ 45a\}$
3	$14 + 16d = 30$	$\{25,\ 30,\ 45a\}$
4	$25 + 30 = 55$	$\{45a,\ 55\}$
5	$45a + 55 = 100$	$\{100\}$ (root)

Reading root-to-leaf paths (left = 0, right = 1) gives:

Character	a	c	b	f	e	d
Codeword	0	100	101	1100	1101	111
Length	1	3	3	4	4	3

Two Ways to Compute the Total Cost

- By leaf depths: $B(T) = \sum_i f_i d_i = 45{\cdot}1 + (12 + 13 + 16){\cdot}3 + (9 + 5){\cdot}4 = 224.$
- By merge weights: $B(T) = 14 + 25 + 30 + 55 + 100 = 224.$

The sums agree because a leaf of weight f contributes f to the merge weight of every internal node on its path to the root — exactly one contribution per level of depth. This identity is the key to both the proof and the optimal-merge application.

8.7 Correctness Proof

Lemma 1: An Optimal Tree Is Full

Every internal node in an optimal prefix-code tree has exactly two children. If an internal node had only one child, contract that node (replace it by its child): no leaf gets deeper, and some leaves get shallower, strictly decreasing $B(T)$ — contradiction.

Lemma 2: Swapping Leaves

Let T' be obtained from T by swapping two leaves x and b. If $f(x) \le f(b)$ and $d(x) \le d(b)$ (move the smaller-frequency leaf down and the larger one up), then

$$B(T') = B(T) + \big(f(x) - f(b)\big)\big(d(b) - d(x)\big) \le B(T),$$

since the first factor is ≤ 0 and the second is ≥ 0. So placing frequent characters nearer the root never increases the cost.

Lemma 3: The Two Smallest Weights Can Be Siblings at Maximum Depth

Let x, y be the two least frequent characters. Take any optimal tree T (full by Lemma 1); it contains two sibling leaves b, c at the deepest level. Assuming $f(x) \le f(b)$ and $f(y) \le f(c)$ (relabel if needed), swap $x \leftrightarrow b$ and $y \leftrightarrow c$. By Lemma 2 the cost does not increase; since T was optimal, the resulting tree is also optimal and now has x, y as sibling leaves at the deepest level.

Lemma 4: Contracting Two Sibling Leaves

If x, y are sibling leaves in T, contract them into one leaf z with $f(z) = f(x) + f(y)$ to obtain T', then

$$\begin{aligned} B(T) &= B(T') - f(z)d(z) + f(x)(d(z) + 1) + f(y)(d(z) + 1) \\ &= B(T') + f(x) + f(y). \end{aligned}$$

Theorem: Huffman Produces an Optimal Tree (Induction on n)

Base ($n = 2$): the only full tree has two leaves at depth 1; it is optimal.
Inductive step: assume Huffman is optimal for alphabets of size $n - 1$. For size n, let H be the Huffman tree and x, y its first merge (the two smallest weights, siblings in H). Contract them to z with $f(z) = f(x) + f(y)$: the contracted tree H' is exactly the tree Huffman builds on the $n - 1$-character alphabet $A' = A - \{x, y\} \cup \{z\}$, so by the induction hypothesis H' is optimal for A', and by Lemma 4

$$B(H) = B(H') + f(x) + f(y).$$

By Lemma 3 there is an optimal tree T for A with x, y as siblings; contract them to get T' for A'. Then

$$B(H) = B(H') + f(x) + f(y) \leq B(T') + f(x) + f(y) = B(T).$$

Since T is optimal, H is optimal too. □

Huffman Caveats

- Huffman is optimal **among prefix codes**, not among all possible codes (arithmetic coding can exploit fractional code lengths).
- Each merge must join the two current smallest weights — joining the smallest with an arbitrary node is not optimal.
- Ties in weight may be broken arbitrarily: different trees can result, all with the same minimum cost.
- Edge case $n = 1$: the single character gets the empty codeword (conventionally one bit may be used).

8.8 Application: Optimal Two-Way Merge

Given n sorted lists $L_1, \ldots, L_n$, merge them into one sorted list, merging only two lists at a time; merging lists of sizes p and q costs $p + q$ comparisons and produces a list of size $p + q$. Find the merge pattern minimizing total comparisons.

A merge pattern is a binary tree: leaves are the initial lists (weight = list size), each internal node is a merge, and the total cost equals the **weighted external path length** — so Huffman (always merge the two smallest lists) is optimal.

Example: three lists of sizes $30, 20, 10$:

Pattern	Computation	Total comparisons
L_1, L_2 first, then L_3	$30 + 20 = 50$, then $50 + 10 = 60$	$50 + 60 = 110$
L_1, L_3 first, then L_2	$30 + 10 = 40$, then $40 + 20 = 60$	$40 + 60 = 100$
L_2, L_3 **first, then** L_1 **(Huffman)**	$20 + 10 = 30$, then $30 + 30 = 60$	$\mathbf{30 + 60 = 90}$

Algorithm 32 Optimal Binary Merge Tree

```
build a min-heap from the n initial list sizes
while heap size ≥ 2 do
    a ← ExtractMin;   b ← ExtractMin
    create an internal node of size a + b with children a, b
    Insert(a + b)
end while
```

Time $O(n \log n)$.

Worked example: initial sizes $2, 3, 5, 7, 9$:

Round	Merge	Heap afterwards
1	$2+3=5$	$\{5,5,7,9\}$
2	$5+5=10$	$\{7,9,10\}$
3	$7+9=16$	$\{10,16\}$
4	$10+16=26$	$\{26\}$

Total cost (sum of merge sizes): $5+10+16+26 = \mathbf{57}$, which equals the weighted external path length of the merge tree:

$$2\cdot 3+3\cdot 3+5\cdot 2+7\cdot 2+9\cdot 2 = 6+9+10+14+18 = 57.$$

Greedy Algorithms at a Glance

Problem	Greedy choice	Complexity	Correctness idea
Interval scheduling	earliest finish time	$\Theta(n \log n)$	exchange argument
Interval covering	furthest reach covering frontier	$O(n \log n)$	exchange argument
Fractional knapsack	largest value/weight	$\Theta(n \log n)$	exchange argument
Interval partitioning	earliest start, reuse a room	$\Theta(n \log n)$	match depth lower bound
Huffman coding	merge two smallest weights	$O(n \log n)$	exchange + induction
Optimal merge	merge two smallest lists	$O(n \log n)$	same as Huffman

9 Dynamic Programming

Dynamic programming (DP) designs an algorithm for an optimization problem by combining the answers of smaller subproblems. It applies when the problem has

- **optimal substructure**: an optimal solution is built from optimal solutions of subproblems;
- **overlapping subproblems**: a plain recursion solves the same subproblem exponentially many times, so the answer must be stored and reused.

Every DP solution in this chapter is obtained by the same five steps:

1. Define the **subproblem** (what the table entry means, and which indices/parameters identify it).
2. Write the **recurrence**: the value of an entry as a function of strictly smaller entries.
3. Fix the **base cases** and the **order** of evaluation (so that all entries used are already computed).
4. Prove the recurrence by a **cut-and-paste** argument (exchange/optimality).
5. Store the **decision** that achieved the optimum in a second table, to reconstruct an optimal solution.

Top-Down (Memoization) versus Bottom-Up

Memoization keeps the recursion but caches each answer, so each of the (few) distinct subproblems is solved once. Bottom-up DP evaluates the subproblems in increasing size order with nested loops, reusing a table. Bottom-up is usually preferred: it has no recursion overhead, allows space optimization (e.g. $\Theta(1)$ instead of $\Theta(n)$ for Fibonacci), and is safer. Top-down wins when the number of reachable subproblems is much smaller than the table size (a sparse problem space), e.g. coin change with huge denominations.

9.1 Warm-up: Stair Climbing and Fibonacci

Stair Climbing

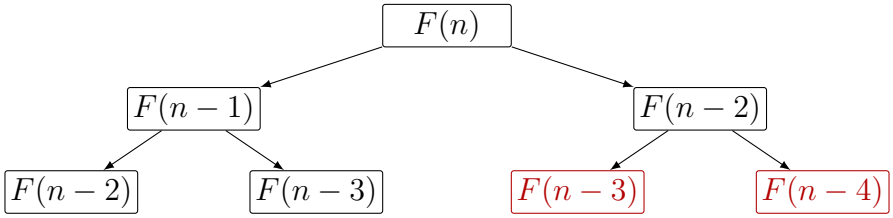
Problem. One step climbs either 1 or 2 stairs. How many different ways are there to climb n stairs?
Solution. Let $F(n)$ be the number of ways to reach stair n. The last step starts at stair $n-1$ (a 1-step) or at stair $n-2$ (a 2-step), and these two cases are disjoint and exhaustive:

$$F(1) = 1, \qquad F(2) = 2, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 3).$$

Hence $F(1), F(2), \ldots$ are the Fibonacci numbers. □

Why the Naive Recursion Is Exponential

A direct recursive implementation solves the same subproblem many times:



duplicated subproblems (red) are solved again and again ···

The call count satisfies $T(n) = T(n-1)+T(n-2)+1$, hence $T(n) = \Theta\big(F(n)\big) = \Theta(\varphi^n)$ with $\varphi = (1+\sqrt{5})/2 \approx 1.618$. There are only n distinct subproblems; storing their answers reduces the time to $\Theta(n)$.

Algorithm 33 Memoized (top-down) and bottom-up Fibonacci

```
allocate F[1..n] = [1, 2, 0, 0, ...]
function TD-F(n)
    if F[n] ≠ 0 then
        return F[n]
    end if
    F[n] ← TD-F(n − 1) + TD-F(n − 2)
    return F[n]
end function

function BU-F(n)
    Fp ← 1;   F ← 2
    for i = 3 to n do
        temp ← Fp;   Fp ← F;   F ← Fp + temp
    end for
    return F
end function
```

The bottom-up version needs only two variables, because $F(n)$ depends on the last two values: time $\Theta(n)$, space $\Theta(1)$.

9.2 The Rod Cutting Problem

Problem. Given a rod of length n and prices p_i for $i = 1, \ldots, n$, where p_i is the price of a rod of length i, cut the rod into pieces (integer lengths) to maximize total revenue.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

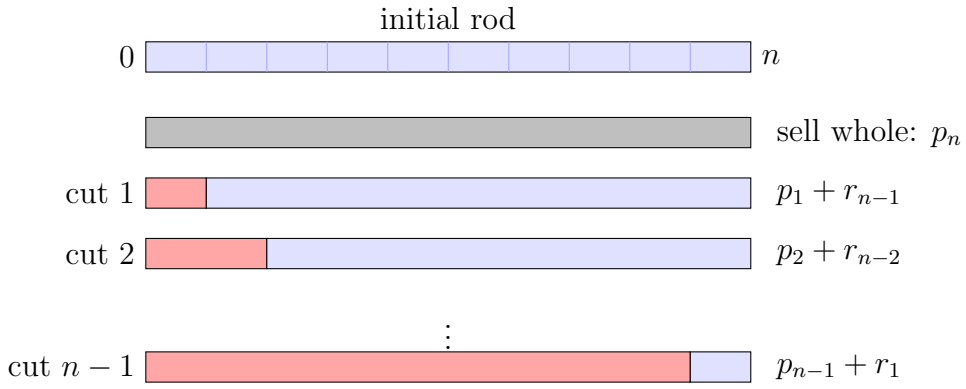
There are 2^{n-1} ways of cutting a rod of size n (a cut or no cut between each pair of consecutive unit lengths), so exhaustive search is hopeless.

Optimal Substructure

Let r_n be the maximum revenue for a rod of length n. Look at the first piece: it has some length $i \in \{1, \ldots, n\}$, earns p_i, and the remainder of length $n-i$ must be cut optimally, i.e. it earns r_{n-i}. Therefore

$$r_n = \max_{1 \le i \le n} \big(p_i + r_{n-i}\big), \qquad r_0 = 0,$$

where $i = n$ is the option "sell the rod in one piece" $(p_n + r_0)$.



Algorithm 34 ROD-CUTTING(p, n)

```
allocate r[0..n], s[1..n]
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        if q < p[i] + r[j − i] then
            q ← p[i] + r[j − i];   s[j] ← i
        end if
    end for
    r[j] ← q
end for
return r[n]
```

Running time $\Theta(n^2)$ (there are $\binom{n}{2}$ pairs (i, j)), space $\Theta(n)$.

Worked Example: the Full r and s Tables

With the prices above, the optimal revenues and the optimal first-piece lengths are

i	0	1	2	3	4	5	6	7	8	9	10
$p[i]$	0	1	5	8	9	10	17	17	20	24	30
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	–	1	2	3	2	2	6	1	2	3	10

Each entry is the maximum of $p_i + r_{j-i}$ over $i = 1, \ldots, j$:

j	**candidates** $p_i + r_{j-i}$	$\boldsymbol{r_j}$ **= max (argmax)**
1	$p_1 + r_0 = 1$	1 (i=1)
2	$1 + r_1 = 2$, $5 + r_0 = 5$	5 (i=2)
3	$1 + r_2 = 6$, $5 + r_1 = 6$, $8 + r_0 = 8$	8 (i=3)
4	$1 + 8 = 9$, $5 + 5 = 10$, $8 + 1 = 9$, 9	10 (i=2)
5	$1 + 10 = 11$, $5 + 8 = 13$, $8 + 5 = 13$, $9 + 1 = 10$, 10	13 (i=2)
6	$1 + 13 = 14$, $5 + 10 = 15$, $8 + 8 = 16$, $9 + 5 = 14$, $10 + 1 = 11$, 17	17 (i=6)
7	$1 + 17 = 18$, $5 + 13 = 18$, $8 + 10 = 18$, $9 + 8 = 17$, $10 + 5 = 15$, $17 + 1 = 18$, 17	18 (i=1)
8	$1 + 18 = 19$, $5 + 17 = 22$, $8 + 13 = 21$, $9 + 10 = 19$, $10 + 8 = 18$, $17 + 5 = 22$, $17 + 1 = 18$, 20	22 (i=2)
9	$1 + 22 = 23$, $5 + 18 = 23$, $8 + 17 = 25$, $9 + 13 = 22$, $10 + 10 = 20$, $17 + 8 = 25$, $17 + 5 = 22$, $20 + 1 = 21$, 24	25 (i=3)
10	$1 + 25 = 26$, $5 + 22 = 27$, $8 + 18 = 26$, $9 + 17 = 26$, $10 + 13 = 23$, $17 + 10 = 27$, $17 + 8 = 25$, $20 + 5 = 25$, $24 + 1 = 25$, 30	30 (i=10)

Reconstructing a Solution

Table r gives only the optimal value. Storing the index $s[j]$ where the maximum occurred lets us recover the pieces:

$j \leftarrow n$
while $j > 0$ **do**
  print $s[j]$; $j \leftarrow j - s[j]$
end while

For $n = 9$: $s[9] = 3$ gives a piece of length 3 and leaves $j = 9 - 3 = 6$; then $s[6] = 6$ gives a piece of length 6 and leaves $j = 0$. The optimal cut is $\{3, 6\}$ with revenue $25 = 8 + 17$. For $n = 7$: $s[7] = 1$, then $s[6] = 6$, giving $\{1, 6\}$ with revenue $18 = 1 + 17$.

9.3 Exercise: Maximum Sum of Non-Adjacent Elements

Maximum Sum of Non-Adjacent Elements

Problem. Given positive numbers $a_1, \ldots, a_n$, choose a subset S maximizing $\sum_{a_i \in S} a_i$ under the constraint that a_i and $a_{i\pm1}$ cannot both be chosen.

Greedy fails. For $A = (7, 8, 6, 3)$ greedy takes the largest, 8, deletes its neighbours 7, 6, then takes 3: value 11. The optimum is $\{7, 6\}$ with value 13.

Solution. Let $A_i = (a_1, \ldots, a_i)$, let S_i be an optimal subset of A_i, let W_i be its sum, and let $b_i =$ true iff $a_i \in S_i$. For the last element there are two cases:

- $a_i \in S_i$: then $a_{i-1} \notin S_i$, and the rest must be optimal for A_{i-2}, so $W_i = W_{i-2} + a_i$;
- $a_i \notin S_i$: then S_i is optimal for A_{i-1}, so $W_i = W_{i-1}$.

Hence

$$W_i = \max\{W_{i-2} + a_i,\ W_{i-1}\}, \qquad W_1 = a_1, \quad W_2 = \max\{a_1, a_2\}.$$

Algorithm 35 Maximum sum of non-adjacent elements

```
W[1] ← a_1;   b[1] ← true
if a_2 > a_1 then
    W[2] ← a_2;   b[2] ← true
else
    W[2] ← a_1;   b[2] ← false
end if
for i = 3 to n do
    if W[i − 2] + a_i > W[i − 1] then
        W[i] ← W[i − 2] + a_i;   b[i] ← true
    else
        W[i] ← W[i − 1];   b[i] ← false
    end if
end for
```

Time $\Theta(n)$, space $\Theta(n)$ (only $W[i-1], W[i-2]$ are needed for the value).

Worked Example

For $A = (1, 8, 6, 3, 7)$:

i	1	2	3	4	5
a_i	1	8	6	3	7
$W[i]$	1	8	8	11	15
$b[i]$	true	true	false	true	true

Check: $W_3 = \max(W_1 + 6, W_2) = \max(7, 8) = 8$; $W_4 = \max(W_2 + 3, W_3) = \max(11, 8) = 11$; $W_5 = \max(W_3 + 7, W_4) = \max(15, 11) = 15$.
Reconstruction: start at $i = n$; if b_i is true, output a_i and set $i \leftarrow i - 2$; otherwise set $i \leftarrow i - 1$.

$b_5 = \mathsf{true} \Rightarrow$ print 7, $i = 3$; $\quad b_3 = \mathsf{false} \Rightarrow i = 2$; $\quad b_2 = \mathsf{true} \Rightarrow$ print 8, $i = 0$.

Optimal subset $\{7, 8\}$, sum 15. □

9.4 Weighted Interval Scheduling

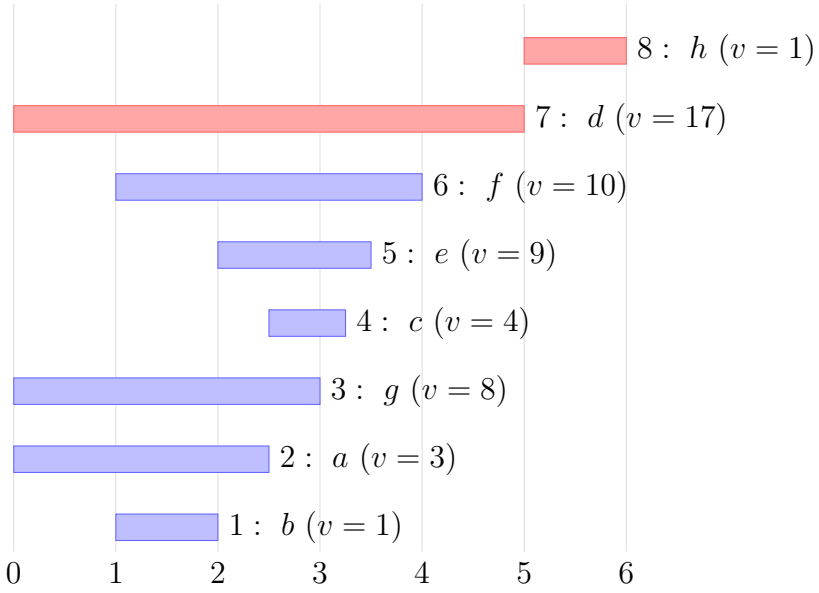
Problem. Job j starts at s_j, finishes at f_j and has weight $v_j > 0$. Two jobs are compatible if they do not overlap (touching is allowed: $f_i \le s_j$). Goal: a maximum-weight subset of mutually compatible jobs.

- If all weights are 1, the greedy algorithm (take jobs in increasing finish time, keep those compatible with the current set) is optimal.
- With arbitrary weights greedy fails badly: a single long job b of weight 999 and a short job a of weight 1 that overlaps b — greedy by finish time may take both, or may take the wrong one, but it cannot know which combination is best.

Notation. Sort the jobs so that $f_1 \le f_2 \le \cdots \le f_n$. Define

$$p(j) = \max\{i < j : \ f_i \le s_j\}$$

(the largest index of a job compatible with j; all jobs i' with $p(j) < i' < j$ are incompatible with j). Since the jobs are sorted by finish time, $p(j)$ is found by a binary search for s_j in $\Theta(\log n)$ time.



jobs sorted by finish time (optimum $\{d, h\}$ in red)

The Recurrence

Let $V(j)$ be the value of an optimal solution using only jobs $1, \dots, j$. Either

- job j is not selected: then $V(j) = V(j-1)$; or
- job j is selected: then all jobs $p(j)+1, \dots, j-1$ are excluded and the rest must be optimal for jobs $1, \dots, p(j)$, so $V(j) = v_j + V(p(j))$.

$$V(j) = \max\big\{v_j + V(p(j)),\ V(j-1)\big\}, \qquad V(0) = 0.$$

The subproblems are again prefixes of the sorted list, so the table is 1-dimensional and filled left to right.

Algorithm 36 WEIGHTED-INTERVAL-SCHEDULING$(\{s_j, f_j, v_j\})$

sort all jobs by finish time; compute $p(j)$ for all j by binary search
$V[0] \leftarrow 0$
for $j = 1$ **to** n **do**
  if $v_j + V[p(j)] > V[j-1]$ **then**
    $V[j] \leftarrow v_j + V[p(j)]$; $keep[j] \leftarrow 1$
  else
    $V[j] \leftarrow V[j-1]$; $keep[j] \leftarrow 0$
  end if
end for
$j \leftarrow n$; **while** $j > 0$: **if** $keep[j] = 1$ **then** print j; $j \leftarrow p(j)$ **else** $j \leftarrow j-1$

Running time $\Theta(n \log n)$ (sorting plus one binary search per job, or $\Theta(n)$ after sorting with a two-pointer scan), space $\Theta(n)$.

Worked Example

The eight jobs of the picture, sorted by finish time, give

j	1	2	3	4	5	6	7	8
job	b	a	g	c	e	f	d	h
$[s_j, f_j]$	$[1, 2]$	$[0, 2.5]$	$[0, 3]$	$[2.5, 3.25]$	$[2, 3.5]$	$[1, 4]$	$[0, 5]$	$[5, 6]$
v_j	1	3	8	4	9	10	17	1
$p(j)$	0	0	0	2	1	0	0	7
$V(j)$	1	3	8	8	10	10	17	18
$keep[j]$	1	1	1	0	1	0	1	1

Sample computations: $j = 5$: $\max\{9 + V(1), V(4)\} = \max\{10, 8\} = 10$; $j = 7$: $\max\{17 + V(0), V(6)\} = \max\{17, 10\} = 17$; $j = 8$: $\max\{1 + V(7), V(7)\} = \max\{18, 17\} = 18$.
Reconstruction: $j = 8$, $keep[8] = 1$: print h, $j \leftarrow p(8) = 7$; $keep[7] = 1$: print d, $j \leftarrow p(7) = 0$. Optimal set $\{h, d\}$ with value 18. □

9.5 Exercise: Highway Billboards

Highway Billboards

Problem. Sites $x_1 \le \cdots \le x_n$ on a highway, revenue $r_i > 0$ if a billboard is placed at x_i, and any two billboards must be at least 5 miles apart. Maximize the revenue.
Question 1: a greedy algorithm. Place a billboard at the site with maximum revenue, delete that site and all sites within 5 miles of it, and repeat.
Question 2: is it optimal? No, for the same reason as in the maximum-sum exercise: with sites spaced 4 miles apart the problem is the non-adjacent maximum-sum problem, for which greedy is suboptimal ($7, 8, 6, \overline{3}$: greedy $8 + 3 = 11$, optimum $7 + 6 = 13$).
DP. Sort the sites; let $p(i) = \max\{j < i : \ x_i - x_j > 5\}$ be the last site compatible with x_i, and let $R(i)$ be the optimal revenue using the first i sites. Then

$$R(i) = \max\big\{R(i-1),\ r_i + R(p(i))\big\}, \qquad R(0) = 0,$$

computed for $i = 1, \ldots, n$; the answer is $R(n)$ and the reconstruction is the same as for weighted interval scheduling. Time $\Theta(n \log n)$. □

9.6 Kadane's Algorithm Derived from DP

Maximum Subarray Revisited

For an array $A[1..n]$, let $V[i]$ be the maximum sum of a subarray that ends at $A[i]$ (a "local" optimum). Then either the subarray is just $A[i]$, or it extends an optimal subarray ending at $A[i-1]$:

$$V[1] = A[1], \qquad V[i] = \max\big(A[i],\ A[i] + V[i-1]\big),$$

and the global answer is $\max_i V[i]$ (the optimum subarray ends somewhere).

Full Trace

For $A = (-2,\ 1,\ -3,\ 4,\ -1,\ 2,\ 1,\ -5,\ 4)$:

i	1	2	3	4	5	6	7	8	9
$A[i]$	-2	1	-3	4	-1	2	1	-5	4
$V[i]$	-2	1	-2	4	3	5	6	1	5
$V_{\max}$	-2	1	1	4	4	5	6	6	6

For instance $V[5] = \max(-1, -1+4) = 3$ and $V[6] = \max(2, 2+3) = 5$; at $i = 8$ the running value drops to 1 but $V_{\max} = 6$ is kept. The optimum subarray is $A[4..7] = (4, -1, 2, 1)$ with sum 6.

Space Optimization and Recovering the Indices

$V[i]$ depends only on $V[i-1]$, so the array is unnecessary: keep a scalar V together with $V_{\max}$. To also return the location, remember i whenever $V_{\max}$ is updated (this is the end), and set $temp \leftarrow i$ whenever V is restarted at $A[i]$ (because $V[i-1] < 0$); the start is the last $temp$ that produced a new maximum. Time $\Theta(n)$, space $\Theta(1)$.

Algorithm 37 KADANE(A, n) with start/end positions

```
V ← 0;  Vmax ← −∞;  start ← 1;  end ← 1;  temp ← 1
for i = 1 to n do
    V ← V + A[i]
    if V < A[i] then
        V ← A[i];  temp ← i
    end if
    if V > Vmax then
        Vmax ← V;  start ← temp;  end ← i
    end if
end for
return (Vmax, start, end)
```

9.7 Minimum Number of Coins

Coin Change

Input: an amount n and denominations $d_1, \ldots, d_k$ (unlimited supply of each). **Output:** the minimum number of coins summing to n.
Greedy is not optimal for arbitrary denominations (e.g. $\{1, 3, 4\}$ and $n = 6$: greedy $4 + 1 + 1$, optimum $3 + 3$).
DP. Let $C[p]$ be the minimum number of coins for amount p. If the last coin used has denomination d_i, the rest must be optimal for $p - d_i$:

$$C[0] = 0, \qquad C[p] = 1 + \min_{1 \le i \le k,\ d_i \le p} \{C[p - d_i]\}.$$

Store in $S[p]$ the denomination of that last coin, then print the solution by repeatedly printing $S[p]$ and setting $p \leftarrow p - S[p]$.

Algorithm 38 DP-COIN(n)

```
C[0] ← 0
for p = 1 to n do
    min ← ∞
    for i = 1 to k do
        if p ≥ d_i and 1 + C[p − d_i] < min then
            min ← C[p − d_i] + 1;  coin ← d_i
        end if
    end for
    C[p] ← min;  S[p] ← coin
end for
while n > 0: print S[n];  n ← n − S[n]
```

Time $\Theta(nk)$, space $\Theta(n)$.

When the Denominations Are Huge

If $n = 19999$ and the denominations are $1000, 5000, 9999, 10000, 50000$, the bottom-up table has 20000 entries although only a few amounts are reachable. Then top-down DP with memoization is better: keep hash tables C, S and call MakeChange(p), which returns the cached $C[p]$ if present, and otherwise tries all $d_i \le p$ recursively. The cost becomes $\Theta\big(k \cdot \#\{\text{reachable amounts}\}\big)$. First scale all values down by their GCD to shrink n.

9.8 The 0/1 Knapsack Problem

Input: items $1, \dots, n$ with weights w_i and values $v_i > 0$, and a knapsack of capacity W. **Goal:** choose $x_i \in \{0, 1\}$ with $\sum x_i w_i \le W$ maximizing $\sum x_i v_i$. Greedy (by value, by weight, or by value/weight) is not optimal.

A Wrong One-Dimensional Definition

Letting $V[w]$ be the best value for capacity w and writing $V[w] = \max_i (v_i + V[w - w_i])$ (as for rod cutting) is **wrong**: nothing prevents the same item from being chosen twice, so the "solution" may be illegal. The state must also say which items are still available.

Correct Two-Dimensional Recurrence

Let $V[i, w]$ be the maximum value obtainable with capacity w using only items $1, \dots, i$. For item i there are two cases:

$$V[i, w] = \max\big\{V[i-1, w],\quad v_i + V[i-1,\ w - w_i]\big\} \quad (w_i \le w),$$

and $V[i, w] = V[i-1, w]$ if $w_i > w$; base cases $V[0, w] = V[i, 0] = 0$. The answer is $V[n, W]$. The table is filled row by row (i increasing) and, inside a row, w increasing, because entry (i, w) needs row $i - 1$ only.

Algorithm 39 0/1-Knapsack(n, W, v, w)

```
allocate V[0..n, 0..W] and keep[0..n, 0..W], all 0
for i = 1 to n do
    for w = 1 to W do
        if w_i ≤ w and v_i + V[i−1, w−w_i] > V[i−1, w] then
            V[i, w] ← v_i + V[i−1, w−w_i];   keep[i, w] ← 1
        else
            V[i, w] ← V[i−1, w];   keep[i, w] ← 0
        end if
    end for
end for
K ← W;   for i = n downto 1: if keep[i, K] = 1 then print i; K ← K − w_i
```

Time and space $\Theta(nW)$; the space can be reduced to $\Theta(n + W)$ (keep one row plus the *keep* bits, or two rows and recompute).

Worked Example (the Full Table)

Items: $v = (10, 40, 30, 50)$, $w = (5, 4, 6, 3)$, $W = 10$.

$V[i,w]$	w=0	1	2	3	4	5	6	7	8	9	10
$i=0$	0	0	0	0	0	0	0	0	0	0	0
$i=1$	0	0	0	0	0	10	10	10	10	10	10
$i=2$	0	0	0	0	40	40	40	40	40	50	50
$i=3$	0	0	0	0	40	40	40	40	40	50	70
$i=4$	0	0	0	50	50	50	50	90	90	90	90

Sample entries: $V[2,4] = \max(0,\ 40+V[1,0]) = 40$; $V[2,9] = \max(10,\ 40+V[1,5]) = \max(10, 50) = 50$; $V[4,3] = \max(0,\ 50+V[3,0]) = 50$; $V[4,7] = \max(50,\ 50+V[3,4]) = \max(50, 90) = 90$.
Reconstruction: $(4,10)$: $keep = 1$ (since $90 > 70$), take item 4, $K = 7$; $(3,7)$: $keep = 0$ (since $70 < 90$... compare $V[2,7] = 40$ with $30+V[2,1] = 30$, so $keep = 0$), skip item 3; $(2,7)$: $keep = 1$ ($40 + V[1,3] = 40 > 0$), take item 2, $K = 3$; $(1,3)$: $keep = 0$. Items $\{2,4\}$, value 90, weight $7 \le 10$. □

Exercise: Two Knapsacks

Problem. Two knapsacks, both of capacity W. Give an $O(nW^2)$ DP for the maximum total value carried.
Solution. Let $V[i, w_1, w_2]$ be the best value using items $1, \dots, i$ with capacities w_1, w_2 left. For item i there are three cases (skip it, put it in knapsack 1, put it in knapsack 2):

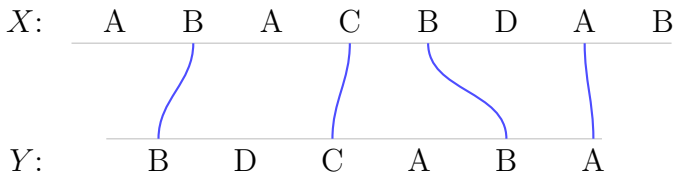
$$V[i,w_1,w_2] = \max\big\{V[i-1,w_1,w_2],\ v_i+V[i-1,w_1-w_i,w_2],\ v_i+V[i-1,w_1,w_2-w_i]\big\},$$

with the terms omitted when the corresponding capacity is insufficient, and base cases $V[0,w_1,w_2] = 0$. Fill with i increasing, then w_1 increasing, then w_2 increasing; the answer is $V[n,W,W]$. Time and space $\Theta(nW^2)$. □

9.9 Longest Common Subsequence

Problem. Given $X = (x_1, \dots, x_m)$ and $Y = (y_1, \dots, y_n)$, a common subsequence is $Z = (z_1, \dots, z_k)$ with $z_p = x_{i_p} = y_{j_p}$ for $i_1 < \cdots < i_k$ and $j_1 < \cdots < j_k$. Find a longest one (LCS).

Equivalently, match characters of X with equal characters of Y so that **no two matched pairs cross**:



$Z = (B, C, B, A)$ is an LCS of length 4

Recurrence

Let $c[i,j]$ be the length of an LCS of $X[1..i]$ and $Y[1..j]$.
- If $x_i = y_j$, then some optimal solution matches x_i with y_j (exchange argument: replace whatever x_i/y_j were matched to), so $c[i,j] = c[i-1, j-1] + 1$.
- If $x_i \neq y_j$, then at least one of them is unmatched, so $c[i,j] = \max\{c[i-1,j],\ c[i,j-1]\}$.

$$c[i,j] = \begin{cases} 0, & i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1, & i, j > 0,\ x_i = y_j, \\ \max\{c[i-1, j],\ c[i, j-1]\}, & i, j > 0,\ x_i \neq y_j. \end{cases}$$

Worked Example: the c Table

$X = A\,B\,A\,C\,B\,D\,A\,B$, $Y = B\,D\,C\,A\,B\,A$:

$c[i,j]$	j=0	1	2	3	4	5	6
$i = 0$	0	0	0	0	0	0	0
$i = 1$ (A)	0	0	0	0	1	1	1
$i = 2$ (B)	0	1	1	1	1	2	2
$i = 3$ (A)	0	1	1	1	2	2	3
$i = 4$ (C)	0	1	1	2	2	2	3
$i = 5$ (B)	0	1	1	2	2	3	3
$i = 6$ (D)	0	1	2	2	2	3	3
$i = 7$ (A)	0	1	2	2	3	3	4
$i = 8$ (B)	0	1	2	2	3	4	4

For example $c[2,1]$: $x_2 = y_1 = B$, so $c[2,1] = c[1,0]{+}1 = 1$; $c[2,6]$: $B \neq A$, so $c[2,6] = \max(c[1,6], c[2,5]) = \max(1,2) = 2$; $c[8,6]$: $B \neq A$, $c[8,6] = \max(c[7,6], c[8,5]) = \max(4,4) = 4$. The LCS length is $c[m,n] = 4$, and the entry $c[7,6] = c[6,5]{+}1 = 3{+}1$ reveals the match $x_7 = y_6 = A$.

Algorithm 40 LCS-Length and Print-LCS

```
allocate c[0..m, 0..n], b[0..m, 0..n], all 0
for i = 1 to m do
    for j = 1 to n do
        if x_i = y_j then
            c[i,j] ← c[i−1, j−1] + 1;   b[i,j] ←diag
        else if c[i−1, j] ≥ c[i, j−1] then
            c[i,j] ← c[i−1, j];   b[i,j] ←up
        else
            c[i,j] ← c[i, j−1];   b[i,j] ←left
        end if
    end for
end for

procedure Print-LCS(b, i, j)
    if i = 0 or j = 0 then
        return
    end if
    if b[i,j] =diag then
        Print-LCS(b, i−1, j−1); print x_i
    else if b[i,j] =up then
        Print-LCS(b, i−1, j)
    else
        Print-LCS(b, i, j−1)
    end if
end procedure
```

Time $\Theta(mn)$, space $\Theta(mn)$; only $\Theta(\min(m,n))$ space is needed if the optimal length alone is required (keep two rows, or one row plus one diagonal variable).

Longest Common Substring

A substring must be contiguous. Define $d[i,j]$ as the length of the longest common

substring of $X[1..i]$, $Y[1..j]$ that ends at x_i and y_j. Then

$$d[i,j] = d[i-1,j-1]+1 \text{ if } x_i = y_j, \qquad d[i,j] = 0 \text{ otherwise},$$

and the answer is $\max_{i,j} d[i,j]$ — the problem is changed (it is no longer a prefix problem), which is allowed because every common substring ends at some pair (i,j). Time $\Theta(mn)$. Example: X = DEADBEEF, Y = EATBEEF $\Rightarrow$ answer BEEF. The brute-force algorithm takes $O(n^4)$.

Exercise: Edit Distance

Definition. $edit(s,t)$ is the minimum number of insertions, deletions and substitutions that turn s into t. For s = abode and t = blog:

$$\text{abode} \xrightarrow{\text{delete } a} \text{bode} \xrightarrow{\text{insert } l} \text{blode} \xrightarrow{\text{delete } d} \text{bloe} \xrightarrow{e\to g} \text{blog},$$

so $edit(s,t) \le 4$; three operations cannot suffice because the lengths differ by one and the strings share only the letters b, o.
DP. Let $edit[i,j]$ be the distance between $s[1..i]$ and $t[1..j]$. Then

$$edit[0,j] = j, \qquad edit[i,0] = i,$$

$$edit[i,j] = \min \begin{cases} 1 + edit[i,j-1] & \text{(insert)}, \\ 1 + edit[i-1,j] & \text{(delete)}, \\ \mathbf{I}[s_i \neq t_j] + edit[i-1,j-1] & \text{(substitute/keep)}, \end{cases}$$

where $\mathbf{I}[\phi] = 1$ if ϕ is true and 0 otherwise. Time and space $\Theta(mn)$; $\Theta(\min(m,n))$ space if only the distance is wanted. □

9.10 Exercise: Choosing m Numbers with a Sliding-Window Constraint

At Least One in Every k Consecutive Positions

Problem. Given $A[1..n]$, choose exactly m numbers maximizing their sum, subject to: every k consecutive positions contain at least one chosen position ($k, m \le n$).
DP. Let $dp[i,j]$ be the maximum sum of i chosen numbers from $A[1..j]$ with $A[j]$ chosen. The previous chosen position is some $t \in [\max(0, j-k),\ j-1]$, hence

$$dp[i,j] = A[j] + \max_{\max(0,j-k) \le t \le j-1} dp[i-1,t],$$

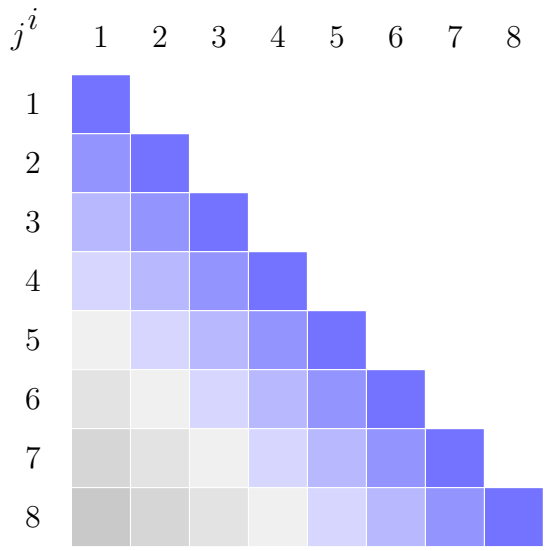
with $dp[0,0] = 0$ and all other entries $-\infty$. The answer is $\max_{n-k+1 \le t \le n} dp[m,t]$ (the last chosen position must still "cover" the final window).
Complexity. Directly the maximum is scanned for each (i,j), giving $O(nmk)$. But when going from layer $i-1$ to layer i the maximum is over a sliding window of size k, which a monotonic deque maintains in $O(1)$ amortized time per entry: $O(nm)$. □

9.11 DP over Intervals: Longest Palindromic Substring

Interval DP

The subproblems are now contiguous intervals $[i,j]$, not prefixes. An entry (i,j) depends on intervals strictly shorter than $j-i+1$ (typically $[i+1,j-1]$, $[i+1,j]$, $[i,j-1]$). Therefore the table is filled **by increasing length**: first the main diagonal (length 1, base cases), then the diagonal of length 2, and so on; the final answer sits

at the top-right corner $(1, n)$.



diagonals filled in order $\ell = 1, 2, \ldots, n$

Longest Palindromic Substring

Problem. A palindrome reads the same forward and backward (radar, level, racecar, madam). Given $X = x_1 \ldots x_n$, find the longest palindromic substring (contiguous). Example: $X =$ ACCABA has palindromic substrings CC, ACCA, ABA, and the longest is ACCA. Brute force takes $O(n^3)$.

DP. Let $p[i, j] =$ true iff $X[i..j]$ is a palindrome.

- base cases: $p[i, i] =$ true for all i; $p[i, i+1] =$ true iff $x_i = x_{i+1}$;
- recurrence: $p[i, j] =$ true iff $x_i = x_j$ and $p[i+1, j-1] =$ true.

The answer is the largest length $\ell = j - i + 1$ with $p[i, j] =$ true.

Worked Example: the Completed Table for $X =$ BABBCCCB

Rows are i, columns are j; only $j \geq i$ entries exist.

$p[i,j]$	j=1	2	3	4	5	6	7	8
$i = 1$	T	F	T	F	F	F	F	F
$i = 2$		T	F	F	F	F	F	F
$i = 3$			T	T	F	F	F	F
$i = 4$				T	F	F	F	T
$i = 5$					T	T	T	F
$i = 6$						T	T	F
$i = 7$							T	F
$i = 8$								T

Reading off the table: $p[1, 3] =$ T because $x_1 = x_3 = B$ and $p[2, 2] =$ T (substring BAB); $p[4, 8] =$ T because $x_4 = x_8 = B$ and $p[5, 7] =$ T (substring BCCCB, length 5, the optimum). Note $p[1, 8] =$ F even though $x_1 = x_8$, because $p[2, 7] =$ F.

Algorithm 41 LONGEST-PALINDROMIC-SUBSTRING(X, n)

```
max ← 1
for i = 1 to n − 1 do
    p[i, i] ← true
    if x_i = x_{i+1} then
        p[i, i + 1] ← true;   max ← 2
    else
        p[i, i + 1] ← false
    end if
end for
p[n, n] ← true
for ℓ = 3 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1
        if x_i = x_j and p[i + 1, j − 1] then
            p[i, j] ← true;   max ← ℓ
        else
            p[i, j] ← false
        end if
    end for
end for
return max
```

Time $O(n^2)$ (there are $n^2/2$ intervals, each in $O(1)$), space $O(n^2)$, reducible to $O(n)$ by keeping only two diagonals.

9.12 Longest Palindromic Subsequence

A Subsequence Need Not Be Contiguous

Let $dp[i, j]$ be the length of the longest palindromic subsequence of $X[i..j]$. Base cases: $dp[i, i] = 1$; $dp[i, i + 1] = 2$ if $x_i = x_{i+1}$ and 1 otherwise. For $\ell \geq 3$:

$$dp[i, j] = \begin{cases} dp[i + 1, j - 1] + 2, & x_i = x_j, \\ \max\{dp[i + 1, j],\ dp[i, j - 1]\}, & x_i \neq x_j. \end{cases}$$

Store in $sol[i, j] \in \{1, 2, 3\}$ which case was used (1: take both ends, 2: drop x_i, 3: drop x_j).

Algorithm 42 LONGEST-PALINDROMIC-SUBSEQUENCE and PRINT-SOL

```
for i = 1 to n − 1 do
    dp[i, i] ← 1
    if x_i = x_{i+1} then
        dp[i, i + 1] ← 2;   sol[i, i + 1] ← 1
    else
        dp[i, i + 1] ← 1;   sol[i, i + 1] ← 2
    end if
end for
dp[n, n] ← 1
for ℓ = 3 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1
        if x_i = x_j then
            dp[i, j] ← dp[i + 1, j − 1] + 2;   sol[i, j] ← 1
        else if dp[i + 1, j] > dp[i, j − 1] then
            dp[i, j] ← dp[i + 1, j];   sol[i, j] ← 2
        else
            dp[i, j] ← dp[i, j − 1];   sol[i, j] ← 3
        end if
    end for
end for

procedure PRINT-SOL(i, j)
    if i > j then
        return
    end if
    if i = j then
        print x_i; return
    end if
    if sol[i, j] = 1 then
        print x_i; PRINT-SOL(i + 1, j − 1); print x_j
    else if sol[i, j] = 2 then
        PRINT-SOL(i + 1, j)
    else
        PRINT-SOL(i, j − 1)
    end if
end procedure
```

Time and space $O(n^2)$.

Can We Just Use LCS with the Reversed String?

- **Substring version: no.** X = abcdba, rev(X) = abdcba: the longest palindromic substring of X is a single letter (length 1), while the longest common substring of X and rev(X) is ab (or ba), length 2 — the common substring need not be a palindrome.
- **Subsequence version: yes, but only the length.** X = abcab, rev(X) = bacba: the longest palindromic subsequence has length 3 (aba, bcb), and the LCS of X and rev(X) also has length 3 (aba, bcb, acb, ...). The values of the LCS need not be palindromes, but the lengths agree (the proof is not immediate).

9.13 Matrix-Chain Multiplication

Cost model. The product of $A_{p\times q}$ and $B_{q\times r}$ is a $p \times r$ matrix C with $c_{ij} = \sum_{k=1}^{q} a_{ik} b_{kj}$; computing it takes $p\,q\,r$ scalar multiplications.

$$\begin{array}{ccccc} A & & B & & C \\ 5\times 4 & \times & 4\times 8 & = & 5\times 8 \end{array}$$

$5 \cdot 4 \cdot 8 = 160$ operations

Problem. Given a chain $A_1 A_2 \cdots A_n$ where A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product so that the number of scalar multiplications is minimum. Different parenthesizations give very different costs.

Three Matrices: $A_{10\times 100}$, $B_{100\times 5}$, $C_{5\times 50}$

Parenthesization	Cost of the two products	Total
$A(BC)$	BC: $100 \cdot 5 \cdot 50 = 25{,}000$; then $A \cdot (BC)$: $10 \cdot 100 \cdot 50 = 50{,}000$	75,000
$(AB)C$	AB: $10 \cdot 100 \cdot 5 = 5{,}000$; then $(AB) \cdot C$: $10 \cdot 5 \cdot 50 = 2{,}500$	**7,500**

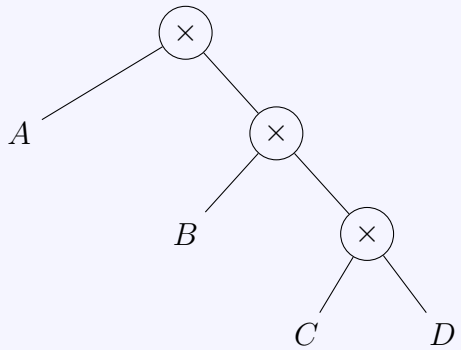
The same three matrices cost ten times more when the parenthesis is put on the wrong side.

Four Matrices: $A_{5\times 5}$, $B_{5\times 4}$, $C_{4\times 8}$, $D_{8\times 2}$

There are $C_3 = \frac{1}{4}\binom{6}{3} = 5$ parenthesizations (C_{n-1} is the $(n-1)$-st Catalan number):

#	Parenthesization and cost of each product	Total
1	$(A(B(CD)))$: $CD = 4 \cdot 8 \cdot 2 = 64$, $B(CD) = 5 \cdot 4 \cdot 2 = 40$, $A(\cdot) = 5 \cdot 5 \cdot 2 = 50$	**154**
2	$(A((BC)D))$: $BC = 160$, $(BC)D = 5 \cdot 8 \cdot 2 = 80$, $A(\cdot) = 50$	290
3	$((AB)(CD))$: $AB = 100$, $CD = 64$, $(\cdot)(\cdot) = 5 \cdot 4 \cdot 2 = 40$	204
4	$((A(BC))D)$: $BC = 160$, $A(BC) = 5 \cdot 5 \cdot 8 = 200$, $\cdot D = 5 \cdot 8 \cdot 2 = 80$	440
5	$(((AB)C)D)$: $AB = 100$, $(AB)C = 5 \cdot 4 \cdot 8 = 160$, $\cdot D = 80$	340

The optimum is $(A(B(CD)))$ with 154 multiplications; a flat binary tree picture of it is



internal nodes are multiplications; leaves are the matrices

Optimal Structure and Recurrence

Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i\cdots j} = A_i \cdots A_j$ (which has dimensions $p_{i-1} \times p_j$). An optimal parenthesization splits $A_{i\cdots j} = A_{i\cdots k} \cdot A_{k+1\cdots j}$ at some k with $i \le k < j$, and both factors must themselves be computed

optimally (otherwise a cheaper factor would lower the total). Hence

$$m[i,j] = \begin{cases} 0, & i = j, \\ \min\limits_{i \le k < j} \{ m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \}, & i < j, \end{cases}$$

where $p_{i-1}p_k p_j$ is the cost of the final multiplication of a $p_{i-1} \times p_k$ matrix by a $p_k \times p_j$ matrix.

Algorithm 43 MATRIX-CHAIN-ORDER(p, n)

```
for i = 1 to n do
    m[i, i] ← 0
end for
for ℓ = 2 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;   m[i, j] ← ∞
        for k = i to j − 1 do
            q ← m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
            if q < m[i, j] then
                m[i, j] ← q;   s[i, j] ← k
            end if
        end for
    end for
end for
return m[ ], s[ ]
```

Time $O(n^3)$ (there are $O(n^2)$ intervals, each trying $O(n)$ splits), space $O(n^2)$. The best known algorithm runs in $O(n \log n)$ (Hu and Shing).

Worked Example: $p = (5, 4, 6, 2, 7, 4)$

So $A_1 : 5 \times 4$, $A_2 : 4 \times 6$, $A_3 : 6 \times 2$, $A_4 : 2 \times 7$, $A_5 : 7 \times 4$.

$m[i,j]$	j=1	2	3	4	5
$i = 1$	0	120	88	158	184
$i = 2$	–	0	48	104	136
$i = 3$	–	–	0	84	104
$i = 4$	–	–	–	0	56
$i = 5$	–	–	–	–	0

$s[i,j]$	j=1	2	3	4	5
$i = 1$	–	1	1	3	3
$i = 2$	–	–	2	3	3
$i = 3$	–	–	–	3	3
$i = 4$	–	–	–	–	4
$i = 5$	–	–	–	–	–

Length-2 chains: $m[1,2] = 0 + 0 + 5 \cdot 4 \cdot 6 = 120$; $m[2,3] = 4 \cdot 6 \cdot 2 = 48$; $m[3,4] = 6 \cdot 2 \cdot 7 = 84$; $m[4,5] = 2 \cdot 7 \cdot 4 = 56$.
Length-3 chains: $m[1,3] = \min\{m[1,1]+m[2,3]+5\cdot4\cdot2,\ m[1,2]+m[3,3]+5\cdot6\cdot2\} = \min\{88,\ 180\} = 88$ $(k = 1)$; $m[2,4] = \min\{0 + 84 + 4 \cdot 6 \cdot 7,\ 48 + 0 + 4 \cdot 2 \cdot 7\} = \min\{252, 104\} = 104$ $(k = 3)$; $m[3,5] = \min\{0 + 56 + 6 \cdot 2 \cdot 4,\ 84 + 0 + 6 \cdot 7 \cdot 4\} = \min\{104, 252\} = 104$ $(k = 3)$.
Length-4 chains: $m[1,4] = \min\{0 + 104 + 140,\ 120 + 84 + 210,\ 88 + 0 + 70\} = \min\{244, 414, 158\} = 158$ $(k = 3)$; $m[2,5] = \min\{0 + 104 + 96,\ 48 + 56 + 32,\ 104 + 0 + 112\} = \min\{200, 136, 216\} = 136$ $(k = 3)$.
Length-5: $m[1,5] = \min\{0+136+80,\ 120+104+120,\ 88+56+40,\ 158+0+140\} = \min\{216, 344, 184, 298\} = 184$ $(k = 3)$.

Reconstruction: $s[1,5] = 3$, so the last multiplication is $(A_1A_2A_3)(A_4A_5)$; $s[1,3] = 1$ gives $(A_1)(A_2A_3)$; $s[2,3] = 2$ gives (A_2A_3); $s[4,5] = 4$ gives (A_4A_5). Hence

$$\big((A_1(A_2A_3))(A_4A_5)\big)$$

is optimal with cost 184. □

9.14 Optimal Binary Search Trees

A search in a BST with n keys takes $\Theta(\log n)$ in the worst case, but if we know the probabilities $f(a_i)$ of searching each key, an unbalanced tree can minimize the expected search cost.

The Optimal BST Problem

Given keys $a_1 < a_2 < \cdots < a_n$ with positive weights $f(a_1), \ldots, f(a_n)$, find a BST T on these keys minimizing

$$B(T) = \sum_{i=1}^{n} f(a_i)\big(d(a_i) + 1\big),$$

where $d(a_i)$ is the depth of a_i (so $d+1$ counts the comparisons, or the number of nodes on the root-to-a_i path). If f gives search probabilities, a minimum-B tree is the minimum expected-cost tree.

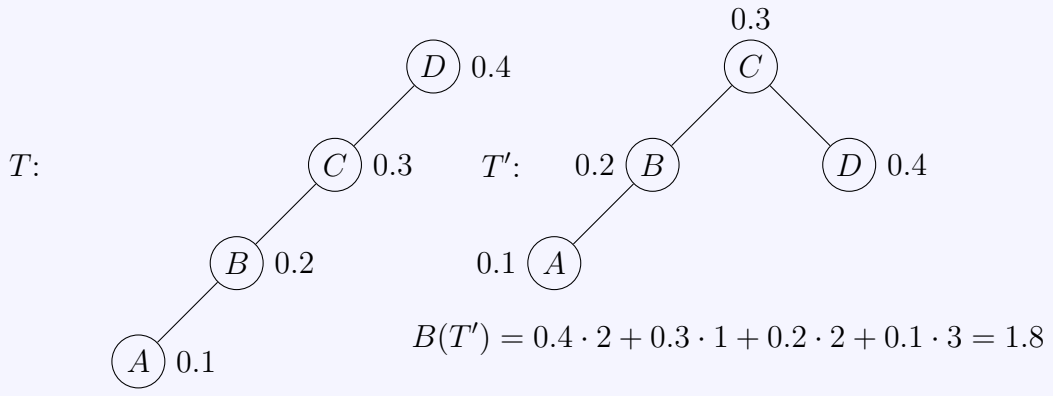
Two Differences from Huffman Coding

- The tree must be a **BST**: in-order traversal gives the sorted keys. A Huffman tree imposes no order on its leaves.
- Keys appear as **both internal and leaf nodes**; in Huffman coding characters appear only at leaves.

Consequently Huffman's greedy algorithm cannot be used.

The "Heaviest Key as Root" Greedy Fails

Keys $A < B < C < D$ with weights $f(A) = 0.1$, $f(B) = 0.2$, $f(C) = 0.3$, $f(D) = 0.4$.



$$B(T') = 0.4 \cdot 2 + 0.3 \cdot 1 + 0.2 \cdot 2 + 0.1 \cdot 3 = 1.8$$

$$B(T) = 0.4 \cdot 1 + 0.3 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 4 = 2.0$$

The greedy tree T has cost 2.0, but T' has cost 1.8: the heaviest key need not be the root.

Recurrence

Let $e[i,j]$ be the minimum cost of any BST on $a_i,\dots,a_j$, and let $w[i,j]=f(a_i)+\cdots+f(a_j)$ (all $w[i,j]$ are precomputed in $O(n^2)$ time). Suppose the root of the optimal tree $T_{i,j}$ is a_k; then its left subtree must be optimal for $[i,k-1]$ and its right subtree optimal for $[k+1,j]$ (cut-and-paste). Every node of both subtrees is one level deeper inside $T_{i,j}$, so

$$B(T_{i,j}) = e[i,k-1] + e[k+1,j] + \underbrace{w[i,k-1] + f(a_k) + w[k+1,j]}_{w[i,j]}.$$

Trying every possible root,

$$e[i,j] = \min_{i\le k\le j}\bigl\{e[i,k-1]+e[k+1,j]+w[i,j]\bigr\},$$

$$e[i,j]=0 \text{ for } i>j, \qquad e[i,i]=f(a_i).$$

Order: increasing interval length $j-i+1$; the answer is $e[1,n]$.

Algorithm 44 OPTIMAL-BST(f,n)

```
allocate e[1..n,1..n], w[1..n,1..n], root[1..n,1..n], all 0
for i = 1 to n do
    w[i,i] ← f(a_i)
    for j = i+1 to n do
        w[i,j] ← w[i,j−1] + f(a_j)
    end for
end for
for ℓ = 1 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;   e[i,j] ← ∞
        for k = i to j do
            t ← e[i,k−1] + e[k+1,j] + w[i,j]
            if t < e[i,j] then
                e[i,j] ← t;   root[i,j] ← k
            end if
        end for
    end for
end for
return CONSTRUCT-BST(root, 1, n)
```

```
procedure CONSTRUCT-BST(root, i, j)
    if i > j then
        return nil
    end if
    create node z;   z.key ← a[root[i,j]]
    z.left ← CONSTRUCT-BST(root, i, root[i,j] − 1)
    z.right ← CONSTRUCT-BST(root, root[i,j] + 1, j);   return z
end procedure
```

Time $O(n^3)$ for the table (splits $\times$ intervals) and $O(n)$ for the construction; space $O(n^2)$. The Knuth–Yao speedup reduces the table to $O(n^2)$.

Worked Example: Weights $f = (5, 20, 30, 50, 10, 20, 10)$ on $A, \ldots, G$

$e[i,j]$	j=1	2	3	4	5	6	7
$i=1$ (A)	5	30	85	185	210	260	290
$i=2$ (B)	–	20	70	170	190	240	270
$i=3$ (C)	–	–	30	110	130	180	210
$i=4$ (D)	–	–	–	50	70	120	150
$i=5$ (E)	–	–	–	–	10	40	60
$i=6$ (F)	–	–	–	–	–	20	40
$i=7$ (G)	–	–	–	–	–	–	10

$root[i,j]$	j=1	2	3	4	5	6	7
$i=1$	1	2	3	3	4	4	4
$i=2$	–	2	3	3	4	4	4
$i=3$	–	–	3	4	4	4	4
$i=4$	–	–	–	4	4	4	4
$i=5$	–	–	–	–	5	6	6
$i=6$	–	–	–	–	–	6	6
$i=7$	–	–	–	–	–	–	7

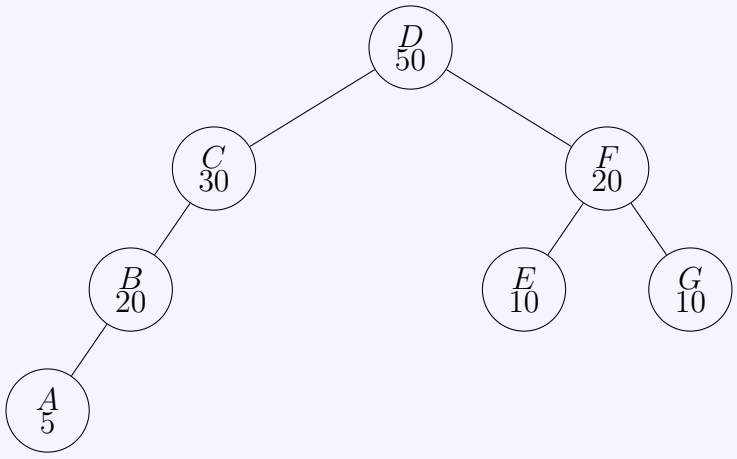
Selected computations ($w[i,j]$ is the sum of the weights on $[i,j]$):

$$e[1,2] = \min\{0+20+25,\ 5+0+25\} = \min\{45, 30\} = 30 \quad (k=2),$$

$$e[1,4] = \min\{0+170+105,\ 5+110+105,\ 30+50+105,\ 85+0+105\}$$
$$= \min\{275, 220, 185, 190\} = 185 \quad (k=3),$$

$$e[1,7] = \min\{415,\ 360,\ 325,\ \mathbf{290},\ 370,\ 365,\ 405\} = 290 \quad (k=4).$$

The optimal tree: root $root[1,7] = 4$ (D); left part $[1,3]$ has root $root[1,3] = 3$ (C) with left part $[1,2]$ rooted at B (left child A); right part $[5,7]$ has root $root[5,7] = 6$ (F) with left child E and right child G:



Cost check: $50{\cdot}1+30{\cdot}2+20{\cdot}2+20{\cdot}3+10{\cdot}3+10{\cdot}3+5{\cdot}4 = 50+60+40+60+30+30+20 = 290$, matching $e[1,7]$. □

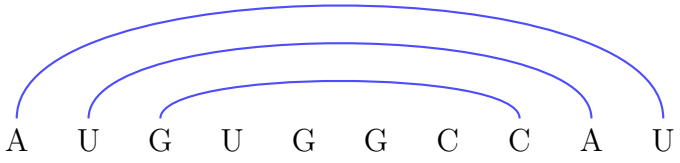
9.15 RNA Secondary Structure

Problem. An RNA molecule is a sequence $B = b_1 b_2 \cdots b_n$ over $\{A, U, C, G\}$. A secondary structure is a set S of pairs (b_i, b_j) such that

(1) Watson–Crick: S is a matching and every pair is A–U, U–A, C–G or G–C;
(2) No sharp turns: if $(b_i, b_j) \in S$ then $i < j - 4$ (at least 4 bases between the two ends);

(3) Non-crossing: there are no $(b_i, b_j), (b_k, b_l) \in S$ with $i < k < j < l$.

Goal: maximize $|S|$ (the number of base pairs), which is assumed proportional to the (negative) free energy.



$S = \{(1,10),(2,9),(3,8)\}$, valid: nested pairs, no sharp turns, $|S| = 3$

Recurrence

Let $M[i,j]$ be the maximum number of pairs in a secondary structure of $b_i \cdots b_j$.

- If $i \geq j-4$: $M[i,j] = 0$ by the no-sharp-turn condition.
- Otherwise either b_j is unmatched, giving $M[i,j-1]$; or b_j pairs with some b_k, $i \leq k \leq j-5$, and the non-crossing condition decouples the rest into the independent intervals $[i,k-1]$ and $[k+1,j-1]$:

$$M[i,j] = \max\Big\{ M[i,j-1],\ \max_{\substack{i \leq k \leq j-5 \\ b_k, b_j \text{ complementary}}} \big(1 + M[i,k-1] + M[k+1,j-1]\big) \Big\}.$$

The answer is $M[1,n]$, filled by increasing length.

Algorithm 45 RNA(b,n) and Construct-RNA

```
allocate M[1..n, 1..n], s[1..n, 1..n], all 0
for ℓ = 1 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;   M[i, j] ← M[i, j − 1]
        for k = i to j − 5 do
            if b_k and b_j are complementary then
                t ← 1 + M[i, k − 1] + M[k + 1, j − 1]
                if t > M[i, j] then
                    M[i, j] ← t;   s[i, j] ← k
                end if
            end if
        end for
    end for
end for

procedure Construct-RNA(i, j)
    if i ≥ j − 4 then
        return
    end if
    if s[i, j] = 0 then
        Construct-RNA(i, j − 1)
    end if
    print "s[i, j]–j";   Construct-RNA(i, s[i, j] − 1);   Construct-RNA(s[i, j] + 1, j − 1)
end procedure
```

Time $O(n^3)$ ($O(n^2)$ intervals, $O(n)$ partner choices each), space $O(n^2)$. With the total number of bases fixed, a secondary structure is found by tracing s from $(1,n)$ downwards.

10 Graph Algorithms

Graphs model relationships, so they are ubiquitous. Rather than solving one application, we design algorithms for abstract graphs and then reuse them everywhere.

Application	Nodes	Edges
Transportation	street intersections	highways
Communication	computers	fiber-optic cables
World Wide Web	web pages	hyperlinks (directed)
Social network	people	relationships
Food web	species	predator–prey
Software systems	functions	function calls
Scheduling	tasks	precedence constraints
Circuits	gates	wires

Two canonical graph problems motivate this chapter.

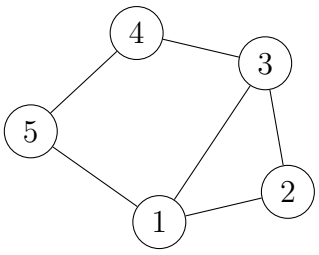
- **Travelling Salesman Problem (TSP).** Given a graph whose edges carry lengths, find a cycle of minimum total weight visiting every vertex. No polynomial-time algorithm is known; the problem is equivalent to $P = NP$.
- **Topological ordering.** Given a directed graph, order the vertices so that for every edge $u \to v$, u comes before v — or report that the graph contains a cycle.

10.1 Graph Basics

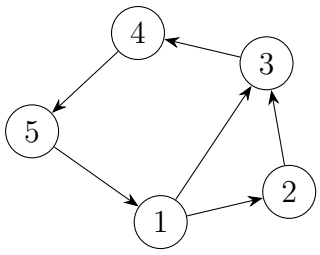
Definition

A graph is a pair $G = (V, E)$ where V is a set of nodes (vertices) and E is a set of edges between pairs of nodes. We abuse notation and also write V and E for the counts $|V|$ and $|E|$, and often set $n = |V|$, $m = |E|$.

- **Undirected graph:** edges have no direction. An edge is an unordered pair $\{u, v\}$.
- **Directed graph:** edges have directions; an edge is an ordered pair (u, v). If an edge exists in both directions we represent it with two opposite directed edges.
- **Simple graph:** at most one edge between a pair of nodes (at most two, in opposite directions, for a directed graph). No self-loops, no duplicate edges.



undirected graph



directed graph

Degrees. In an undirected graph, $\deg(v)$ is the number of edges touching v. In a directed graph, $\deg_{out}(v)$ counts edges leaving v and $\deg_{in}(v)$ counts edges entering v.

Handshaking identity

For every undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$: each edge contributes exactly 1 to the

degree of each of its two endpoints. For every directed graph,

$$\sum_{v \in V} \deg_{out}(v) = \sum_{v \in V} \deg_{in}(v) = |E|,$$

because each edge has exactly one tail and one head.

Maximum number of edges. Each node can connect to at most $V - 1$ other nodes, so a directed simple graph has at most $V(V-1)$ edges. Replacing each pair of opposite directed edges with a single undirected edge gives at most $V(V-1)/2$ edges in an undirected simple graph. Thus

$$0 \le |E| \le \frac{|V|(|V|-1)}{2} \text{ (undirected)}, \qquad 0 \le |E| \le |V|(|V|-1) \text{ (directed)}.$$

Degree parity exercises

Problem. (a) Show that if every node of a graph has degree 3, then $|V|$ is even. (b) Show that if every node has odd degree, then $|V|$ is even.
Solution. (a) By handshaking, $3|V| = 2|E|$. The right side is even, hence $3|V|$ is even, hence $|V|$ is even.
(b) Write $d(v_i) = 2k_i + 1$ with $k_i \in \mathbb{Z}$. Then

$$\sum_{i=1}^{|V|} d(v_i) = \sum_{i=1}^{|V|} (2k_i + 1) = |V| + 2\sum_{i=1}^{|V|} k_i = 2|E|,$$

so $|V| = 2|E| - 2\sum_i k_i$ is a difference of two even numbers and therefore even. □

The party handshake

Problem. At a party, show that the number of guests who shake hands an odd number of times must be even.
Solution. Build $G = (V, E)$ with one node per guest and one edge per handshake. The claim becomes: in every graph, the number of odd-degree nodes is even. Split $V = V_{odd} \cup V_{even}$. The sum of the even degrees is even, and $\sum_v \deg(v) = 2|E|$ is even, so the sum of the odd degrees is even. Writing each odd degree as $2k_i + 1$,

$$\sum_{v_i \in V_{odd}} \deg(v_i) = |V_{odd}| + 2\sum_{v_i \in V_{odd}} k_i \text{ is even} \implies |V_{odd}| \text{ is even}.$$

□

Two degree puzzles

Problem 1. In a group of 8 people some pairs shake hands. Is it possible that everyone shakes hands with a different number of people?
Solution. Each person's handshake count lies in $\{0, 1, \ldots, 7\}$, seven values for eight people, so two must coincide — unless the counts are exactly $0, 1, \ldots, 7$. But the person with 7 handshakes shook hands with everybody, including the person with 0, a contradiction. Hence at least two people have the same count, so it is not possible. □
Problem 2. In a simple connected graph on 6 vertices, five vertices have degrees $1, 2, 3, 4, 5$. What is the degree of the sixth vertex?
Solution 1. Name the vertices $v_1, \ldots, v_5$ with $\deg(v_i) = i$ and let v_6 be the unknown one. Since $\deg(v_5) = 5$, vertex v_5 is adjacent to all others, so the only neighbour of v_1 is v_5. Since $\deg(v_4) = 4$, v_4 is adjacent to every vertex except v_1; hence the two

neighbours of v_2 are v_4 and v_5. Thus v_6 is adjacent to v_4 and v_5, and is not adjacent to v_1 or v_2; the same is true of v_3. Since $\deg(v_3) = 3$, v_3 must be adjacent to v_6. Therefore v_6 is adjacent exactly to v_4, v_5, v_3, so $\deg(v_6) = 3$.
Solution 2. The sum of degrees is even, so the missing degree is odd: 1, 3 or 5. As above, v_6 is adjacent to v_4 and v_5 but not to v_1 or v_2, which rules out 5 (needs adjacency to v_1, v_2) and 1 (needs adjacency only to one vertex). Hence $\deg(v_6) = 3$. □

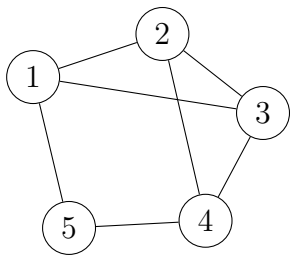
10.2 Graph Representation

Operation	Adjacency list	Adjacency matrix
Storage	$\Theta(\|V\| + \|E\|)$	$\Theta(\|V\|^2)$
List all neighbours of u	$\Theta(\deg(u))$	$\Theta(\|V\|)$
Test whether $(u, v) \in E$	$\Theta(\deg(u))$	$O(1)$

An **adjacency list** is a node-indexed array of lists: node u stores exactly its neighbours. An **adjacency matrix** is a $|V| \times |V|$ table with $A[u][v] = 1$ iff $(u, v) \in E$. One can convert between the two representations in $\Theta(|V|^2)$ time.

Which representation?

Adjacency lists are used far more often because most real graphs are sparse ($|E| \ll |V|^2$); they also let BFS/DFS enumerate neighbours in time proportional to the degree. Matrices win when the graph is dense or when $O(1)$ edge queries are essential. Both BFS and DFS run in $\Theta(|V| + |E|)$ with adjacency lists, but $\Theta(|V|^2)$ with a matrix.

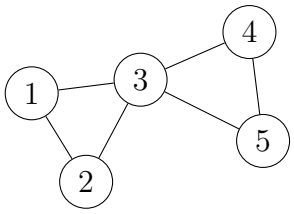


adjacency list:
1: 2, 5, 3
2: 1, 3, 4
3: 2, 1, 4
4: 3, 2, 5
5: 4, 1

10.3 Paths, Connectivity, and Trees

Paths and cycles

A **path** in G is a sequence of nodes $v_1, v_2, \ldots, v_k$ such that $(v_i, v_{i+1}) \in E$ for every i; its **length** is $k - 1$, the number of edges. A path is **simple** if all its nodes are distinct. A **cycle** is a path with $v_1 = v_k$ and $k > 2$; it is **simple** if $v_1, \ldots, v_{k-1}$ are all distinct. An undirected graph is **connected** if every pair of nodes is joined by a path. A **tree** is a connected undirected graph with no cycle; a **forest** is a graph with no cycle (a collection of trees).



simple cycle: $1, 3, 2, 1$; nonsimple cycle: $2, 1, 3, 2, 4, 5, 3, 2$

Tree characterisation

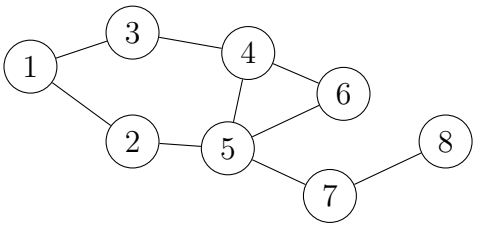
Let G be an undirected graph. Any two of the following statements imply the third, and hence G is a tree: (1) G is connected; (2) G contains no cycle; (3) $|E| = |V| - 1$. In particular, a connected graph always satisfies $|E| \geq |V| - 1$.

$|E| \geq |V| - 1$ for connected graphs

Start from a spanning tree on all $|V|$ nodes (it has exactly $|V| - 1$ edges) and add the remaining edges one by one: connectivity is preserved and every added edge creates a cycle. Since a forest on $|V|$ nodes has at most $|V| - 1$ edges, any connected graph has at least $|V| - 1$ edges, with equality iff it is a tree.

s–t connectivity and shortest paths

The s–t **connectivity problem** asks whether a path from s to t exists. In a directed graph this is called **reachability** and is not symmetric: s may reach t while t cannot reach s.
The s–t **shortest path problem** asks for the minimum number of edges (or, with weights, the minimum total weight) on an s–t path; that number is the **distance** from s to t. In an undirected graph $s \rightsquigarrow t$ and $t \rightsquigarrow s$ have the same distance, but in a directed graph they may differ.



path $6, 5, 7, 8$ has length 3 from 6 to 8

Wireless network connectivity

Problem. In a wireless network of n devices (n even), each device is within range of at least $n/2$ other devices. Show all devices are connected.
Solution. Model the network as an undirected graph G in which every node has degree $\geq n/2$. Take any two nodes u, v.
If $(u, v) \in E$ they are connected. Otherwise consider the $n - 2$ nodes other than u, v: each of u and v has at least $n/2$ neighbours among these $n - 2$ nodes, and

$$\frac{n}{2} + \frac{n}{2} - (n - 2) = 2 > 0,$$

so by the pigeonhole principle they share a common neighbour w, giving the path u, w, v. Since u, v were arbitrary, G is connected. □
Follow-up. If the threshold is lowered to $n/2 - 1$ the claim fails: take two disjoint cliques of $n/2$ nodes each. Every node has degree $n/2 - 1$ but no edge crosses between the two cliques, so the graph is disconnected.

Rooted trees

Given a tree T, choose a **root** r and orient every edge away from r. The result is a **rooted tree**: every node $v \neq r$ has exactly one **parent** (its predecessor toward r) and zero or more **children**. A node with no children is a **leaf**. This is exactly the structure produced by the BFS and DFS trees below.

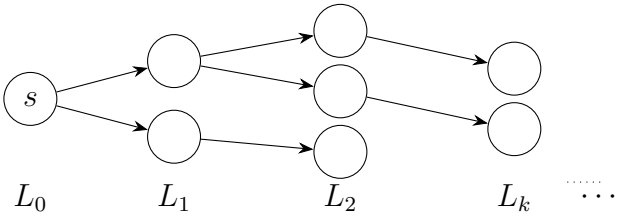
10.4 Breadth-First Search

BFS layers

Breadth-first search explores outward from a start node s in all directions, adding nodes one layer at a time:

$$L_0 = \{s\}, \qquad L_{i+1} = \{v \notin L_0 \cup \cdots \cup L_i : \ (u, v) \in E \text{ for some } u \in L_i\}.$$

L_1 is the set of neighbours of s; L_2 consists of the still unseen nodes with an edge into L_1; and so on. The **distance** from u to v is the number of edges on a shortest u–v path.



BFS layer theorem

For every i, the layer L_i consists of exactly the nodes at distance i from s. Consequently there is a path from s to t if and only if t appears in some layer.

Each node stores three fields:

- **color**: white = undiscovered, **gray** = discovered but neighbours not fully processed, black = discovered and fully processed;
- **distance** d: length of a shortest s–u path;
- **parent** p: predecessor of u on that shortest path.

Algorithm 46 BFS from a single source s

```
1:  procedure BFS(G, s)
2:      for each vertex u ∈ V − {s} do
3:          u.color ← white;   u.d ← ∞;   u.p ← nil
4:      end for
5:      s.color ← gray;   s.d ← 0
6:      initialize an empty queue Q;   ENQUEUE(Q, s)
7:      while Q ≠ ∅ do
8:          u ← DEQUEUE(Q)
9:          for each v ∈ Adj[u] do
10:             if v.color = white then
11:                 v.color ← gray;   v.d ← u.d + 1;   v.p ← u
12:                 ENQUEUE(Q, v)
13:             end if
14:         end for
15:         u.color ← black
16:     end while
17: end procedure
```

Running time and BFS tree

Each node enters and leaves the queue at most once, and its adjacency list is scanned exactly once when it is dequeued:

$$\sum_{u} (1 + \deg(u)) = \Theta(|V| + |E|),$$

which is $\Theta(|E|)$ when the graph is connected. The parent pointers form a **BFS tree** rooted at s: the parent of every node lies one layer earlier, and following parent pointers from v back to s yields a shortest path. Hence BFS computes the shortest path from s to every other node in unweighted graphs.

Queue contents

The nodes inside Q are exactly the gray nodes: already seen, but not yet fully processed. A node turns black only after all of its neighbours have been examined. Because Q is FIFO, nodes leave in nondecreasing order of distance, which is precisely why layers are completed one at a time.

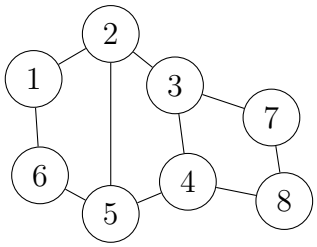
10.5 Connected Components and Reachability

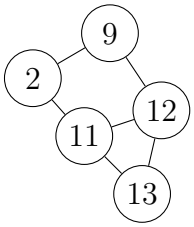
Connected components via BFS

The **connected component** of s is the set of all nodes reachable from s; running BFS from s finds exactly this set. To find all components, rename the single-source procedure BFS-Visit and repeatedly launch it from any still-white node.

Algorithm 47 Finding all connected components

```
1:  procedure BFS-Visit(G, s)
        assumes s is white
2:      s.color ← gray;   s.d ← 0
3:      initialize an empty queue Q;   Enqueue(Q, s)
4:      while Q ≠ ∅ do
5:          u ← Dequeue(Q)
6:          for each v ∈ Adj[u] do
7:              if v.color = white then
8:                  v.color ← gray;   v.d ← u.d + 1;   v.p ← u;   Enqueue(Q, v)
9:              end if
10:         end for
11:         u.color ← black
12:     end while
13: end procedure
14: procedure BFS(G)
15:     for each vertex u ∈ V do
16:         u.color ← white;   u.d ← ∞;   u.p ← nil
17:     end for
18:     for each vertex u ∈ V do
19:         if u.color = white then
20:             BFS-Visit(G, u)
21:         end if
22:     end for
23: end procedure
```

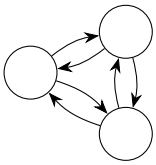




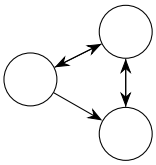
BFS-Visit(1) blackens $\{1, \ldots, 8\}$, BFS-Visit(2) blackens $\{2, 9, 10, 11, 12, 13\}$ (the component containing 2), and every call from a node already blackened does nothing. Each vertex is initialised once and scanned once, so the total time is still $\Theta(|V| + |E|)$.

Directed reachability and strong connectivity

Problem. For a directed graph, decide whether every node can reach every other node.
Solution. Pick any node s and run BFS from s in G. Then build G^{rev} (all edges reversed) and run BFS from s in G^{rev}. Return true iff both searches reach all nodes. Why one source suffices: call s strong if s reaches every t and every t reaches s. If some strong vertex exists, then every vertex is strong (compose $u \to s \to v$ and $v \to s \to u$), so the graph is strongly connected iff a single fixed s passes the two tests. □



strongly connected



not strongly connected

Algorithm 48 Strongly connected components (simple version)

```
1:  procedure StronglyConnectedComponents(G)
2:      create G^rev, the graph G with every edge reversed
3:      while there are nodes left do
4:          u ← any remaining node
5:          run BFS in G from u;   run BFS in G^rev from u
6:          C ← {nodes reached in both BFS runs}
7:          output C as a strongly connected component
8:          remove C and all its edges from G and G^rev
9:      end while
10: end procedure
```

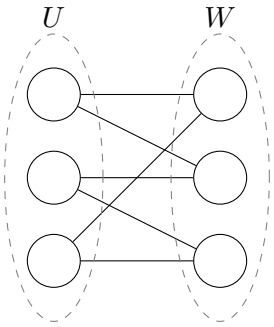
Each round removes at least one node and costs $O(|V| + |E|)$, so the procedure runs in $O(|V||E|)$; a $\Theta(|V| + |E|)$ algorithm exists but is outside the scope of this course.

Bipartite graphs

A graph is **bipartite** if V can be partitioned into two disjoint sets U and W such that every edge has one endpoint in U and the other in W. To test it on a connected undirected graph:

- run BFS from any vertex and let $d[v]$ be the distance computed;
- put the even-distance vertices in S and the odd-distance ones in $V - S$;
- the graph is bipartite iff every edge $\{u, v\}$ joins vertices of different parity.

Equivalently, a graph is bipartite iff it contains no odd cycle: during BFS, an edge whose two endpoints sit in the same layer immediately certifies an odd cycle and non-bipartiteness.



BFS modelling exercises

1. Knight moves. Find the minimum number of knight moves from (x, y) to (x', y') on a chessboard.
Solution. Build an implicit graph whose nodes are board squares, and whose edges join squares one knight move apart: from (x, y) the neighbours are $(x \pm 2, y \pm 1)$ and $(x \pm 1, y \pm 2)$ whenever they stay on the board. Run BFS from (x, y); the first time (x', y') is discovered its distance is the minimum number of moves. □
2. Binary maze. Given a binary matrix, find the length of a shortest path from (x, y) to (x', y') using only cells equal to 1, moving up/down/left/right.
Solution. Same reduction: nodes are the cells with value 1, edges join vertically or horizontally adjacent cells. BFS from (x, y); the distance assigned to (x', y') is the answer. Both problems run in time proportional to the number of nodes plus edges of the implicit graph. □

10.6 Worked BFS Example

Consider the undirected graph with the following adjacency lists.

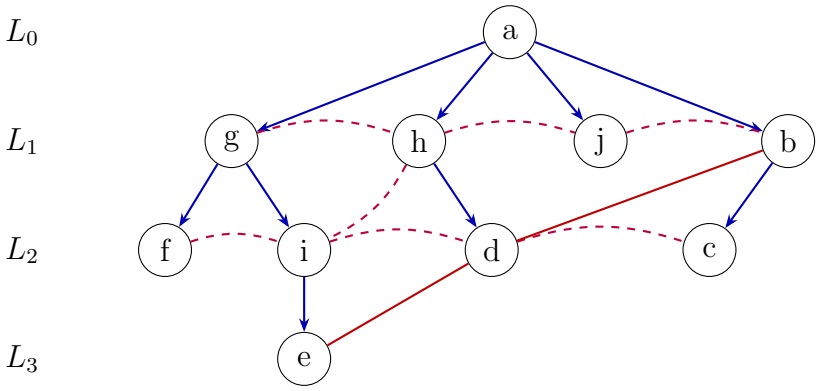
Vertex	Adjacency list
a	g, h, j, b
b	a, j, d, c
c	d, b
d	e, i, h, b, c
e	i, d
f	g, i
g	a, h, f, i
h	a, g, i, d, j
i	f, g, h, d, e
j	h, a, b

Run BFS from a. The queue evolves as follows.

Step	Dequeued u	Newly discovered (distance, parent)	Q afterwards
1	a	$g(1,a)$, $h(1,a)$, $j(1,a)$, $b(1,a)$	g, h, j, b
2	g	$f(2,g)$, $i(2,g)$	h, j, b, f, i
3	h	$d(2,h)$	j, b, f, i, d
4	j	—	b, f, i, d
5	b	$c(2,b)$	f, i, d, c
6	f	—	i, d, c
7	i	$e(3,i)$	d, c, e
8	d	—	c, e
9	c	—	e
10	e	—	$\emptyset$

Hence $L_0 = \{a\}$, $L_1 = \{g, h, j, b\}$, $L_2 = \{f, i, d, c\}$, $L_3 = \{e\}$, and the distances are

$$d(a) = 0, \quad d(g) = d(h) = d(j) = d(b) = 1, \quad d(f) = d(i) = d(d) = d(c) = 2, \quad d(e) = 3.$$



Solid blue arrows are the tree edges of the BFS tree ($9 = |V| - 1$ of them). Purple dashed arcs join nodes on the same level; the two red edges (b–d and d–e) join consecutive levels. Note that h–d is the tree edge, while b–d and d–e are non-tree edges.

Reading the figure

The first time a node u is seen is when a neighbour v is processed off the queue; then u turns gray, receives $d(u) = d(v) + 1$ and parent $p(u) = v$. The edges $(u, p(u))$ form the BFS tree. All tree edges go from level i to level $i+1$, so a path in this tree realises the shortest distance.

10.7 Depth-First Search

BFS is broad: when a node is processed it scans all its neighbours before any other node is processed. Depth-first search instead goes deep: when u examines a neighbour v, it immediately starts processing v (which starts processing the children of v, and so on); only after the whole subtree below v is finished does u resume with its next neighbour.

Algorithm 49 Depth-first search

```
1:  procedure DFS(G)
2:      for each vertex u ∈ V do
3:          u.color ← white;   u.p ← nil
4:      end for
5:      for each vertex u ∈ V do
6:          if u.color = white then
7:              DFS-Visit(u)
8:          end if
9:      end for
10: end procedure
11: procedure DFS-Visit(u)
12:     u.color ← gray
13:     for each v ∈ Adj[u] do
14:         if v.color = white then
15:             v.p ← u;   DFS-Visit(v)
16:         end if
17:     end for
18:     u.color ← black
19: end procedure
```

Running time and DFS tree

Every vertex is visited exactly once and every adjacency entry is inspected exactly once, so the running time is $\Theta(|V|+|E|)$. The parent pointers form the **DFS tree**; its edges are exactly the pairs $(u, p(u))$. Before DFS-Visit(u) returns, all nodes of the connected component containing u are blackened, so DFS-Visit is entered exactly once per connected component of G.

Colors mean different things

Gray means on the current recursion stack; black means finished. A gray neighbour is an ancestor in the DFS tree (a back edge), whereas a black neighbour can belong to an unrelated subtree. This distinction drives the edge classification and the cycle-detection algorithms below.

10.8 Worked DFS Example

Use the following undirected graph (the same graph as the lecture example).

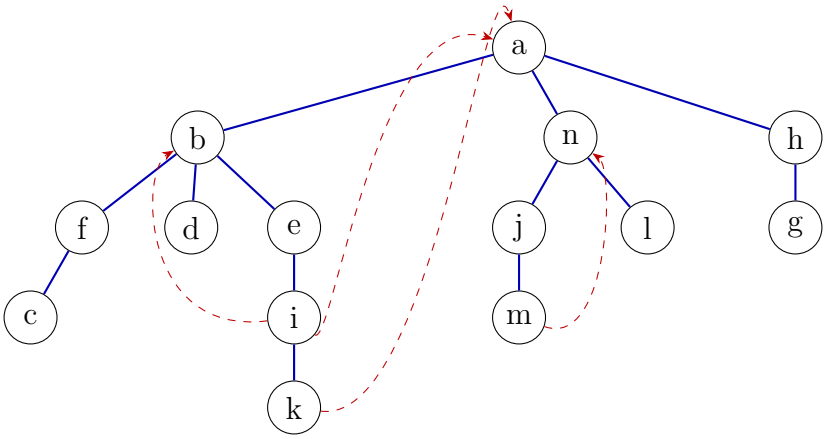
Vertex	Adjacency list
a	b, i, k, n, h
b	a, f, d, e, i
c	f
d	b
e	b, i
f	c, b
g	h
h	a, g
i	e, b, k, a
j	n, m
k	i, a
l	n
m	j, n
n	a, j, m, l

Starting at a and scanning each adjacency list from left to right, the recursion proceeds

as follows:

$$a \to b \to f \to c, \quad b \to d, \quad b \to e \to i \to k,$$
$$a \to n \to j \to m, \quad n \to l, \quad a \to h \to g.$$

- discovery order: $a, b, f, c, d, e, i, k, n, j, m, l, h, g$;
- finish order: $c, f, d, k, i, e, b, m, j, l, n, g, h, a$.



Bold blue edges are the $13 = |V| - 1$ tree edges. The four red dashed arcs are the **back edges**, each pointing from a node to a proper ancestor: (i, b), (i, a), (k, a) and (m, n). The first back edge encountered is (i, b); the DFS-tree path $b \to e \to i$ together with the edge (i, b) closes the cycle b, e, i, b.

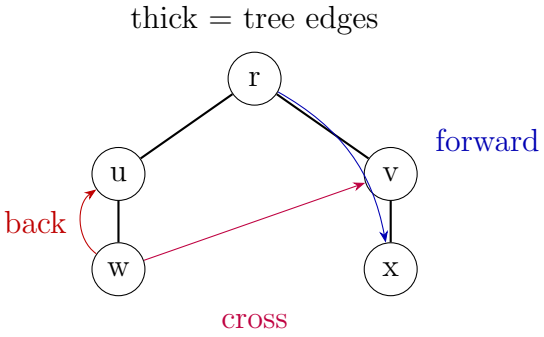
Why no other edge types appear

In an undirected DFS every non-tree edge joins a node to an ancestor, i.e. is a back edge. There are no forward or cross edges in undirected graphs — this is proved in the next subsection.

10.9 Classifying Edges

After a BFS or DFS, every edge of G falls into exactly one of four classes:

- **tree edges**: the edges traversed by the search (the parent pointers);
- **back edges**: from a node to a proper ancestor in the search tree;
- **forward edges**: from a node to a proper descendant (other than a child);
- **cross edges**: between two nodes with no ancestor/descendant relationship.



Theorem: undirected DFS has no cross edges

In a DFS of an undirected graph, every edge is a tree edge or a back edge.

Proof. Let (u, v) be any edge and assume without loss of generality that u is discovered first. When u's adjacency list is scanned, v cannot be black yet: if v were black, then u would have finished before v was discovered, so when u scanned (u, v) the node v was white and would have become a child of u — contradicting that u finished first. Hence at that moment v is either white or gray. If v is white, (u, v) becomes a tree edge. If v is gray, then v was discovered while u was still gray, so v lies in the DFS subtree of u; if $p(v) = u$ the edge is a tree edge, otherwise it joins v to a proper ancestor u and is a back edge. No edge can join two incomparable subtrees, so there are no cross edges. □

Theorem: undirected BFS has no forward or back edges

In a BFS of an undirected graph, every non-tree edge is a cross edge; equivalently every edge joins two nodes on the same level or on consecutive levels.

Proof. Let $(u, v) \in E$. Because v is a neighbour of u, BFS gives

$$|d(u) - d(v)| \leq 1 :$$

whichever endpoint is dequeued first relaxes the other with distance at most one more. A forward or back edge would join a node to a proper ancestor or descendant, and in a BFS tree such nodes differ in level by at least 2, contradicting $|d(u) - d(v)| \leq 1$. Hence no forward or back edges exist, leaving only tree edges and cross edges. □

10.10 Cycle Detection

Counting edges per component

A connected component is a tree iff it has exactly $|V_c| - 1$ edges. Run BFS/DFS to find all components and count the edges inside each one: if some component has at least as many edges as vertices, it contains a cycle. Time $\Theta(|V| + |E|)$.

Cycle detection by DFS back edges

Run DFS on each component. If (u, v) is a back edge, then v is a proper ancestor of u, so the DFS-tree path from v down to u plus the edge (u, v) is a cycle. If no back edge exists, the component consists only of tree edges, hence is a tree. Only tree edges are traversed before the first non-tree edge is found, and there are at most $|V| - 1$ of them, so the search costs $\Theta(|V|)$.

Algorithm 50 Cycle detection with DFS

```
 1: procedure CYCLE-DETECTION(G)
 2:     for each vertex u ∈ V do
 3:         u.color ← white;   u.p ← nil
 4:     end for
 5:     for each vertex u ∈ V do
 6:         if u.color = white then
 7:             DFS-VISIT(u)
 8:         end if
 9:     end for
10:     return "No cycle"
11: end procedure
12: procedure DFS-VISIT(u)
13:     u.color ← gray
14:     for each v ∈ Adj[u] do
15:         if v.color = white then
16:             v.p ← u;   DFS-VISIT(v)
17:         else if v ≠ u.p then
            back edge (u, v) found
18:             output "Cycle found:"
19:             while u ≠ v do
20:                 output u;   u ← u.p
21:             end while
22:             output v;   return
23:         end if
24:     end for
25:     u.color ← black
26: end procedure
```

Cycle detection by BFS cross edges

Run BFS on each component. If (u, v) is a cross edge, let $\ell = \mathrm{lca}(u, v)$ be the lowest common ancestor of u and v in the BFS tree; then the path $\ell \rightsquigarrow u$, the path $\ell \rightsquigarrow v$ and the edge (u, v) form a cycle. If no cross edge exists the component is a tree.
To find ℓ: if $d(u) = d(v)$ climb from both nodes simultaneously; if $d(u) > d(v)$, first climb $d(u) - d(v)$ steps from u, then climb together. This costs $O(|V|)$ per edge, so the whole procedure is $O(|V||E|)$ in the worst case.

Unique simple paths

Problem. Given a directed graph G and a node s, decide whether s has a unique simple path to every other node.
Solution. Run BFS/DFS from s and first check that every node is reachable. Then classify every edge. Tree edges and back edges are harmless (a back edge only closes a cycle, which a simple path cannot use, so it creates no second simple path). A forward edge or a cross edge (u, v) gives a second route from s to v: the tree path $s \rightsquigarrow u$ followed by (u, v), which differs from the tree path $s \rightsquigarrow v$ because u is not the parent of v. Hence the answer is "yes" iff no forward and no cross edge exists.
Classifying an edge (u, v) while scanning the adjacency list of u:

Colour of v	DFS	BFS
white	tree edge	tree edge
gray	back edge	back edge
black	forward or cross edge	cross edge or back edge (ambiguous)

BFS has no forward edges, but it cannot always separate cross from back edges; DFS is the right tool for this test. □

10.11 Topological Sorting

DAGs and topological order

A **directed acyclic graph** (DAG) is a directed graph with no cycle. A **topological ordering** of a DAG is a linear ordering of its vertices such that for every edge (u, v), u appears before v. The order is usually not unique: a DAG with 10 vertices may admit many valid orders.

Key observation

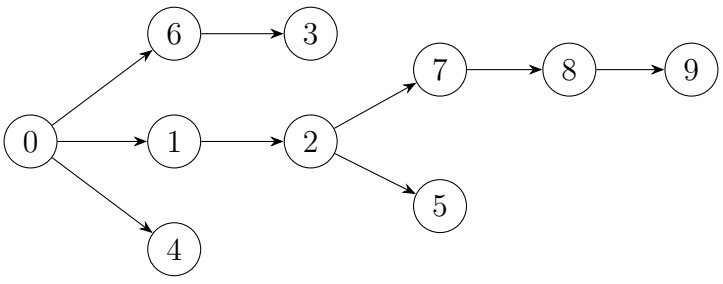
Every DAG has at least one vertex with in-degree 0. (Otherwise, following incoming edges backwards forever must repeat a vertex and hence produce a cycle.)

Algorithm 51 Topological sort (Kahn's algorithm)

```
1:  procedure TOPOLOGICAL-SORT(G)
2:      initialise an empty queue Q
3:      for each u ∈ V do
4:          if indegree(u) = 0 then
5:              ENQUEUE(Q, u)
6:          end if
7:      end for
8:      while Q is not empty do
9:          u ← DEQUEUE(Q);   output u
10:         for each v ∈ Adj(u) do
11:             indegree(v) ← indegree(v) − 1
12:             if indegree(v) = 0 then
13:                 ENQUEUE(Q, v)
14:             end if
15:         end for
16:     end while
17: end procedure
```

Correctness. Removing a vertex together with its outgoing edges from a DAG leaves a DAG, so at every stage the current graph has a vertex of in-degree 0 and the algorithm can always output another vertex; hence all vertices are output. Suppose the output order were not topological: then some edge (u, v) would have v output before u. But v could only enter the queue when its in-degree dropped to 0, which requires the edge (u, v) to have been removed, which requires u to have been output already — a contradiction. Therefore the output is a topological order.

If the graph is not acyclic, the process gets stuck before outputting all $|V|$ vertices; comparing the number of outputs with $|V|$ therefore also detects cycles. With adjacency lists the running time is $\Theta(|V| + |E|)$.



Dequeued	Q afterwards	Output so far
—	0	—
0	6, 1, 4	0
6	1, 4, 3	0,6
1	4, 3, 2	0,6,1
4	3, 2	0,6,1,4
3	2	0,6,1,4,3
2	7, 5	0,6,1,4,3,2
7	5, 8	0,6,1,4,3,2,7
5	8	0,6,1,4,3,2,7,5
8	9	0,6,1,4,3,2,7,5,8
9	$\emptyset$	0,6,1,4,3,2,7,5,8,9

The resulting order $0, 6, 1, 4, 3, 2, 7, 5, 8, 9$ is one valid topological order; for instance $0, 4, 1, 6, 2, 5, 3, 7, 8, 9$ is another.

Topological order requires a DAG

A directed graph has a topological ordering if and only if it is acyclic. On a graph containing a cycle, Kahn's algorithm outputs only the vertices that are never blocked, and the shortfall $|V| - \#\text{outputs}$ is exactly the set of vertices lying on or downstream of a cycle.